

Univ.-Prof. Dr. Ulrike Baumöl
Dipl.-Ök. Sarah Hackstein
Alexander Kornrumpf, M. Sc.

32711

Business Intelligence

Einheit 2
Methoden und Instrumente der Business Intelligence

Fakultät für
**Wirtschafts-
wissenschaft**

Übersicht zum Modul 32711

Business Intelligence

Einheit 1: Grundlagen der Business Intelligence

Einheit 2: Methoden und Instrumente der Business Intelligence

Einheit 3: Intelligente Datenhaltung und -bereitstellung

Einheit 4: Neue Entwicklungen und Anwendungsbeispiele der Business Intelligence

Das Werk ist urheberrechtlich geschützt. Die dadurch begründeten Rechte, insbesondere das Recht der Vervielfältigung und Verbreitung sowie der Übersetzung und des Nachdrucks, verbleiben bei der FernUniversität. Das Werk darf gemäß § 53 UrhG im privaten Rahmen zum Beispiel durch Kopien oder digitale Speicherung vervielfältigt werden.

Inhaltsverzeichnis

Abbildungsverzeichnis	II
Tabellenverzeichnis.....	III
Abkürzungs- und Symbolverzeichnis.....	IV
Liste der Algorithmen	VII
1 Einführung	1
1.1 Anwendungsfelder von Knowledge Discovery in Databases.....	3
1.2 Inhalte und Lernziele der Einheit „Methoden und Instrumente der Business Intelligence“	5
2 Knowledge Discovery in Databases	7
2.1 Definition, Ziele und Aufgaben	7
2.2 KDD-Prozesse in der Literatur	8
2.3 Einordnung von KDD in den Managementkreislauf	11
3 Phase I: Vorbereitende Schritte	15
3.1 Problemdefinition	15
3.2 Auswahl der Daten	21
3.3 Bereinigung und Aufbereitung der Daten.....	24
3.4 Projektion und Reduktion von Attributen	32
4 Phase II: Data Mining.....	40
4.1 Auswahl der Aufgabe	40
4.2 Auswahl des Algorithmus	43
4.3 Entscheidungsbaumverfahren	54
4.4 Regressionsanalyse.....	62
4.5 Clusteranalyse.....	65
4.6 Apriori-Algorithmus	75
4.7 Künstliche Neuronale Netze	79
4.8 Weitere Klassifizierungsverfahren.....	81
5 Phase III: Nachbereitende Schritte	84
5.1 Evaluation des Modells und der Ergebnisse	84
5.2 Interpretation und Verwendung der Ergebnisse.....	86
6 Zusammenfassung.....	88
Literaturverzeichnis.....	91

Abbildungsverzeichnis

Abb. 1:	Logarithmische Darstellung der Informationskapazität	2
Abb. 2:	Exponentielle Regression der Informationskapazität	2
Abb. 3:	Der KDD-Prozess als Spezialfall des Intelligence Cycle	13
Abb. 4:	Wechselwirkung des KDD-Prozesses im Unternehmen	14
Abb. 5:	Führungstätigkeiten in der Unternehmung	17
Abb. 6:	Drei Entstehungsmodelle für ein Konzept	25
Abb. 7:	Gauß'scher Noise	29
Abb. 8:	Lineare Regression eines nicht-linearen Zusammenhangs	47
Abb. 9:	Graphische Darstellung der linearen Diskriminanzanalyse.....	48
Abb. 10:	Beispielhafter Entscheidungsbaum.....	49
Abb. 11:	Beispiel für Clusteranalyse	50
Abb. 12:	Soft-Margin Support Vektor Maschine	51
Abb. 13:	Beispiel für Partitionierung durch einen Entscheidungsbaum.....	55
Abb. 14:	Klassifikationsbaum nach Beendigung des Beispielfahrens.....	60
Abb. 15:	Beispiel für lineare Regression.....	64
Abb. 16:	Vergleich zwischen agglomerativem und divisivem Clustering	67
Abb. 17:	Übersicht über Typen von Clusteringverfahren.....	67
Abb. 18:	Beispiel für ein Dendrogramm	72
Abb. 19:	Das Perceptron als idealisiertes Neuron.	80

Tabellenverzeichnis

Tab. 1:	Entwicklung der weltweiten Informationskapazität	1
Tab. 2:	Überblick und Vergleich verschiedener KDD-Prozesse.....	12
Tab. 3:	Zuordnung der Fehlerarten zu Fehlerklassen	30
Tab. 4:	Beispiel für Eins-aus-N Kodierung.....	35
Tab. 5:	Beispiel für n-aus-N Kodierung.....	36
Tab. 6:	Beispiel für Dummy Kodierung.....	36
Tab. 7:	Beispiel für Effekt Kodierung.....	37
Tab. 8:	Beispiel für unäre Kodierung.....	37
Tab. 9:	Übersicht zu Regressionsverfahren	47
Tab. 10:	Übersicht zu Projektionsverfahren	48
Tab. 11:	Übersicht zu Entscheidungsbaum-Verfahren	49
Tab. 12:	Übersicht zu Clusteringverfahren	51
Tab. 13:	Übersicht zu Soft-Computing.....	52
Tab. 14:	Übersicht zu Rule Induction	52
Tab. 15:	Eager Learning, Lazy Learning und Ensemble Learning	53
Tab. 16:	Beispieldaten für das Entscheidungsbaumverfahren	58
Tab. 17:	Distanzberechnung bei agglomerativem Clustering.....	70
Tab. 18:	Kundenmerkmale als Beispieldaten für hierarchisches Clustering.....	70
Tab. 19:	Beispieldaten für den Apriori-Algorithmus.....	78
Tab. 20:	Liste der im Beispiel gefundenen Regeln	79
Tab. 21:	Konfusionsmatrix.....	85

Abkürzungs- und Symbolverzeichnis

A	Alternativenmenge
D_i	Wertebereich der i-ten Entscheidungsvariable
E_i	Wertebereich der i-ten Umfeldvariable
G_i	Wertebereich der i-ten Zielvariable
I_{Tr}	Indexmenge der Trainingsinstanzen
K	Kantenmenge
M	Menge potentieller Nachrichten
N_A	Anzahl der Alternativen
N_D	Anzahl der Entscheidungsvariablen
N_E	Anzahl der Umfeldvariablen
N_G	Anzahl der Zielvariablen
N_Q	Anzahl der Attribute
N_X	Anzahl der Instanzen
N_Z	Anzahl der Zustände
Q	Menge der Attribute
R	Ergebnismenge
T	Knotenmenge
U	Universum
V_i	Wertebereich des i-ten Attributs
X	Konzept
X_{Tr}	Menge der Trainingsdaten
Z	Zustandsmenge
$\overline{x_q}$	Stichprobenmittel
η	Lernrate
D	Distanzmatrix
S	Struktur des Informationssystems
H	Modellklasse
(X)	Featureraum

μ	Erwartungswert
Ψ	Informationsfunktion
ρ	Ergebnisfunktion
σ	Standardabweichung
θ	Modellparameter
dist	Distanzfunktion
s^2	Stichprobenvarianz

BI	Business Intelligence
BIS	Betriebliches Informationssystem
BSC	Balanced Scorecard
CART	Classification and Regression Tree
CRM	Customer-Relationship-Management
DM	Data Mining
DQM	Datenqualitätsmanagement
DWH	Data Warehouse
EB	Entscheidungsbaum
EM	Expectation–Maximisation
ERM	Entity-Relationship-Modell
ES	Evolutionäre Strategie
FL	Fuzzy Logik
IJCAI	International Joint Conferences on Artificial Intelligence
IKT	Informations- und Kommunikationstechnik
IS	Informationssystem
KDD	Knowledge Discovery in Databases
KNN	künstliche neuronale Netze
LDA	Lineare Diskriminanzanalyse
MDP	Markov Decision Process
ML	maschinelles Lernen
o. B. d. A.	ohne Beschränkung der Allgemeinheit
OLAP	Online Analytical Processing
PCA	Hauptkomponentenanalyse
PDCA	plan – do – check – act
RFM	recency, frequency, monetary
SC	Soft-Computing
SVM	Support Vektor Maschine
TDIDT	Top-Down Induction of Decision Trees

Liste der Algorithmen

Algorithmus 1: Generischer TDIDT-Algorithmus.....	56
Algorithmus 2: Funktion zur Bestimmung des PurityGain.....	57
Algorithmus 3: Aufspaltung quantitativer Attribute am Mittelwert	58
Algorithmus 4: Gini-Index zur Attributauswahl	61
Algorithmus 5: Binäre Aufspaltung von Attributen	61
Algorithmus 6: InformationGain zur Attributauswahl.....	62
Algorithmus 7: Gemischte Aufspaltung von Attributen.....	62
Algorithmus 8: Clustering durch Varianzminimierung	74
Algorithmus 9: k-means Clustering.....	75
Algorithmus 10: Apriori-Algorithmus.....	77
Algorithmus 11: Perceptron-Algorithmus.....	80

Diese Seite bleibt aus technischen Gründen frei!

1 Einführung

In Einheit 1 dieses Lehrbriefs wurde bereits auf den übertriebenen Optimismus hingewiesen, den eine rein technische Sicht auf Business Intelligence (BI) begünstigt. Ebenso wurde die Herausforderung benannt, die ein gestiegenes Datenaufkommen für Unternehmen darstellt. Dabei ist das Problem alles andere als neu. Aus dem Jahr 1992 ist die folgende Aussage eines Managers überliefert:

Herausforderungen für die BI

„Computers have promised us a fountain of wisdom but delivered a flood of data.“ (Frawley et al., 1992).

Eine der grundlegenden Herausforderungen für die BI ist es daher, ganz im Sinne der in Einheit 1 vorgenommenen Differenzierung, aus Daten (bzw. plakativ einer Datenflut) Informationen und schließlich Wissen (plakativ sogar Weisheit) zu gewinnen. Um die Situation zu illustrieren, sei auf eine Studie von Hilbert und López (2011) verwiesen. Die Autoren vergleichen, methodisch außerordentlich gut fundiert, die Menge an weltweit theoretisch speicherbarer Information. Das Ergebnis ist in Tab. 1 dargestellt. Angegeben ist jeweils die weltweit insgesamt speicherbare Informationsmenge, d. h. die Menge von Information, die zum jeweiligen Zeitpunkt – mit der verfügbaren Technologie und unter Berücksichtigung von deren Verbreitung – maximal analog oder digital gespeichert werden konnte. Ein Exabyte entspricht 10^{18} Bytes bzw. einer Milliarde Gigabyte.

Die Informationskapazität wächst exponentiell

Tab. 1: Entwicklung der weltweiten Informationskapazität
Quelle: (Hilbert & López, 2011)

Jahr	Information [Exabyte]
1986	2,6
1993	15,8
2000	54,5
2007	295

Die logarithmische Darstellung der Informationskapazität, welche die Autoren wählen (Abb. 1), suggeriert, dass die häufig getroffene Annahme, auch das Datenvolumen würde exponentiell wachsen, zutreffend ist, auch wenn das tatsächliche Gesamtvolumen nicht sinnvoll messbar ist.

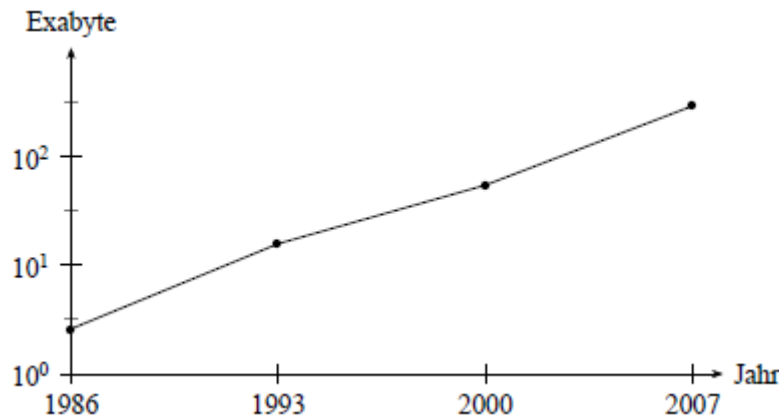


Abb. 1: Logarithmische Darstellung der weltweiten Informationskapazität
Quelle: in Anlehnung an (Hilbert & López, 2011)

Abb. 2 zeigt eine exponentielle Regression der von Hilbert und López (2011) ermittelten Daten. Demgemäß ist von einer Verdopplung der globalen Informationskapazität innerhalb von ca. 36 Monaten auszugehen, wobei dieses Wachstum über den Verlauf von fast 30 Jahren zu beobachten war. Ältere Schätzungen gehen von einer Verdopplung des Datenvolumens innerhalb von ca. 20 Monaten aus (Frawley et al., 1992). Unabhängig davon, wie schnell das Wachstum genau ist, erklärt schon alleine seine exponentielle Natur, warum das Problem 1992 als genauso drängend empfunden wurde wie 2011. Exponentielles Wachstum bedeutet nämlich vor allem, dass die hinzukommende Datenmenge relativ zur bereits vorhandenen Datenmenge nie kleiner wird.

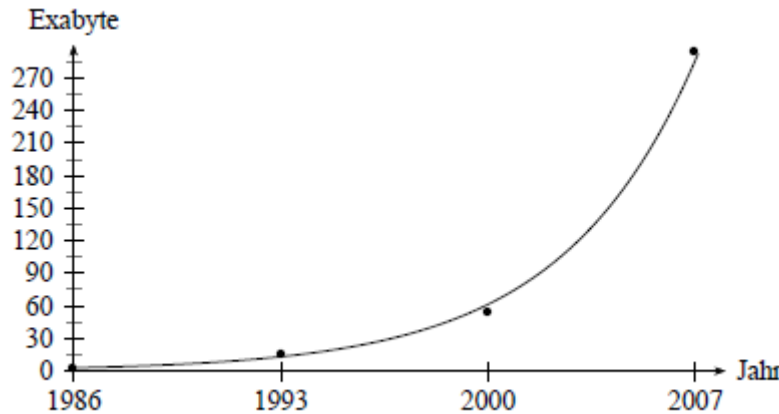


Abb. 2: Exponentielle Regression der Informationskapazität

Das Datenvolumen wächst in mehr als einer Dimension

Aufgrund der technischen Weiterentwicklungen haben Unternehmen die Möglichkeit, mehr Daten zu sammeln als je zuvor. Das gestiegene Datenvolumen resultiert nicht nur aus der Anzahl der Datensätze, sondern auch aus dem Anstieg der gesammelten Dimensionen je Datensatz. So können Unternehmen beispielsweise über ihre Kunden nichtmehr nur den Namen und Wohnort erfassen, sondern vielmehr auch deren Kaufgewohnheiten speichern, analysieren und für erforderliche Entscheidungen nutzen. Diese Erhöhung der Dimensionalität führt gleichzeitig auch zu einer Erhöhung der Komplexität von Entscheidungsproblemen. Der mögliche Suchraum vergrößert sich, da nun mehr Lösungsalternativen zur Verfügung stehen, die bewertet werden müssen. Ein weiteres Phänomen, das zum Anstieg des gesammelten Datenvolumens führt, ist die häufige Veränderung der Daten. In der Vergangenheit gefundene Muster sind möglicherweise bereits nach einem kurzen Zeitraum nicht mehr

gültig und müssen erneut auf ihre Gültigkeit hin überprüft werden (Fayyad, Piatetsky-Shapiro & Smyth, 1996).

Wie schon in Einheit 1 soll auch in der vorliegenden Einheit 2 eine vornehmlich prozessorientierte Sicht auf die BI verfolgt werden. Gemäß den vorstehenden Überlegungen und der in Einheit 1 vorgestellten Definition für BI ist also ein Prozess gefragt, der aus einer Menge von Daten wettbewerbsrelevante Information extrahiert und die Generierung von Wissen unterstützt. Darüber hinaus müssen die Daten so aufbereitet werden, dass aus ihnen gewonnenes Wissen zur Entscheidungsunterstützung herangezogen werden kann. Um dies zu erreichen, können Daten mittels verschiedener Methoden und Instrumente der BI aufbereitet und analysiert werden. Die Anwendung der Methoden und Instrumente führt aber nur dann zum gewünschten Ergebnis, wenn die Daten vorab geeignet aufbereitet und selektiert wurden. Der Prozess, der den Weg von den Rohdaten bis hin zum Wissen beschreibt, wird als Knowledge Discovery in Databases (KDD)-Prozess bezeichnet.

Ein Prozess zur
Extraktion
wettbewerbs-
relevanter
Information

Der zentrale Schritt im KDD-Prozess ist das Data Mining (DM). DM umfasst im Wesentlichen die Mustersuche und -erkennung innerhalb der vorbereiteten Daten. Hierzu werden, je nach Zielsetzung des Entscheiders, verschiedene Methoden und Instrumente angewendet, um die Daten hinsichtlich der formulierten Zielsetzung geeignet interpretieren zu können. Der KDD-Prozess und speziell DM finden in der Praxis in vielen Bereichen, in denen die Behandlung von sehr großen Datenmengen erforderlich ist, Anwendung. Im Folgenden werden, zum Einstieg in das Thema, einige Anwendungsbeispiele aufgegriffen und mögliche Fragestellungen sowie Lösungsmöglichkeiten erläutert.

Rolle des Data
Mining im KDD-
Prozess

1.1 Anwendungsfelder von Knowledge Discovery in Databases

Der Bereich des „Database Marketing“, insbesondere als Methode des Customer-Relationship-Management (CRM), nimmt für viele Unternehmen eine wachsende Bedeutung ein. Database Marketing umfasst vor allem die datenbankgestützte Zielgruppenauswahl sowie Erfolgsbewertung bei Direktwerbeaktionen. Eine Zielsetzung könnte hierbei beispielsweise eine gezielte und personalisierte Kundenansprache sein. Dazu können die Kunden mittels Clusteranalyse (vgl. Abschnitt 4.5) in Segmente unterteilt werden, um dann für jedes Segment individuelle Ansprachen zu entwerfen. Auch Aufgaben wie die Analyse von Cross-Selling-Produkten können interessant sein, ein einfaches Verfahren hierzu ist z. B. die Warenkorbanalyse (vgl. Abschnitt 4.6) (Petersohn, 2005, S. 14).

Database Marketing

Ein althergebrachter Ansatz im Marketing ist die Segmentierung von Kunden nach den Kennzahlen recency, frequency, monetary (RFM), die sogenannte RFM-Analyse. Dabei bezeichnet, jeweils bezogen auf einen bestimmten Kunden, recency die Zeit seit der letzten Transaktion, frequency die Häufigkeit von Transaktionen mit diesem Kunden und monetary den generierten Umsatz. Trotz aller praktischen Erfolge muss jedoch konstatiert werden, dass es sich um eine willkürliche Beschränkung auf drei Kennzahlen handelt. Darüber hinaus ist Segmentierung nicht für alle Anwendungsfälle der richtige Ansatz. Um z. B. zu prognostizieren, ob das Anschreiben eines bestimmten Kunden eine Reaktion hervorrufen wird, ist Klassifizierung (vgl. Abschnitt 4.3) häufig der bessere Ansatz. Generell gilt, dass moderne DM-Verfahren nicht

Beispiel: RFM-
Analyse

auf bestimmte Kennzahlen wie RFM festgelegt sind. Sie können z. B. zusätzlich demografische Eigenschaften der Kunden berücksichtigen. Dies ist ggf. ein Vorteil gegenüber einfacheren, traditionellen Ansätzen der Datenanalyse (McCarty & Hastak, 2007).

Beispiel: Cross-Selling

Ein weiterer, lange bekannter Ansatz des CRM ist das Cross-Selling. Als solches wird, vereinfacht ausgedrückt, der Verkauf von mehr als einem Produkt oder einer Dienstleistung an denselben Kunden bezeichnet. Typischerweise haben Cross-Selling Verkäufe an Bestandskunden geringere Akquisitionskosten als Verkäufe an Neukunden. Des weiteren erhöht jeder zusätzliche Verkauf die Anzahl der Berührungspunkte zwischen Kunde und Unternehmen und somit die Wechselkosten für den Kunden und auch die Kundenbindung. Aufgabe des KDD ist es dabei, aus einem Datenbestand über Produkte und Dienstleistungen, die von einem Kunden in der Vergangenheit gekauft wurden, auf solche Produkte und Dienstleistungen zu schließen, die der Kunde künftig interessant finden könnte (Kamakura, Wedel, Rosa & Mazzon, 2003).

Erstellen einer Balanced Scorecard

Das Konzept einer Balanced Scorecard (BSC) wurde bereits an anderer Stelle ausführlich diskutiert (vgl. Modul Informationsmanagement, Einheit 3), sodass hier nur auf die Anwendung von KDD bei der Erstellung einer solchen BSC eingegangen werden soll. DM-Verfahren können dabei als Analyseverfahren verwendet werden, um Werte für die Kennzahlen zu ermitteln, welche in der BSC erfasst werden. So können neue Zusammenhänge aufgedeckt werden oder aber Maßnahmen zur Erreichung der Ziele in den vier Perspektiven der BSC (Finanzperspektive, Kundenperspektive, Prozessperspektive und Potentialperspektive) abgeleitet werden. (Pietsch & Memmler, 2003, S. 137).

Perspektiven einer BSC

So ist z. B. in der Finanzperspektive eine Möglichkeit der Anwendung von DM die Analyse der signifikanten Abweichungen wichtiger Finanzkennzahlen von Normwerten. So können Abweichungen früh erkannt und Maßnahmen eingeleitet werden (Pietsch & Memmler, 2003, S. 123). Ein mögliches Ziel in der Kundenperspektive ist es, wie im Database Marketing auch, die Kunden, aber auch den Markt, zu segmentieren (Pietsch & Memmler, 2003, S. 124). In der Prozessperspektive ist ein Einsatz z. B. im Bereich der Betriebsprozesse möglich. Mögliche Fragestellungen umfassen u. a. die Identifikation zuverlässiger Lieferanten, die Analyse von Fehlern und Qualitätsproblemen aller Art, die Identifikation von Engpässen und die Prognose von Absatzmengen (Pietsch & Memmler, 2003, S. 128). In der Potentialperspektive kann der Fokus auf verschiedene Bereiche gelegt werden. Eine Möglichkeit ist das Personalpotential. So kann beispielsweise eine Analyse der Verwendung unternehmensinterner Informationssysteme (ISe) durch die Mitarbeiter hilfreich sein, um das Kommunikationsverhalten zu verbessern (Pietsch & Memmler, 2003, S. 129).

Einsatz im Finanzsektor

Ein weiterer Bereich, in dem verstärkt DM-Methoden eingesetzt werden, ist der Finanzsektor. Es werden hier nur Beispiele genannt. So kann es interessant sein, die zurückgezahlten Kredite oder die in Verzug geratenen Raten des letzten Monats, Quartals oder Jahres zu analysieren, um daraus Prognosen für einen Zeitraum in der Zukunft abzuleiten. Der Fokus kann aber z. B. auch auf die zurückgezahlten bzw. nicht zurückgezahlten Raten gelegt werden. So könnte mittels DM untersucht werden, ob die Kombination verschiedener Attributausprägungen bei einzelnen Kunden häufig zu einer Verzögerung der Ratenzahlungen führt. Eine weitere spezifische Anwendung ist die automatisierte Erkennung von Verhaltensmustern, bei denen der Verdacht auf Betrug oder Geldwäsche besteht (Han & Kamber, 2006, S. 649f.).

Aufgrund einer stark gewachsenen Produktpalette ist das Datenvolumen auch im Telekommunikationssektor stetig gestiegen. Telekommunikationsanbieter bieten nicht mehr nur einfache Telefonverträge an, sondern ergänzen dieses

**Einsatz im
Telekommunikations-
sektor**

Angebot um weitere Leistungen, wie z. B. Internet- oder Mobilfunkverträge. Auch die Wettbewerbssituation hat sich verändert: Es sind viele Anbieter auf dem Markt, somit verstärkt sich auch der Wettbewerbsdruck. Daher sollten die Anbieter die Wünsche ihrer Kunden genau kennen und durch entsprechende Leistungen zu erfüllen versuchen. Auch hierzu kann KDD einen Beitrag leisten (Han & Kamber, 2006, S. 652f.). Viele Unternehmen kennen bereits den Mehrwert, den sie durch individuelle Angebote generieren. So bieten viele Unternehmen unterschiedliche Vertragstypen an, sodass sie viele verschiedene „Kundenklassen“ vom „Vieltelefonierer“ (Flatrate), über den „Viel-SMS-Schreiber“ (Frei-SMS) bis hin zum „Wenig-Nutzer“ (Prepaid) abdecken.

1.2 Inhalte und Lernziele der Einheit „Methoden und Instrumente der Business Intelligence“

In der Einheit 2 wird ein ganzheitliches Begriffsverständnis von BI, wie es bereits in Einheit 1 dieses Lehrbriefs eingeführt wurde, zu Grunde gelegt. Der Aufbau der vorliegenden Einheit orientiert sich dabei am KDD-Prozess. Dabei wird nicht nur eine technische Sichtweise, sondern auch eine betriebswirtschaftliche Sichtweise eingenommen. Zu Beginn der Einheit wird in Kapitel 2 der Begriff KDD definiert und die Ziele und Aufgaben des KDD-Prozesses erläutert. Abschließend erfolgt eine Einordnung von KDD in den Managementkreislauf.

Begriffsverständnis

Der eigentliche KDD-Prozess lässt sich grob in vorbereitende Schritte, DM und nachbereitende Schritte einteilen. Nur wenn der gesamte KDD-Prozess mit diesen drei Phasen durchlaufen wird, kann auch eine erfolgreiche Entscheidungsunterstützung gewährleistet werden. Daher werden die einzelnen Phasen mit ihren Aufgaben und Zielen in jeweils einem eigenen Kapitel ausführlich betrachtet. Es werden z. B. Ansätze und Vorgehensmodelle aufgezeigt, die in den einzelnen Phasen des Prozesses genutzt werden können, um die jeweiligen Ziele der Phase zu erreichen.

KDD-Prozess

Ein wesentlicher Schritt im KDD-Prozess ist das DM. Die wichtigsten Methoden und Verfahren des DM sollen daher in Kapitel 4 erläutert werden. Es wird gezeigt, welche Methoden bei welcher Art von Zielsetzung am besten angewendet und wie die dadurch generierten Ergebnisse interpretiert werden können.

Data Mining

Die *Lernziele* dieser Einheit lassen sich wie folgt zusammenfassen:

Kapitel 2: Knowledge Discovery in Databases

- Sie können erklären, was unter dem Begriff KDD verstanden wird und was die wesentlichen Ziele und Aufgaben sind.
- Sie verstehen, welche verschiedenen Definitionen von KDD-Prozessen in der Literatur zu finden sind und können diese voneinander abgrenzen.
- Sie können den KDD-Prozess in den Managementkreislauf einordnen und kennen die Bedeutung dieser Einordnung.

Kapitel 3: Vorbereitende Schritte

- Sie kennen die Teilschritte zur Vorbereitung im KDD-Prozess und verstehen, welche Bedeutung, Ziele und Aufgaben diese Teilschritte haben.
- Sie kennen verschiedene Fehlerarten, welche die Datenqualität vermindern können.
- Sie kennen Methoden zur Auswahl und Transformation der Daten.

Kapitel 4: Data Mining

- Sie wissen, was unter dem Begriff DM zu verstehen ist und können die Ziele und Aufgaben, die damit einhergehen, erklären.
- Sie kennen die wichtigsten Methoden des DM und können ausgewählte Methoden am Beispiel anwenden.
- Sie können Stärken und Schwächen der einzelnen Methoden einschätzen.

Kapitel 5: Nachbereitende Schritte

- Sie sind in der Lage, das Ergebnis des KDD-Prozesses zu interpretieren und zu evaluieren.

Kapitel 6: Zusammenfassung

- Sie erhalten noch einmal einen kurzen Überblick über die behandelten Themen und deren Zusammenhänge.

2 Knowledge Discovery in Databases

In diesem Kapitel sollen der Begriff des KDD und die damit zusammenhängenden Prozesse definiert, erörtert und in einen größeren Kontext eingeordnet werden. Die genauen historischen Anfänge eines wissenschaftlichen Fachgebiets sind oft nur bedingt zu bestimmen, da sie durch eine große begriffliche Diversität geprägt sind, bis sich bestimmte Begriffe durchsetzen. Aus heutiger Sicht lässt sich festhalten, dass die wesentlichen Grundsteine für modernes KDD und DM in den 1980er Jahren gelegt wurden (Frawley et al., 1992) und KDD während der 1990er Jahre zu einer eigenständigen Disziplin weiterentwickelt wurde (Fayyad et al., 1996, S. 3). Als „Geburtsstunde“ des KDD-Begriffs als solchem kann der erste KDD-Workshop im Rahmen International Joint Conferences on Artificial Intelligence (IJCAI) 1989 gelten (Piatetsky-Shapiro, 1991). Bemerkenswert ist jedoch auch, dass einige Verfahren, die heute im Bereich des DM als Teilschritt des KDD eingesetzt werden, wie z. B. künstliche neuronale Netze (KNN) (vgl. Abschnitt 4.7) und Entscheidungsbaum (EB)-Verfahren (vgl. Abschnitt 4.3), dem KDD-Begriff historisch vorausgehen (Hastie, Tibshirani & Friedman, 2001).

Historische Rückschau

2.1 Definition, Ziele und Aufgaben

In Anlehnung an Frawley et al. (1992) definieren Fayyad et al. (1996) KDD wie folgt:

Definition von KDD

„*Knowledge Discovery in Databases* is the non-trivial process of identifying valid, novel, potentially useful, and ultimately understandable patterns in data.“ (Fayyad et al., 1996, S. 6).

Diese Definition wird häufig zitiert und kann weitestgehend als unstrittig angesehen werden. Weitere stark daran angelehnte Definitionen finden sich etwa bei Cios, Pedrycz und Swiniarski (1998, S. 2) sowie bei Ester und Sander (2000, S. 1).

Strittiger als die Definition von KDD ist deren Abgrenzung vom Begriff des DM. Fayyad et al. (1996) vertreten den auch in der Einleitung (Kapitel 1) und im weiteren Verlauf dieser Einheit eingenommenen Standpunkt, dass der Begriff DM Methoden und Instrumente beschreibt, welche in einem Teil des übergreifenden KDD-Prozesses eingesetzt werden. Dabei bezeichnet KDD eine höhere und abstraktere Ebene der Betrachtung, die von den eingesetzten Methoden weitgehend unabhängig ist. Die Autoren weisen jedoch auch darauf hin, dass der Begriff des DM traditionell eher in der Statistik und der Wirtschaftsinformatik beheimatet ist, während KDD seine Wurzeln im Bereich der künstlichen Intelligenz und des maschinellen Lernens hat (Fayyad et al., 1996, S. 5f.).

Abgrenzung vom DM

Andere Autoren verzichten auf die Einführung eines expliziten KDD-Begriffs oder verwenden die Begriffe KDD und DM synonym. So schreiben beispielsweise Wiedmann, Buckler und Buxel (2001):

Alternative Sichtweisen

„KDD und DM sind im betriebswirtschaftlichen Kontext Synonyme.“ (Wiedmann et al., 2001, S. 17).

Es ist nicht von der Hand zu weisen, dass dieses Begriffsverständnis heute viele Anhänger hat, es findet sich z. B. implizit bei Linoff und Berry (2011). Auch dieses Verständnis kommt jedoch nicht

ohne einen übergreifenden Prozessbegriff aus, der dann ebenfalls DM genannt wird. Somit fehlt ein Begriff für den zentralen DM-Schritt im übergreifenden Prozess, den Wiedmann et al. (2001) dann als „DM i. e. S.“ bezeichnen. Ein solcherart überladener DM-Begriff birgt die Gefahr von Missverständnissen, daher soll in diesem Lehrbrief die Trennung der Begriffe KDD und DM beibehalten werden und für KDD die eingangs genannte Definition gelten.

Interpretation der KDD-Definition

Zum besseren Verständnis sei hier eine Interpretation der KDD-Definition nach Düsing (2006) dargestellt. Ein *nicht-trivialer* Prozess zeichnet sich demnach dadurch aus, dass Zusammenhänge in den Daten ermittelt werden, die über die reine Wiedergabe oder Aggregation der einzelnen Datenreihen hinausgehen. Es wird vielmehr ein, mehr oder weniger komplexes, Modell entwickelt, das die vorhandenen Zusammenhänge abbildet. Die Eigenschaften *valide, neuartig, potentiell nützlich und leicht verständlich* beziehen sich auf die Güte der ermittelten Zusammenhänge. Die Zusammenhänge haben dabei die Form von Mustern, d. h. Regelmäßigkeiten oder Auffälligkeiten in den Daten, die durch ein Modell dargestellt werden können und ihrerseits eine Hypothese über die Domäne des Datenbestandes bilden (Düsing, 2006).

Ziele von KDD

Das unmittelbare Ziel des KDD-Prozesses, *Muster in Datensätzen* zu erkennen und explizit zu machen, ordnet sich den Wissenszielen des Unternehmens unter (Düsing, 2006). Wie für die gesamte BI gilt also auch hier, dass die durch KDD gewonnene Information vom Entscheider um Erfahrungen und Anwendungszusammenhänge angereichert werden muss, um somit in den Status von Wissen erhoben zu werden. Dieses Wissen kann dann Handlungsgrundlage für die Lösung eines Entscheidungsproblems sein.

Grundaufgaben von KDD

Zur Erreichung dieser Ziele identifiziert Düsing (2006) vier Grundaufgaben. Bei der *Bildung von Gruppen* werden Datensätze, welche aufgrund ihrer Attribute ähnlich sind, zu einer Gruppe zusammengefasst. Unter den in Abschnitt 1.1 beispielhaft genannten Anwendungsfeldern fällt die Clusteranalyse (vgl. Abschnitt 4.5) in diese Kategorie. Bei der *Erklärung von Gruppen* dagegen ist bereits bekannt, zu welcher Gruppe ein gegebener Datensatz gehört. Gesucht ist eine Erklärung der Zugehörigkeit in Form eines funktionalen Zusammenhangs zwischen den Attributen des Datensatzes und seiner Zuordnung zu einer Gruppe. Dies ist das Anwendungsfeld der Klassifizierungsverfahren (vgl. Abschnitte 4.3 und 4.8). Während es sich bei der Gruppenzugehörigkeit um ein diskretes Attribut handelt, können auch Funktionszusammenhänge kontinuierlicher Attribute untersucht werden. Dabei handelt es sich um die Aufgabe der *Erklärung von Zusammenhängen*. Ein typisches Anwendungsfeld ist die Regressionsanalyse (vgl. Abschnitt 4.4). Der Kreis der Grundaufgaben schließt sich mit der *Beschreibung von Zusammenhängen* zwischen den einzelnen Attributen der Datensätze. Die Assoziationsanalyse untersucht, welche Werte eines Attributs häufig mit welchen Werten eines anderen Attributs gemeinsam auftreten (vgl. Abschnitt 4.6).

2.2 KDD-Prozesse in der Literatur

Auch diesem Abschnitt liegt die Annahme zugrunde, dass DM ein Teil im übergreifenden KDD-Prozess ist. Der KDD-Prozess in der dargestellten Form ist, im Gegensatz zu einem konkreten DM-Algorithmus, kein Prozess, der vollständig automatisiert ablaufen kann. Vielmehr ist in jedem Schritt menschlicher Input erforderlich. Die Rolle der beteiligten Person soll hier als Analyst bezeichnet werden, wobei der

**Der KDD-Prozess ist
wissensintensiv und
analystenzentriert**

Analyst im Einzelfall mit dem Entscheider identisch sein kann. Auch wenn es kontra-intuitiv erscheinen mag, ist KDD, also Wissensentdeckung, nicht ohne Vorwissen möglich. Insbesondere kann keiner der Schritte ohne Annahmen und Wissen über die Domäne durchgeführt werden. Diesen Sachverhalt bringen Brachman und Anand (1996) auf den Punkt:

„knowledge discovery is a *knowledge-intensive* task consisting of *complex interactions, protracted over time*, between a *human* and a (large) database, possibly supported by a *heterogeneous suite of tools*“ (Brachman & Anand, 1996, Hervorhebungen im Original).

Geradezu prototypisch ist der KDD-Prozess nach Fayyad et al. (1996). Dieser Prozess umfasst neun Schritte, welche im Folgenden aufgelistet werden sollen (Fayyad et al., 1996):

KDD-Prozess nach
Fayyad et al. (1996)

- *Problemdefinition*: Problemstellung und Ziele sowie die Domäne des KDD Projekts werden definiert.
- *Auswahl der Daten*: Aus der Gesamtheit der vorhandenen Daten werden diejenigen ausgewählt, die analysiert und zur Lösung der zuvor definierten Problemstellung herangezogen werden sollen. Beispielsweise kann es sein, dass ein Unternehmen die Umsatzzahlen aus dem letztjährigen Sommer heranziehen möchte, um die zukünftigen Umsatzzahlen im nächsten Sommer zu prognostizieren. Diesem Vorgehen liegen offensichtlich Annahmen des Analysten über die Zusammenhänge zwischen vergangenen und zukünftigen Zahlen zugrunde. Diese Annahmen an sich können nicht durch KDD verifiziert werden.
- *Bereinigung und Aufbereitung der Daten*: Behandlung von Datenqualitätsproblemen wie z.B. verrauschten oder fehlenden Werten, Ausreißern und dergleichen.
- *Reduktion oder Projektion*: Einzelne Attribute können verworfen oder miteinander verrechnet werden. Daten können in eine reduzierte oder inhaltlich äquivalente Form überführt werden, die für die Problemstellung geeigneter ist. Eine mögliche sehr einfache Transformation könnte so aussehen, dass den Eigenschaften „männlich“ und „weiblich“ zur weiteren Verarbeitung binäre Werte 0 (männlich) und 1 (weiblich) zugeteilt werden. Auch hier benötigt der Analyst Vorwissen, um beurteilen zu können, wie sich die Transformation später auf verschiedene DM-Methoden auswirken wird.
- *Auswahl der Aufgabe*: Aus den Grundaufgaben des KDD (vgl. S. 8) wird diejenige ausgewählt, welche für Fragestellung und Zielsetzung geeignet ist.
- *Auswahl des Algorithmus*: Es werden ein oder mehrere DM-Methoden bzw. Algorithmen ausgewählt, welche die definierte Aufgabe sinnvoll erfüllen können.
- *Data Mining*: Dies ist der zentrale Schritt, in dem gezielt nach Mustern in den ausgewählten Daten gesucht wird.

- *Interpretation:* Die gefundenen Muster werden mit Blick auf Fragestellung und Zielsetzung evaluiert und interpretiert. Gegebenenfalls muss zu einem der vorherigen Schritte zurückgekehrt werden, sodass sich Schleifen ergeben können.
- *Verwendung:* Das erworbene Wissen wird mit vorhandenem Wissen abgeglichen, dokumentiert, verbreitet, visualisiert, zur Entscheidungsfindung eingesetzt etc.

Bezug zu weiteren KDD-Prozessen

Der vorgestellte neunschrittige Prozess nach Fayyad et al. (1996) ist im Wesentlichen kompatibel zu anderen KDD-Prozessen, die in der Literatur beschrieben werden. Je nach Fokus der Darstellung werden einzelne Schritte zusammengefasst oder weiter aufgeschlüsselt. Fayyad et al. (1996) selbst nehmen z. B. eine Einteilung in fünf Phasen vor, die sich ganz ähnlich auch bei Ester und Sander (2000) findet. Hier aufgelistet jeweils mit der Entsprechung im neunschrittigen KDD-Prozess:

1. *Fokussieren: Schritte 1 & 2*
2. *Vorverarbeitung: Schritt 3, im erweiterten Sinne jedoch auch die Anbindung an ein Data Warehouse (DWH) und in dem Zusammenhang auch die Integration verschiedener Datenquellen (vgl. Einheit 3).*
3. *Transformation: Schritt 4*
4. *DM: Schritte 5–7*
5. *Evaluation: Schritte 8 & 9*

KDD-Prozess nach Düsing (2006)

Düsing (2006) wählt für den KDD-Prozess eine Einteilung in fünf Phasen, welche einen anderen Fokus setzt als die zuvor beschriebene, aber dennoch mit den Schritten nach Fayyad et al. (1996) konsistent ist. Dies soll nun in Anlehnung an Düsing (2006) dargestellt werden:

1. *Auswahl:* Diese Phase umfasst zunächst die Bestimmung der Ausgangslage und die Definition eines Wissensziels. Des Weiteren werden Daten ausgewählt, deren Analyse zur Erreichung dieses Ziels dienlich erscheint. Darüber hinaus wird sichergestellt, dass der angestrebten Nutzung der betreffenden Daten keine sonstigen, z. B. juristischen, Gründe entgegenstehen. Dies entspricht den Schritten 1 & 2 bei Fayyad et al. (1996).
2. *Aufbereitung:* Diese Phase zielt vor allem auf die Verbesserung der Datenqualität ab. Sie entspricht den Schritten 3 & 4 bzw. im weiteren Sinne den Phasen Vorverarbeitung und Transformation bei Fayyad et al. (1996).
3. *Festlegung:* Die Festlegung einer konkreten KDD-Aufgabe und damit verbunden eines Algorithmus, wird hier als eigene Phase verstanden. In der Regel ist damit auch das Eingabeformat der Daten und die Struktur der entdeckbaren Muster festgelegt. Dies entspricht den Schritten 5 & 6.

4. *Analyse*: Zu dieser Phase zählt Düsing (2006) nicht nur die Anwendung von DM-Verfahren i. e. S. (Schritt 7), sondern auch die manuelle Analyse und Hypothesenverifizierung, z. B. mittels Online Analytical Processing (OLAP). Im Kontext dieses Lehrbriefs ist die Diskussion von OLAP in Einheit 3 zu finden.
5. *Interpretation*: Zur Interpretation im engeren Sinne zählt die Ableitung von explizitem Wissen aus den Analyseergebnissen. Im weiteren Sinne umfasst sie zudem die Prüfung der Ergebnisse nach zielbezogenen Gütekriterien. Darüber hinaus umfasst diese Phase die Ableitung von Handlungsempfehlungen aus dem gewonnenen Wissen, auch wenn diese nicht immer eindeutig möglich ist. Dies entspricht somit den Schritten 8 & 9.

Linoff und Berry (2011) definieren den Begriff DM wesentlich weiter als nur als einen Schritt im KDD-Prozess. Nach ihrem Verständnis wird beim DM zunächst das zu lösende Problem definiert, bevor im nächsten Schritt, dem eigentlichen DM, die Daten in Information transformiert werden. Diese Information wird dann genutzt, z. B. um eine bestimmte Marketingaktion zu starten. Nach der Umsetzung der Information in eine Handlung, wird der Erfolg, in diesem Fall beispielsweise die Antwortrate, gemessen, um daraus ggf. neue Probleme abzuleiten, die es dann zu lösen gilt. Somit ist das hier gezeigte Verständnis des DM bzw. KDD-Prozesses zyklisch (Linoff & Berry, 2011, S. 15ff.).

DM-Prozess nach Linoff und Berry (2011)

Tab. 2 zeigt die verschiedenen zuvor diskutierten Sichtweisen im Überblick. Es zeigt sich, dass der Prozess der Wissensentdeckung immer ähnlich verläuft, unabhängig davon, ob die Autoren von KDD oder DM sprechen. Zum Zwecke der weiteren Gliederung dieser Einheit und um die zentrale Rolle des DM im KDD-Prozess zu betonen, lassen sich die einzelnen Schritte im ursprünglichen KDD-Prozess bzw. Phasen in den abgeleiteten Prozessen grob in drei Meta-Phasen gliedern:

Zusammenfassung

- I. Vorbereitende Schritte
- II. DM
- III. Nachbereitende Schritte

2.3 Einordnung von KDD in den Managementkreislauf

Abschließend ist zu klären, wie sich KDD in den Kontext des Unternehmens im Allgemeinen und der BI im Besonderen einordnen lässt. KDD ist hierbei kein Selbstzweck, sondern ordnet sich den Wissenszielen der Unternehmung bzw. des Entscheiders unter (vgl. S. 8). Im Rahmen der BI bezieht sich dieses Wissensziel auf ein konkretes Entscheidungsproblem bzw. einen Entscheidungsprozess (vgl. Einheit 1, Abschnitt 2.3). Daher bietet sich eine Verknüpfung des KDD-Prozess mit der handlungsorientierten Entscheidungsmodellierung (vgl. Einheit 1, Abschnitt 2.2.2) an. Prinzipiell können Wissensdefizite in jeder Phase des Entscheidungsprozesses auftreten und somit den Einsatz von KDD zu beliebigen Zeitpunkten im Entscheidungsprozess rechtfertigen¹. Idealisiert kann jedoch, dem DM-Prozess nach Linoff und

KDD im Entscheidungsprozess

¹ An dieser Stelle sei an die rekursive Struktur des plan – do – check – act (PDCA)-Kreislauf (Deming, 1982) erinnert.

Berry (2011, S. 15) folgend, angenommen werden, dass mit dem Entscheidungsproblem zugleich auch Wissensziele definiert werden. KDD wird damit zum integralen Bestandteil der Informationsbeschaffung im Rahmen der Entscheidungsvorbereitung, zu der auch die Alternativenbewertung zu zählen ist. Die Entscheidung mündet in eine Handlung, deren Wirkung wiederum analysiert werden muss. Aus der so entstehenden neuen Situation kann sich ein neues Entscheidungsproblem ergeben (vgl. S. 11).

Tab. 2: Überblick und Vergleich der einzelnen Phasen und Schritte verschiedener KDD-Prozesse

Meta-Phase	Phase			Einzelschritt
	(Ester & Sander, 2000)	(Düsing, 2006)	(Linoff & Berry, 2011)	(Fayyad et al., 1996)
I	Fokussieren	Auswahl	Problemdefinition	Problemdefinition
			Transformation (Daten→Wissen)	Auswahl der Daten
	Vorverarbeitung	Aufbereitung		Bereinigung und Aufbereitung der Daten
	Transformation			Reduktion oder Projektion
II	DM	Festlegung		Auswahl der Aufgabe
				Auswahl des Algorithmus
		Analyse		Data Mining
III	Evaluation	Interpretation		Interpretation
			Handlung	Verwendung

Unterstützende Funktion von KDD

Ausgehend von o. g. Betrachtungsweise zeigt sich, dass der KDD-Prozess nicht unabhängig zu sehen ist, sondern im Kontext des soeben beschriebenen, übergeordneten Managementprozesses. Letzterer besteht in einer stark aggregierten Form aus einem Kreislauf aus Entscheidungsproblem, Entscheidungsvorbereitung, Entscheidung und Wirkungsanalyse (vgl. (Linoff & Berry, 2011; Wilde, 2001)). Der KDD-Prozess hat in Bezug auf diesen Managementkreislauf eine unterstützende Funktion, wobei die Unterstützung vor allem die Entscheidungsvorbereitung (Linoff & Berry, 2011, S. 15) und die Wirkungsanalyse (Wilde, 2001) betrifft. Da das unmittelbare Ziel des KDD-Prozess Wissensgenerierung und nicht die eigentliche Entscheidung im engeren Sinne ist, kann der Prozess unmittelbar als Spezialfall des Intelligence Cycle (Nonaka & Takeuchi, 1995) aufgefasst werden (vgl. Einheit 1). Dies ist in Abb. 3 dargestellt.

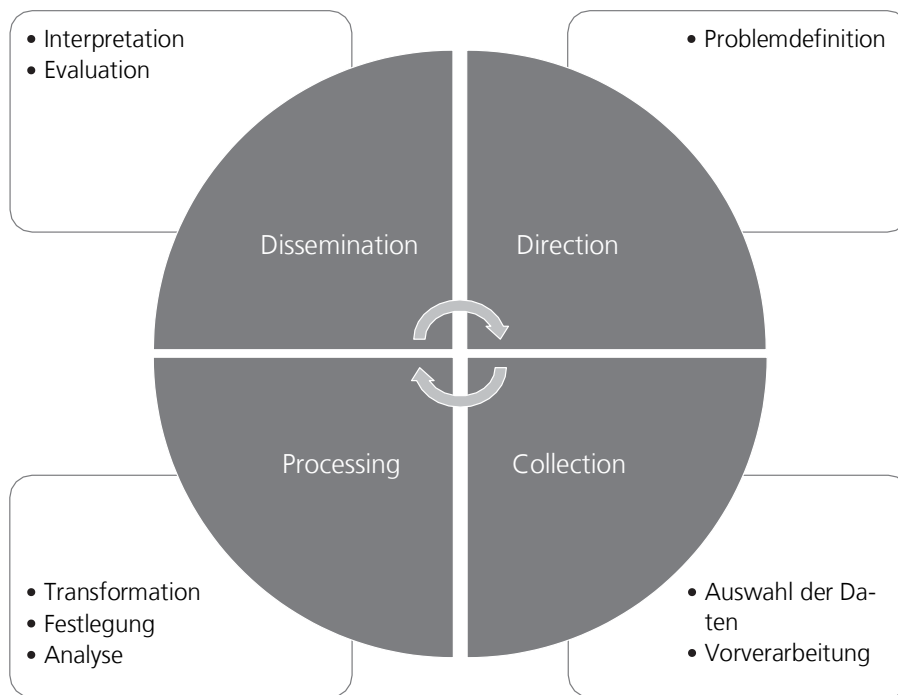


Abb. 3: Der KDD-Prozess als Spezialfall des Intelligence Cycle

Aus der Werkzeugunterstützung (vgl. S. 9) ergibt sich ein Bezug zwischen dem KDD-Prozess und der Informations- und Kommunikationstechnik (IKT). Neben DM-Werkzeugen i. e. S., welche vor allem in den Schritten 5–7 Anwendung finden, sind hier DWHs sowie OLAP zu nennen, welche Schwerpunkt der Einheit 3 dieses Lehrbriefs sind. Ein DWH unterstützt maßgeblich die Schritte 2 & 3 (vgl. S. 9), liefert aber auch Input für das OLAP (Wilde, 2001). OLAP seinerseits unterstützt vor allem die Schritte 3 & 8 (Wilde, 2001), kann aber auch schon in Schritt 7 zum Einsatz kommen (vgl. S. 9).

Unterstützung durch die IKT

Als Geschäftsprozess verstanden, steht der KDD-Prozess also in Wechselwirkung mit der Strategiebene und der IKT-Ebene des Unternehmens. Dabei stellen insbesondere die beiden Schritte Problemdefinition und Verwendung Schnittstellen zur Unternehmensstrategie dar. Abb. 4 zeigt vor diesem Hintergrund die Einordnung des KDD-Prozesses in das ganzheitliche Unternehmensmodell (vgl. Einheit 1, Abschnitt 3.1) unter besonderer Berücksichtigung des o. g. Managementkreislaufs.

Ganzheitliche Betrachtung

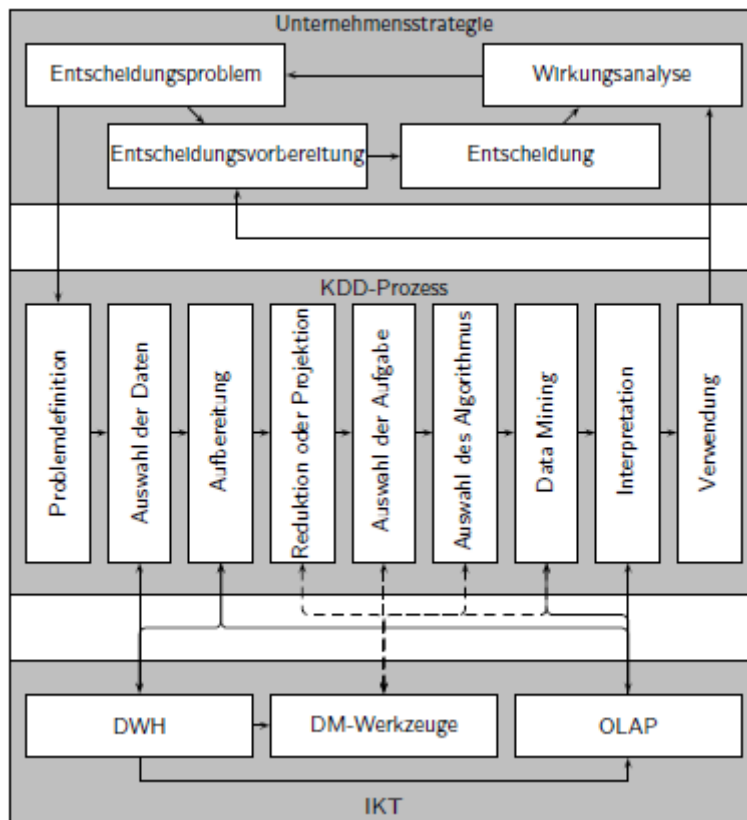


Abb. 4: Wechselwirkung des KDD-Prozesses mit Strategie und IKT im Unternehmen

Integration in andere Geschäftsprozesse

Auch innerhalb der Geschäftsprozessebene kann der KDD- Prozess nicht für sich allein stehen, sondern muss in das Unternehmen und weitere Prozesse eingebettet werden, damit gute Resultate entstehen können. Je nach Problemstellung sollte der KDD-Prozess dann beispielsweise in Marketing-, Vertriebs- oder Produktdesignprozesse integriert werden. Denn KDD sollte nicht nur die technische Aufbereitung von Daten zu Information sein, sondern vielmehr, im Sinne einer echten Wissensgenerierung, eine Hilfestellung zur Vorbereitung und Beurteilung von Entscheidungen bieten. Den KDD-Prozess alleine, ohne die Einbindung in die Prozesse im Unternehmen, zu betrachten, ist ineffizient (Linoff & Berry, 2011, S. 19 u. 25) Im Idealfall sollten alle Geschäftsprozesse in einem kontinuierlichen Lernprozess verbessert werden. Um zu lernen, ist jedoch Wissen erforderlich, das durch KDD generiert werden kann. Wenn beispielsweise KDD ergeben hat, dass Kunden weniger auf Werbung reagieren, wenn diese sehr häufig geschaltet wird, dann sollte der Prozess „Werbung schalten“ in seiner Frequenz reduziert werden oder möglicherweise andere Kanäle genutzt werden.

3 Phase I: Vorbereitende Schritte

Wie im vorherigen Kapitel 2 dargelegt, ist der Auslöser für den KDD-Prozess das Auftreten eines Entscheidungsproblems. In der Praxis wird zumeist dann ein Entscheidungsprozess angestoßen, wenn negative Symptome, z. B. ein Rückgang der Verkaufszahlen und des Gewinns, auftreten. Dann wird versucht, aus den Symptomen eine Problemstellung abzuleiten. Häufig besteht das Problem in der Entscheidung über Maßnahmen, mittels derer sich die aufgetretenen Symptome beheben lassen. Je nach Komplexität der Symptome muss daher zur genauen Definition des Problems zunächst ein Suchprozess erfolgen, mit dessen Hilfe mögliche Maßnahmen klar definiert werden können (Laux, Gillenkirch & Schenk-Mathes, 2012, S. 12f.).

Auslöser

Auch ein methodisch perfekt durchgeführtes DM kann nicht zu brauchbaren Ergebnissen kommen, wenn die Eingangsdaten falsch oder irrelevant sind. Es gilt der Aphorismus „garbage in – garbage out“ (Linoff & Berry, 2011, S. 68). Die vorbereitenden Schritte dienen zur Minimierung dieses Problem indem u. a.

Bedeutung und Inhalt der Vorbereitung

- Daten ausgewählt werden, die repräsentativ für die relevante Grundgesamtheit sind. Eine Umfrage unter Kunden eines Unternehmens kann z. B. nur eingeschränkt Auskunft über die Gruppe der nicht-Kunden geben (Linoff & Berry, 2011, S. 70f.)
- Datenqualitätsprobleme, wie z. B. fehlende Werte, nach Möglichkeit unschädlich gemacht werden (Linoff & Berry, 2011, S. 685ff.).
- Daten auf genau der Ebene aggregiert werden, welche die für die geplante Untersuchung erforderliche Granulität aufweist (Linoff & Berry, 2011, S. 71ff.).
- Abgeleitete Variablen konstruiert werden, um den Daten mehr Aussagekraft zu verleihen (Linoff & Berry, 2011, S. 693–733).
- Irrelevante und irreführende Variablen entfernt werden (Linoff & Berry, 2011, S. 735–774).

Somit ist die erste Phase von besonderer Bedeutung und bildet eine wesentliche Grundlage zur Erzielung von guten Ergebnissen im Rahmen des KDD-Prozesses.

3.1 Problemdefinition

In Einheit 1 des vorliegenden Lehrbriefs wurde primär eine handlungsorientierte Modellierung von Entscheidungsprozessen zugrunde gelegt (vgl. Einheit 1, Abschnitt 2.2.2). Die betreffenden Entscheidungsmodelle sind qualitativ zu verstehen und bestehen in der Regel aus vier, nur lose voneinander abgegrenzten, Phasen, welche sich zyklisch wiederholen. Dies trägt dem Umstand Rechnung, dass in der Praxis – jenseits von Routineentscheidungen – die meisten Entscheidungsprobleme schlecht-strukturiert sind (Reichmann, 2011, S. 9). Eine strenge Aufteilung in einzelne Phasen kann zwar der gedanklichen Strukturierung dienen, wäre aber zu starr, um in der Praxis umgesetzt zu werden (Laux et al., 2012, S. 15).

Die Problemdefinition steht zu Beginn des Entscheidungsprozesses

Struktur von Entscheidungsproblemen

Auch wenn sie ein in der Praxis oft nicht zutreffendes Ideal beschreibt, so kann doch eine formale Modellierung von Entscheidungsprozessen, wie sie die *Entscheidungstheorie* beschreibt, Aufschluss über die Struktur von Entscheidungsproblemen geben und somit auch rahmengebend für die Problemdefinition innerhalb des KDD-Prozesses sein. In Anlehnung Reichmann (2011, S. 9) gilt ein Entscheidungsproblem als *wohl-strukturiert*, wenn es die folgenden vier Eigenschaften aufweist:

- Es existiert eine endliche Menge von Handlungsalternativen, welche vollständig bekannt ist.
- Zu jeder Handlungsalternative sind die Konsequenzen und Ergebnisse bekannt².
- Der Entscheider kann Ziele bezüglich der möglichen Ergebnisse formulieren.
- Die Handlungsalternativen können (algorithmisch) in eine eindeutige Rangfolge bezüglich der Ziele gebracht werden.

Lösung von Entscheidungsproblemen

Der in Einheit 1 (Abschnitt 2.2.1) vorgestellte Markov Decision Process (MDP) ist somit ein Beispiel für ein wohl-strukturiertes Entscheidungsproblem. Im Gegensatz dazu heißt ein Entscheidungsproblem, welches ein oder mehrere der obigen Merkmale nicht aufweist, schlecht-strukturiert. Zur Lösung wohl-strukturierter Entscheidungsprobleme sei an dieser Stelle auf die einschlägige Fachliteratur verwiesen: z. B. (Bamberg, Coenenberg & Krapp, 2008; Eisenführ & Weber, 2003; Laux et al., 2012). Der zugehörige Entscheidungsprozess aus Führungssicht soll hier nur kurz erläutert werden³. Im Anschluss daran findet sich eine, ebenfalls kurze, Darstellung der Struktur von Entscheidungsproblemen. Diese soll hier dazu dienen, die Dimensionen aufzuzeigen, welche bei der Definition des Entscheidungsproblems zu berücksichtigen sind.

² ggf. auch in Abhängigkeit von einem nicht beeinflussbaren und/oder nicht bekannten Umweltzustand

³ Eine ausführliche Diskussion findet sich im Lehrbrief „Informationsmanagement“ (KE 1 und 2)

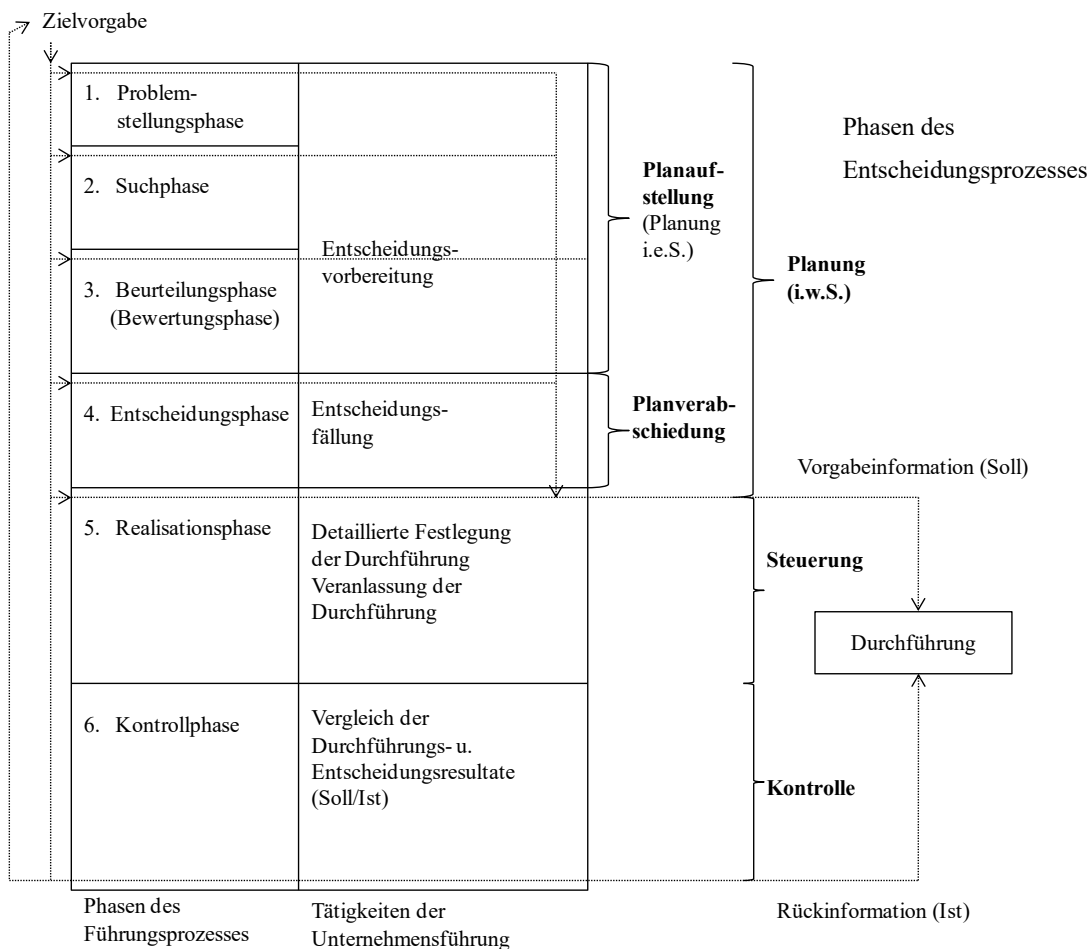


Abb. 5: Führungstätigkeiten in der Unternehmung
Quelle: entnommen aus (Hahn, 1996, S. 46)

Abb. 5 zeigt den Problemlösungsprozess in seiner Interpretation als Führungsprozess nach Hahn (1996). Zur Entscheidungsvorbereitung muss zunächst in der *Problemstellungsphase* die Entscheidungsaufgabe ermittelt werden. Neben einem Soll-Ist-Vergleich des derzeitigen Zustands mit dem angestrebten Zustand und der daraus abgeleiteten Problemstellung müssen die Ursachen für das betrachtete Problem analysiert werden. In der *Suchphase* werden mögliche Handlungsalternativen zusammengestellt werden. Die günstigsten Alternativen werden ausgewählt und detaillierter untersucht. Die *Beurteilungsphase* dient der Bewertung der Handlungsalternativen hinsichtlich der Zielsetzungen, die in der ersten Phase definiert wurden. In der *Entscheidungsphase* wählt der Entscheider nach einer ausführlichen Analyse und Diskussion die Alternative mit dem höchsten Zielerreichungsgrad aus. In der *Realisationsphase* wird die Handlungsalternative durchgeführt. In der letzten Phase, der *Kontrollphase*, werden die gewonnenen Ergebnisse analysiert und daraus, wenn die gewonnenen Ergebnisse nicht den Anforderungen entsprechen, neue Entscheidungsprozesse abgeleitet (Hahn, 1996, S. 38).

Der Problemlösungsprozess als Führungsprozess

Naturgemäß liegt ein Entscheidungsproblem nur dann vor, wenn mindestens zwei *Handlungsalternativen* vorhanden sind, zwischen denen überhaupt entschieden werden kann. Zudem müssen die Alternativen sich auf Sachverhalte beziehen, die der Entscheider auch tatsächlich beeinflussen kann. Letztere werden durch *Entscheidungsvariablen* quantifiziert (Laux et al., 2012, S. 30f.). Beispielsweise ist „Steigerung des Umsatzes auf Betrag u “ keine Handlungsalternative, da der Umsatz nicht direkt festgelegt werden kann und demnach keine plausible Entscheidungsvariable ist. Dagegen stellt „Festsetzung des Preises für Produkt y “ eine Handlungsalternative dar.

Alternativenmenge

auf den Betrag p'' ein Handlungsalternative bezüglich der Entscheidungsvariablen „Preis des Produktes y'' “ dar. Eine einzelne Handlungsalternative kann mehrere Entscheidungsvariablen berücksichtigen, z. B. den Preis für mehr als ein Produkt.

Für stetige Entscheidungsvariablen ist die Anzahl der Alternativen potentiell unendlich groß. Daher soll im Folgenden von einer Vorauswahl endlich-vieler Alternativen ausgegangen werden (Eisenführ & Weber, 2003, S. 18). Dies kann erreicht werden, indem der Wertebereich der einzelnen Entscheidungsvariablen begrenzt wird. Im Beispiel ist der Preis etwa in der Regel natürlicherweise durch 0 nach unten begrenzt. Darüber hinaus kann z. B. vereinfachend davon ausgegangen werden, dass nur einige bestimmte Preispunkte, als Repräsentanten des jeweiligen Preisniveaus, berücksichtigt werden und nicht das gesamte Spektrum an möglichen Preisen.

Formal ausgedrückt gilt für die Alternativenmenge (A) mit der Anzahl der Alternativen (N_A), der Anzahl der Entscheidungsvariablen (N_D) und dem Wertebereich der (i)-ten Entscheidungsvariable (D_i):

$$A = \{a_1, a_2, \dots, a_{N_A}\} \quad \text{und} \quad (3.1)$$

$$\forall i \in \{1, \dots, N_A\}: a_i = (d_{i,1}, d_{i,2}, \dots, d_{i,N_D}) \quad \text{und} \quad (3.2)$$

$$\forall j \in \{1, \dots, N_D\}: d_{i,j} \in D_j \quad (3.3)$$

Vollkommene Alternativenstellung

Für das Vorliegen eines Entscheidungsproblems muss die Alternativenmenge so gewählt sein, dass der Entscheider gezwungen ist, eine der betrachteten Alternativen zu ergreifen und dabei gleichzeitig nur genau eine der Alternativen realisieren kann (Bamberg et al., 2008, S. 16).

Alternativen-ermittlung

In einigen Fällen ergibt sich die Alternativenmenge „natürlich“. So ist z. B. die Preissetzung in der Regel eine natürliche Voraussetzung zur Marktteilnahme. In

anderen Fällen muss die Alternativenmenge erst erzeugt werden. In solchen Fällen bietet es sich z. B. an, die Entscheidungsvariablen aufgrund ihrer mutmaßlichen Auswirkungen auf eine andere Zielgröße auszuwählen, die eigentlich im Fokus der Entscheidung steht. Es kann ebenfalls hilfreich sein, zunächst die „ideale“ Alternative auszumalen, um dann in deren Umfeld verschiedene umsetzbare Alternativen zu betrachten (Eisenführ & Weber, 2003, Kap. 4).

Zustandsmenge

Neben den Entscheidungsvariablen, die der Entscheider direkt beeinflussen kann, ist das Ergebnis einer Handlungsalternative in der Regel von zusätzlichen Faktoren abhängig. Hierzu zählen z. B. die Entwicklung von Zinssätzen oder Nachfrage am Markt. Die Kombination aller relevanten Einflussfaktoren wird dabei als Zustand bezeichnet, wobei ein Zustand durch Umfelddaten beschrieben wird. Welche Umfelddaten für ein konkretes Entscheidungsproblem relevant sind, lässt sich nicht allgemein sagen. Im Folgenden soll, analog zur Alternativenmenge, davon ausgegangen werden, dass der Entscheider endlich-viele relevante Umfeldvariablen sowie, aufbauend darauf, endlich viele repräsentative Zustände auswählt. Für die Entscheidung können dabei sowohl der gegenwärtige Ist-Zustand als auch zukünftige Zustände relevant sein. Ohne auf die Details der Entscheidung unter Unsicherheit bzw. Risiko eingehen zu wollen, sei angemerkt, dass der gegenwärtige Zustand in der Regel nicht vollständig bekannt ist. Dies gilt erst recht für den zukünftigen Zustand (Bamberg et al., 2008, S. 18f.).

Formal gilt, ebenfalls ganz analog zu A , für die Zustandsmenge (Z) mit der Anzahl der Zustände (N_Z), der Anzahl der Umfeldvariablen (N_E) und dem Wertebereich der i -ten Umfeldvariable (E_i):

$$Z = \{z_1, z_2, \dots, z_{N_Z}\} \quad \text{und} \quad (3.4)$$

$$\forall i \in \{1, \dots, N_Z\}: z_i = (e_{i,1}, e_{i,2}, \dots, e_{i,N_E}) \quad \text{und} \quad (3.5)$$

$$\forall j \in \{1, \dots, N_E\}: e_{i,j} \in E_j \quad (3.6)$$

Eine bedeutungsvolle Handlungsalternative hat *Konsequenzen* für mindestens eine Größe, die für den Entscheider von Interesse ist. Eine solche Größe wird als *Zielvariable* bezeichnet. Die Kombination aller relevanten Zielvariablen bestimmt das *Ergebnis* der Handlungsalternative. Handlungsalternativen sollten nach ihrem (erwarteten) Ergebnis beurteilt werden (Laux et al., 2012, S. 31f.). Die Funktion, die einer Kombination aus Handlungsalternative und Zustand ein Ergebnis zuordnet, heißt *Ergebnisfunktion* (Bamberg et al., 2008, S. 22).

Handlungsergebnis

Anders als bei (A) und (Z) können keine Ergebnisse von vornherein ausgeschlossen werden. Welche Ergebnisse auftreten können, ist allein durch die Ergebnisfunktion (ρ) bestimmt. Für die Ergebnismenge (R) gilt daher mit der Anzahl der Zielvariablen (N_G) und dem Wertebereich der (i)-ten Zielvariable (G_i):

$$R = G_1 \times G_2 \times \dots \times G_{N_G} \quad (3.7)$$

$$\rho: A \times Z \rightarrow R \quad (3.8)$$

Das Aufstellen eines *Zielsystems* über den Zielvariablen und die Auswahl der optimalen Alternative in Bezug auf dieses Zielsystem soll hier nicht weiter diskutiert werden. Vorgehensweisen dazu liefert die Entscheidungstheorie. Vielmehr sollen hier nur grundsätzliche Anforderungen an Ziele genannt werden. Dabei sind Ziele durch *Inhalt*, *angestrebtes Ausmaß* und *zeitlichen Bezug* charakterisiert. Der Inhalt eines Ziels muss *operational* sein. Das heißt, der Inhalt muss durch eine Zielvariable messbar sein, und es muss bekannt sein, welche möglichen Werte der Zielvariable welchen Zielerreichungsgrad ausmachen. Das Ausmaß des Ziels kann *unbegrenzt* sein, d. h. ein immer höherer Zielerreichungsgrad wird angestrebt, z. B. durch unbegrenztes Gewinnstreben. In der Praxis existieren jedoch Begrenzungsfaktoren, deren Berücksichtigung zu einer *begrenzten* Zielsetzung führt. Schließlich kann die Zielerreichung nur dann sinnvoll überprüft werden, wenn das Ziel einen Zeitraumbezug hat. Ein *statisches* Ziel bezieht sich dabei auf einen festen Zeitpunkt oder Zeitraum, z. B. ein bestimmtes Geschäftsjahr. Ein *kinetisches* Ziel berücksichtigt den Zeitverlauf und kann z. B. monatlich definiert sein. Ein *dynamisches* Ziel dagegen berücksichtigt zusätzlich den Zielerreichungsgrad zu einem anderen Zeitraum, z. B. eine Steigerung im Vergleich zum Vormonat (Heinen, 1985/1992, S. 98ff.).

Anforderungen an Ziele

Generell lassen sich drei verschiedene Beziehungsarten zwischen Zielen unterscheiden. *Indifferenz* zwischen zwei Zielen liegt vor, wenn sich die Realisierung des einen Ziels nicht auf die Realisierung des anderen Ziels auswirkt. Beispielsweise sind die Ziele Verringerung der Lagerzeit und Erhöhung der Produktqualität indifferent zueinander, wenn sich die Qualitätseigenschaften des Produktes nicht durch Lagerung ändern. Die zweite Zielbeziehungsart, *Komplementarität*, liegt dann vor, wenn die Erreichung des einen Ziels die Erreichung des anderen Ziels positiv beeinflusst. Die dritte mögliche Art der Beziehung zwischen zwei Zielen

Zielbeziehungen

ist die *Konkurrenz*. Dies bedeutet, dass die Realisierung eines Ziels zur Vermeidung oder Verschlechterung eines anderen Ziels führt. Die Ziele stehen im Konflikt zueinander. So können z. B. die Erhöhung der Produktqualität und die Senkung der Produktionskosten in der Regel nicht Hand in Hand erreicht werden (Heinen, 1985/1992, S. 101f).

Symmetrie der Zielbeziehungen

Bei der Komplementarität von zwei Zielen lassen sich zwei Fälle unterscheiden. *Symmetrische* Komplementarität besagt, dass die Erreichung der beiden Ziele wechselseitig voneinander abhängt, wohingegen bei der *asymmetrischen* Komplementarität die Realisierung des einen Ziels die Erreichung des anderen Ziels zwar positiv beeinflusst, der umgekehrte Fall jedoch nicht vorliegt. Bei symmetrischen, komplementären Zielen kann die Zielfunktion in den meisten Fällen auf die Realisierung eines der beiden Ziele beschränkt werden. Beispielsweise sind die Erhöhung des Gewinns und die Erhöhung der Eigenkapitalrendite symmetrisch komplementäre Ziele. Eine Erhöhung des einen Ziels führt automatisch zur Erhöhung des anderen Ziels. Somit reicht es aus, wenn lediglich eins der beiden Ziele verfolgt wird (Bamberg et al., 2008, S. 50).

Erstellung eines Projekts

Neben der Analyse der Problemstellung und Zielsetzung sollte, je nach Größe und Aufwand des geplanten KDD-Vorhabens, ein Projektplan erstellt werden. Hierzu ist, im Anschluss an die Definition der Zielsetzung, eine Analyse der verfügbaren Ressourcen, der Anforderungen und Beschränkungen sowie der Risiken erforderlich. Bei der *Analyse und Beschreibung der Ressourcen* sollten insbesondere Personalressourcen berücksichtigt werden. Es muss überprüft werden, ob genug qualifiziertes Personal sowohl zur technischen Durchführung des KDD als auch zur späteren Interpretation der Ergebnisse zur Verfügung steht. Wenn beispielsweise eine Werbeaktion geplant ist und hierzu die Kunden in verschiedene Kundentypen geclustert werden sollen, muss Personal zur Verfügung stehen, welches die Clusteranalyse technisch durchführen kann, aber es muss auch genügend Personal aus dem Marketingbereich verfügbar sein, welches das Ergebnis auswertet und daraus Handlungsempfehlungen ableiten kann. Neben den Personalressourcen sollten auch die benötigten Sachressourcen zum gewünschten Zeitpunkt zur Verfügung stehen. Hierin inbegriffen ist z. B. die Hard- und Software zur Durchführung des DM. Das Ziel der *Anforderungsanalyse* ist es, eine möglichst genaue Beschreibung der Anforderungen an die Ergebnisse des KDD-Prozesses zu erhalten. Hierzu sollte ein Zeitplan entworfen werden, aus dem insbesondere der Fertigstellungszeitpunkt, aber auch verschiedene gesetzte Meilensteine abgelesen werden können. Außerdem müssen Anforderungen an die Qualität und Nachvollziehbarkeit der Ergebnisse gestellt werden. Es muss vorab klar definiert werden, in welcher Form die Ergebnisse zur Verfügung gestellt werden. Ein weiterer wichtiger Aspekt, ist die *Analyse der Beschränkungen*. Diese ist einerseits stark mit der Ressourcenverfügbarkeit verknüpft, andererseits können auch Beschränkungen anderer Art zu einem Nichterfüllen der gesetzten Ziele führen. So können z. B. gesetzliche Vorschriften dazu führen, dass bestimmte Datenfelder von Kundendaten nicht zum DM genutzt werden dürfen. Andere Beschränkungen können z. B. die Datenqualität oder das Datenvolumen betreffen. Der letzte wichtige Punkt bei der Projektplanung ist die *Analyse der Projektrisiken*, um z. B. auf mögliche Verzögerungen vorbereitet zu sein und ggf. vorab schon Gegenmaßnahmen planen zu können. Zusätzlich sollten sich die Verantwortlichen auch mit möglichen Gründen für das Scheitern auseinandersetzen, damit Tendenzen in diese Richtungen frühzeitig erkannt werden und diesen entgegengesteuert werden kann (Hippner & Wilde, 2001).

3.2 Auswahl der Daten

Nachdem die Problemstellung analysiert und daraus ein konkretes Entscheidungsproblem abgeleitet wurde, muss überprüft werden, inwiefern die zur Verfügung stehende Datenbasis dazu geeignet ist, dieses zu lösen. Benötigt werden dazu grundsätzlich diejenigen Daten, die zuvor als relevante Umfelddaten identifiziert wurden, aber ggf. auch Daten über den Ist-Wert von Entscheidungs- und Zielvariablen zum Betrachtungszeitpunkt.

Benötigte Daten

Häufig kann dieser Bedarf jedoch nicht erfüllt werden. Dies liegt u. a. daran, dass viele Attribute in der Vergangenheit noch nicht erhoben wurden, weil sie bis dato für nicht nützlich erachtet wurden. Dies ist häufig der Fall bei Daten, die keinen direkten Nutzen für die Geschäftsprozesse haben. Diese Daten werden dann entweder gar nicht, oder nicht mit der nötigen Sorgfalt erfasst, sodass der Datenbestand Lücken aufweist und somit die Anforderungen der DM-Methoden nicht erfüllt. Darüber hinaus können drei verschiedene Problemarten auftreten. *Technische Probleme* führen dazu, dass die Anforderungen, die DM-Methoden an die Datenbasis stellen, nicht erfüllt werden können. Somit können die entsprechenden DM-Methoden im KDD-Prozess nicht herangezogen werden. Des Weiteren können *organisatorische Probleme* auftreten. So können beispielsweise fehlende Zugriffsrechte dazu führen, dass auf verschiedene Datenbanken nicht zugegriffen werden kann, sodass diese nicht nutzbar sind. Die letzte Problemklasse sind *rechtliche Einschränkungen*. So können unterschiedliche Gesetze dazu beitragen, dass bestimmte Daten nicht erfasst bzw. genutzt werden dürfen. Insbesondere private Daten werden vom Gesetzgeber als schützenswert angesehen (Düsing, 2006).

Problematik der Datenversorgung

„In der besten aller Welten“, schreiben Linoff und Berry (2011), „befinden sich die benötigten Daten bereits in einem DWH: bereinigt, verfügbar, historisch akkurat und häufig aktualisiert.“. In der Praxis dagegen sind die benötigten Daten oft auf verschiedenste Quellen verteilt, deren Auswahl wiederum sehr stark vom betreffenden Entscheidungsproblem abhängt (Linoff & Berry, 2011, S. 165). Die (proaktive) Bereitstellung von Daten entsprechend dem o. g. Ideal ist Thema der Einheit 3 dieses Lehrbriefs. Hier dagegen soll der Datenbegriff für die Zwecke des KDD aufbauend auf dem Entscheidungsproblem weiter formalisiert werden. Auch wenn also inhaltlich keine allgemeinen Aussagen darüber getroffen werden können, welche Daten für ein konkretes Entscheidungsproblem relevant sind, kann ein formal und methodisch sorgfältiges Vorgehen dazu beitragen, diese Auswahl zu erleichtern.

Datenquellen

Im Kontext der Entscheidungstheorie repräsentieren Umweltzustände $z \in Z$ denjenigen Teil der realen Welt, der für das Entscheidungsproblem relevant ist, aber nicht vom Entscheider beeinflusst werden kann. Eine solche Trennung zwischen beeinflussbaren und nicht beeinflussbaren Größen der realen Welt ist für die Entscheidungstheorie sehr nützlich und spiegelt implizit die Annahme wider, dass die beeinflussbaren Größen dem Entscheider ohnehin bekannt sind. Für KDD dagegen soll, unabhängig von Einflussmöglichkeiten, ein Ausschnitt der realen Welt betrachtet werden. Ein solcher Realweltauusschnitt wird als *Miniwelt* oder *Universe of Discourse* bezeichnet (Elmasri & Navathe, 2002, S. 24), oder kurz als *Universum* (U). Dabei soll U als endliche, geschlossene Menge verstanden werden (Cios et al., 1998, S. 30). In (3.11)–(3.13) gilt z. B. $U = Z$. Denkbar ist aber auch $U = A \times Z$ oder, in anderen Kontexten, eine völlig andere Wahl.

Betrachtetes Universum

Formalisierung der benötigten Daten

Benötigte Daten, im Sinne dieses Abschnitts, entsprechen gewünschten Beobachtungen des Inhalts von U , d. h. Beobachtungen über Eigenschaften von *Objekten* der realen Welt, soweit sie für das betrachtete Problem relevant sind. Es steht dabei nicht die gegenständliche Abgrenzung der Objekte im Vordergrund, sondern vielmehr deren konzeptionelle Abgrenzung als Bezugsobjekt für eine Reihe von Eigenschaften, die sinnvoll als Einheit betrachtet werden können. Die Objekte stehen in einem *logischen Zusammenhang* zueinander und die Eigenschaften einzelner Objekte haben eine *inhärente Bedeutung*. Die Festlegung des Universums und seiner Objekte dient einem gemeinsamen *Zweck* (Elmasri & Navathe, 2002, S. 24).

Begriffsklärung IS

Wie schon in Einheit 1 (Abschnitt 4) dieses Lehrbriefs ausgeführt, wird der Begriff des *IS*, je nach Kontext, höchst unterschiedlich verwendet. Für diesen Lehrbrief wird prinzipiell das Begriffsverständnis als Betriebliches Informationssystem (BIS) zugrunde gelegt. Für die Zwecke des KDD, und insbesondere im Kontext der hier diskutierten Auswahl der Daten, kann jedoch ein wesentlich datenorientierteres Begriffsverständnis des IS hilfreich sein. Um die Bezeichnungen einfach und in Übereinstimmung mit der Literatur zu halten, soll ein solches *IS für die Zwecke des KDD* im Weiteren einfach als IS bezeichnet werden. Wo Verwechslungsgefahr mit dem BIS besteht, wird gesondert darauf hingewiesen.

Formalisierung der Datenversorgung

Gemäß der in Abschnitt 3.1 getroffenen Definitionen, ist der gegenwärtige Umweltzustand $z \in Z$ dem Entscheider ggf. nicht genau bekannt. Die zuvor beschriebene Problematik der Datenversorgung zeigt, dass dieses Problem auch in der Verallgemeinerung von Z auf U besteht. Formal sei angenommen, dass über ein Objekt in U nur das hinreichend bekannt ist, was als *Nachricht* in einem IS vorliegt. Der Zusammenhang zwischen Objekten und Nachrichten sei durch die Struktur des Informationssystems (S) beschrieben. Dabei sei S nicht deterministisch, d. h. der Zusammenhang sei nicht eindeutig. Darüber hinaus sei es möglich, dass S selbst unbekannt ist. Dann sei ebenfalls unbekannt, wie gut die vorliegenden Nachrichten die zugrundeliegenden Objekte repräsentieren. Dies bildet die Problematik der Datenversorgung ab.

Seien U und die Menge potentieller Nachrichten (M) gegeben durch

$$M = \{m_1, m_2, \dots, m_{N_M}\} \quad (3.9)$$

$$U = \{o_1, o_2, \dots, o_{N_U}\} \quad (3.10)$$

so gilt

$$S = \begin{pmatrix} s_{1,1} & \cdots & s_{1,N_M} \\ \vdots & \ddots & \vdots \\ s_{N_U,1} & \cdots & s_{N_U,N_M} \end{pmatrix} \quad (3.11)$$

$$s_{i,j} = \mathbb{P}(m_j | o_i) \quad (3.12)$$

$$\forall i \in \{1, \dots, N_U\} : \sum_{j=1}^{N_M} s_{i,j} = 1 \quad (3.13)$$

das heißt: $s_{i,j}$ ist die bedingte Wahrscheinlichkeit, dass zur Beobachtung von Objekt o_i die Nachricht m_j realisiert wird (Bamberg et al., 2008, S. 19f.).

Ziel der folgenden Überlegungen ist es, den für KDD zur Verfügung stehenden Datenbestand als Sammlung von Nachrichten, die in einem IS realisiert wurden, zu etablieren. Die dabei verwendeten Begriffe sind in Anlehnung an Cios et al. (1998, S. 29f.) und Witten, Frank und Hall (2011, S. 39) gewählt. Der für das betrachtete KDD-Problem relevante Datenbestand werde als *Konzept* (X) bezeichnet. X ist also ein uneindeutiges und potentiell fehlerhaftes Abbild von U . Die Rolle von KDD im Rahmen des Entscheidungsprozesses, wie sie in Abschnitt 2.3 beschrieben wurde, kann in diesem Rahmen formal als das Schließen von X auf U aufgefasst werden. Insbesondere im Spezialfall $U=Z$ integriert sich diese Deutung nahtlos in die Entscheidungstheorie.

Formalisierung von Nachrichten

Modellhaft sei X im IS als Datenbanktabelle realisiert, wobei die Zeilen den Nachrichten und die Spalten den Attributen entsprechen. Die Realisation einer konkreten Nachricht $x_i \in M$ werde als *Instanz* mit Anzahl der Instanzen (N_X) bezeichnet, wobei x_i die i -te Zeile von X meint. Eine Instanz $x_i \in M$ sei charakterisiert durch *Attribute* mit einer festgelegten Anzahl der Attribute (N_Q) und dem Wertebereich des i -ten Attributs (V_i). Ein Attribut wird auch als *Feature* bezeichnet und der Vektor der Attributwerte einer Nachricht dementsprechend als *Featurevektor*. Es gilt also:

Repräsentation eines Konzepts

$$X = \begin{pmatrix} x_{1,1} & \cdots & x_{1,N_Q} \\ \vdots & \ddots & \vdots \\ x_{N_X,1} & \cdots & x_{N_X,N_Q} \end{pmatrix} \quad (3.14)$$

$$\forall i \in \{1, \dots, N_X\} : x_i = (x_{i,1}, \dots, x_{i,N_Q}) \in M \quad \text{und} \quad (3.15)$$

$$\forall j \in \{1, \dots, N_Q\} : x_{i,j} \in V_j \quad (3.16)$$

Es ist ständige Aufgabe von Analysten und Entscheidern, sich darüber zu versichern, dass das betrachtete IS die relevanten Teile von U auch tatsächlich geeignet abbildet. Dies ist Voraussetzung, wenn in den folgenden Abschnitten dieser Einheit innerhalb des hier vorgestellten IS-Begriffs argumentiert wird. Während also die konkrete Auswahl der Daten problemabhängig ist, liegt mit dem IS, im hier beschriebenen Sinne, ein Formalismus vor, innerhalb dessen die Qualität der getroffenen Auswahl beurteilt werden kann.

Bedeutung der Struktur des IS

Der Fokus der bisherigen Ausführungen lag darauf, aufzuzeigen, dass einerseits das Problem der Datenversorgung gelöst werden muss und andererseits aus den verfügbaren Datenquellen diejenigen ausgewählt werden müssen, welche für die Fragestellung relevant sind. Zudem ging es um die Unsicherheit, welche stets mit diesem Prozess verbunden ist. Diesem Themenkomplex gegenüber steht das bereits in Einheit 1 angerissene Problem, dass auch ein „Zuviel“ an Daten vorliegen kann. Grundsätzlich wird unter dem Stichwort „Big Data“ versucht, Systeme zu entwickeln, die inhärent auf große Datenmengen optimiert sind. Für die klassische BI dagegen ist oft noch eine *Datenreduktion* das Mittel der Wahl, um die Datenmenge beherrschbar zu machen.

Behandlung großer Datenmengen

„Das hauptsächliche Unterscheidungsmerkmal zwischen Business Intelligence und Big Data ist die Ausrichtung auf die gesammelten und verarbeiteten Daten. Business Intelligence Lösungen

setzen strukturierte, konsistente und beständige Daten voraus, wohingegen Big Data Lösungen speziell auf unstrukturierte und möglicherweise nicht konsistente Daten hin optimiert sind.“ (Klein, Tran-Gia & Hartmann, 2013).

Datenreduktion durch Aggregation oder Stichproben

Ein großer Datenbestand resultiert häufig daraus, dass die gesammelten Daten zu gering aggregiert sind. In einigen Fällen bietet es sich an, mehrere Instanzen zu einer einzigen Instanz zusammenzufassen. So könnte ein Unternehmen z. B. die Umsatzzahlen eines einzelnen Kunden für jeden Monat eines Jahres festhalten. Je nach Zielsetzung könnten diese zwölf einzelnen Messpunkte beispielsweise zu einer einzigen Instanz zusammengefasst werden. Der Wert könnte sich entweder aus der Summierung aller einzelnen Umsätze ergeben und würde dann dem Jahresumsatz eines Kunden entsprechen, oder aber es könnte der Mittelwert über alle zwölf Instanzen gebildet werden, was dann als durchschnittlicher Monatsumsatz des Kunden interpretiert werden könnte. In diesem Beispiel würden sich die zu analysierenden Instanzen auf $\frac{1}{12}$ reduzieren. Es sollte jedoch berücksichtigt werden, dass bei jeder Aggregation die Information, die sich auf einer unteren Aggregationsebene befindet, verloren geht. Es besteht jedoch die Möglichkeiten, dass neben den aggregierten Daten noch weitere Daten zur Beschreibung gesammelt und zur Auswertung mittels DM genutzt werden. So könnte im genannten Beispiel neben dem Jahresumsatz auch die Standardabweichung der Monatsumsätze genutzt werden, um so die Aggregation etwas transparenter zu machen. Eine weitere Möglichkeit zur Reduktion der Datenmenge bietet das Ziehen und Auswerten von Stichproben an Stelle einer Analyse und Verwendung aller Daten. (Hippner & Wilde, 2001).

3.3 Bereinigung und Aufbereitung der Daten

Ziel der Datenaufbereitung

Die wichtigste Aufgabe im Bereich der Datenaufbereitung ist die Datenbereinigung. Mit Hilfe der Datenbereinigung soll die Qualität der Daten gesteigert werden. Datenbereinigung kann dabei definiert werden, als eine nachträgliche Korrektur der bereits entstandenen Fehler, bei der am Ende eine fehlerfreie Datenbasis wiederhergestellt ist. Die Datenbereinigung erfolgt dabei nicht permanent, sondern zu definierten Zeitpunkten im Prozess. Sie kann jedoch auch mehrmalig durchgeführt werden (Zwirner, 2008). Das Ziel der Datenbereinigung ist es, inkorrekte, inkonsistente oder unvollständige Daten nachträglich zu korrigieren und so für die Anwendung von DM nutzbar zu machen (Bauer & Günzel, 2001, S. 92).

Kurz- und langfristige Perspektive

Datenbereinigung schafft temporäre Abhilfe bei Datenqualitätsproblemen, indem sie den Datenbestand zum Zeitpunkt der Bereinigung in einen möglichst gültigen Zustand versetzt. Um die Datenqualität langfristig zu sichern, sollte ein Unternehmen immer auch aus den gemachten Fehlern lernen. Das Datenqualitätsmanagement (DQM) zur Integration der Fehlererkennung, Datenqualitätssicherung und -überprüfung in die Prozesse des Unternehmens, dient dem Ziel, die Datenqualität sicherstellen, überwachen und steuern zu können. Entsprechende Maßnahmen und deren Unterstützung durch ein geeignete DWH sind Thema der Einheit 3.

Einordnung der Datenbereinigung

Im vorherigen Abschnitt 3.2 wurde deutlich, dass eine Dichotomie zwischen den tatsächlich verfügbaren Daten und den zugrundeliegenden Objekten be-

steht. Vereinfacht ausgedrückt, können die Daten die Realität fehlerhaft abbilden. In diesem Abschnitt soll nun weiter präzisiert werden, wo im Rahmen von KDD Fehler auftreten können, welche Fehlerarten es allgemein gibt, wie Fehler innerhalb der Daten zu erkennen sind und wie mit Fehlern umgegangen werden kann.

Für einen Überblick über die grundsätzlichen Möglichkeiten, wie im Rahmen von KDD überhaupt Fehler entstehen können, sei zunächst auf Abb. 6 verwiesen. Fonseca und Fieller (2006) unterscheiden zwischen dem *idealen*, dem *tatsächlichen* und dem *angenommenen* Prozess der Datengenerierung. In der Nomenklatur des vorherigen Abschnitts, welche hier weiter befolgt werden soll, liegen die Daten als Konzept X vor, daher könnte auch von Konzeptgenerierung gesprochen werden.

Drei Modelle der Datengenerierung

Ideal wäre es, die Zeilen von X , also die Instanzen, wären eine Stichprobe von U sowie unabhängig und identisch verteilt. X wäre dann ein getreues Abbild von U . Dennoch wäre unbekannt, welche Vorgänge in U zu genau dieser Beobachtung geführt haben. Wenn die Objekte in U sich nicht rein zufällig verhalten, was jede Mustererkennung unmöglich machen würde, gibt es ein unbekanntes Modell von U , welches die Verhältnisse in U genau beschreibt. Dieses wäre das ideale Modell der Datengenerierung.

Ideales Modell

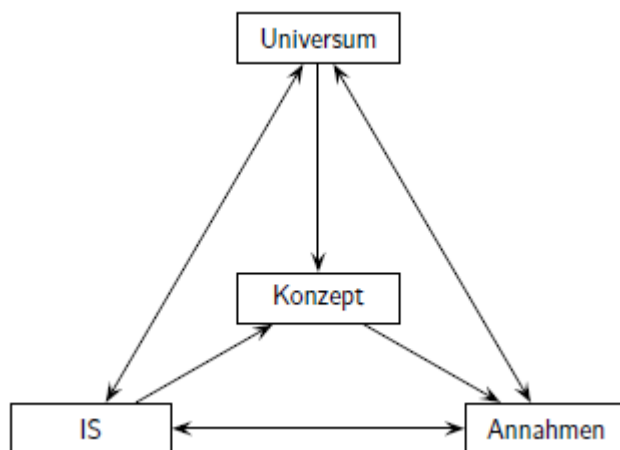


Abb. 6: Drei Entstehungsmodelle für ein Konzept
Quelle: in Anlehnung an (Fonseca & Fieller, 2006)

Tatsächlich ist dieses Modell nicht nur unbekannt, sondern wird zusätzlich noch durch die Sammlung der Daten in einem IS beeinflusst. Ein IS innerhalb dessen jedem Objekt genau eine Nachricht zugeordnet ist, d. h. für jedes i existiert genau ein j , sodass $s_{i,j} = 1$, heißt *vollkommenes* IS (Bamberg et al., 2008, S. 20). Bei Vorliegen eines vollkommenen IS sind das ideale und das tatsächliche Modell der Datengenerierung gleich, jedoch auch gleichermaßen unbekannt. In der Praxis wird dagegen meist ein unvollkommenes IS vorliegen. Ein unvollkommenes IS ist dadurch charakterisiert, dass die Zuordnung von Nachrichten und Objekten nicht eindeutig ist. Es gibt weniger Nachrichten als Objekte, oder es existieren Paare (i, j) , sodass $s_{i,j} \notin \{0,1\}$ (Bamberg et al., 2008, S. 21). In diesem Fall muss das ideale Modell zusätzlich noch um S als ein statistisches Modell des IS ergänzt werden, um das tatsächliche Modell der Datengenerierung zu bilden. Auch S kann zusätzlich unbekannt sein.

Tatsächliches Modell

Beispiel

Ein Unternehmen beauftragt ein Marktforschungsinstitut zu einer telefonischen Konsumentenbefragung. Von Interesse sei vereinfachend hier nur das Alter potentieller Kunden. Ein konkreter Proband gebe sein Alter mit 32 an. Im *Idealfall* wäre der Proband auch wirklich 32 Jahre alt, die 32 würde als Nachricht im System realisiert und der Anteil der 32-jährigen an den Befragten in X entspräche deren Anteil an allen potentiellen Kunden in U . Insbesondere wäre $s_{32,32} = 1$, d. h. ein reales Alter von 32 würde auch wirklich als solches im System realisiert. *Tatsächlich* dagegen könnte der Interviewer sich verhören, oder bei der Eingabe vertippen. Der Proband könnte auch von vornherein absichtlich eine falsche Angabe gemacht haben. Im tatsächlichen IS gelte also $s_{32,32} < 1$ und es wäre unbekannt, wie sich die Restwahrscheinlichkeit auf die anderen Antworten verteilt.

Angenommenes Modell

Die formale Aufgabe von KDD, nämlich aus X zurück auf U zu schließen (vgl. S. 23), erweist sich somit als nicht-trivial, da im Allgemeinen weder die „wahren“ Objekte in U noch die „wahre“ Struktur S bekannt sind. Vielmehr erfordert ein solcher Schluss die Bildung eines angenommenen Modells der Datengenerierung. Die DM-Verfahren, welche in Kapitel 4 vorgestellt werden, sind von ihrem Wesen her Verfahren, die es ermöglichen, automatisiert und strukturiert die Bildung eines solchen angenommenen Modells zu unterstützen. Da diese Verfahren aber nicht direkt auf U operieren, was schon prinzipiell unmöglich wäre, sondern auf X , muss es das Ziel der Datenbereinigung sein, dass X ein möglichst getreues Abbild von U ist.

Fehlerbegriff

Von einem Fehler kann in Bezug auf Abb. 6 also dann gesprochen werden, wenn wechselseitig Abweichungen zwischen einigen oder allen drei Modellen der Datengenerierung auftreten. Im Rahmen des KDD ist das Konzept das Ergebnis des tatsächlichen Modells der Datengenerierung, steht aber stellvertretend für das unbekannte ideale Modell und ist gleichzeitig Quelle für das angenommene Modell. In diesem Abschnitt werden Abweichungen zwischen Universum und Konzept behandelt, die Ausdruck der Unvollkommenheit des IS sind. Als Fehler soll daher hier der Fall bezeichnet werden, dass die Daten das Universum entweder *nicht korrekt abbilden* oder der abgebildete Realitätsausschnitt *nicht dem gewünschten Realitätsausschnitt entspricht*. Es wird versucht, das tatsächliche Modell dem idealen Modell anzunähern. Daneben gibt es Fehlerquellen, welche das angenommene Modell betreffen. Diese werden in den Kapiteln 4 und 5 diskutiert.

Fehlerklassen

Grundsätzlich lassen sich die auftretenden Fehler in drei Fehlerklassen einordnen. Semantische Fehler, Coverage-Fehler und syntaktische Fehler (Müller & Freytag, 2003). *Semantische Fehler* sind, im Sinne des vorherigen Absatzes, solche Fehler, bei denen die als Konzept vorhandenen Daten das Universum nicht korrekt abbilden, die Semantik der Daten also nicht stimmt. Diese Fehlerklasse kann z. B. ganz trivial durch fehlerhafte Eingabe der Daten auftreten. Darüber hinaus treten semantische Fehler auf, wenn die Realität nicht hinreichend genau beobachtet werden kann. So fallen in diese Klasse alle Arten von Messfehlern, die durch ungenaue Sensorik entstehen, aber auch Messfehler im erweiterten Sinne. Beispielsweise könnten im Rahmen von Kundenbefragungen ermittelte Zahlungsbereitschaften von der realen Zahlungsbereitschaft der Kunden abweichen. Eine weitere Fehlerklasse im Sinne des vorherigen Absatzes sind *Coverage-Fehler*. Diese Fehler führen dazu, dass der durch die vorhandene Datenmenge beschriebene Realitätsausschnitt kleiner ist als ursprünglich angenommen, etwa weil Teile der Gesamtdatenmenge aus unterschiedlichen Gründen nicht verwendet werden können.

Syntaktische Fehler sind, genau wie semantische Fehler, dadurch gekennzeichnet, dass die Daten das Universum nicht korrekt abbilden. Während die Daten jedoch bei semantischen Fehlern inhaltlich von der Realität abweichen, liegt bei syntaktischen Fehlern, ein Fehler in der Form der Daten vor, welcher es unmöglich macht, diese korrekt zu interpretieren. Zum genauen Verständnis sei der Zusammenhang zwischen Universum, IS und Konzept (Abb. 6) nochmals in anderem Lichte betrachtet. Der Begriff des IS wurde in diesem Abschnitt bislang gebraucht, um den strukturierten Übergang von U zu X über S zu beschreiben. Dabei wurde für die Zwecke von KDD angenommen, dass X in einem einfachen Nachrichtenformat vorliegt. Wie sich im weiteren Verlauf dieser Einheit zeigen wird, ist dieses Format, insbesondere V_i aber nicht eindeutig. Für das Konzept selbst muss also ein viertes Modell, das Datenmodell, implizit mitgedacht werden. Ein solches Datenmodell kann z. B. durch ein Entity-Relationship-Modell (ERM) (vgl. Einheit 1, Abschnitt 3.1) ausgedrückt werden. Semantische Fehler sind also Fehler, bei denen die Daten innerhalb eines gegebenen Datenmodells inhaltlich von der Realität abweichen. Syntaktische Fehler dagegen sind solche, bei denen die Daten strukturell von den Annahmen des gegebenen Datenmodells abweichen und somit in Bezug auf das Datenmodell unverständlich sind.

Die Diskussion des ERM in Einheit 1 hat bereits gezeigt, dass es kein „einzig richtiges“ Datenmodell für einen bestimmten Realitätsausschnitt geben kann, sondern das Datenmodell im Rahmen der BI immer als Teil eines ganzheitlichen Unternehmensmodells verstanden werden muss. Über diese fachlich motivierten Annahmen hinaus können einem Datenmodell auch vordergründig willkürliche Annahmen bzw. *Konventionen* zugrunde liegen, deren Verletzung zu syntaktischen Fehlern führt. Beispielsweise könnte zur Analyse des Kaufverhaltens von Männern und Frauen u. a. das Geschlecht der Kunden in einer Kundendatenbank erhoben werden. Per Konvention sei angenommen, dass hierbei die Kodierung (männlich = 0, weiblich = 1) gelte. Der Datensatz (Mustermann | Max | 0 | ...) wäre somit eine korrekte Abbildung der Realität. Dagegen wäre der Datensatz (Mustermann | Max | 1 | ...) semantisch falsch, da 'Max Mustermann' nicht weiblich ist. Syntaktisch wäre der Datensatz jedoch korrekt, da er innerhalb des gegebenen Datenmodells eindeutig verstanden werden kann. Umgekehrt wäre der Datensatz (Mustermann | Max | m | ...) semantisch korrekt, unter der naheliegenden Annahme, dass m hier für 'männlich' stehen soll, er wäre jedoch syntaktisch falsch, da es für das m keine Interpretation im gegebenen Datenmodell gibt. Darüber hinaus wäre der Datensatz (Mustermann | Max | 2 | ...) sowohl syntaktisch als auch semantisch falsch, da Herr Mustermann weder in der Realität einem fiktiven dritten Geschlecht angehört, noch das Datenmodell die Kodierung eines solchen Geschlechts erlauben würde. Das bloße Weglassen des Wertes in der Form (Mustermann | Max | | ...) wiederum wäre ein Coverage-Fehler, da der Realitätsausschnitt „Geschlecht des Kunden“ hier gar nicht abgebildet wird.

Beispiel

Nachdem die drei Fehlerklassen allgemein und durch Beispiele beschrieben wurden, sollen mögliche Fehlerursachen und Fehlerarten im Detail diskutiert werden. Die Abgrenzung und Benennung der einzelnen Fehlerarten ist hierbei in der Literatur nicht immer eindeutig. Unter einer *Verschmutzung* von Daten kann zunächst ganz allgemein der Tatbestand verstanden werden, dass Daten semantisch von der Realität abweichen (Fonseca & Fieller, 2006). Wie im Absatz zu Fehlerklassen schon angerissen wurde, können die Ursachen dafür vielfältig sein. Im engeren Sinne soll hier daher dann von einer Verschmutzung gesprochen werden, wenn die Abweichung von der Realität entstanden ist, obwohl das korrekte Datum vorgelegen

Verschmutzung von Daten

hätte. Beispielsweise könnte eine Verschmutzung durch manuelle Falscheingabe oder (auch mutwillige) nachträgliche Veränderung von Daten auftreten.

„Noisy data“

Der Begriff „noisy data“, zu deutsch etwa „verrauschte Daten“ bezieht sich auf den informationstheoretischen Begriff des *Rauschens*. Intuitiv ist hierunter das Konzept zu verstehen, dass bei der Nachrichtenübermittlung, in der Regel durch physikalische Limitierung des Trägermediums, neben dem gewollten Nutzsignal auch Störsignale beim Empfänger ankommen. Die Gesamtheit der Störsignale kann als Rauschen oder Noise bezeichnet werden. Auf KDD bezogen, benennen Felici und Weitschek (2012) drei Arten von Noise, welche hier unter Beibehaltung der bisher verwendeten Bezeichnungen aufgeführt werden:

1. Instanzen, welche nicht zum betrachteten Universum gehören.
2. Attribute, welche keinen Aufschluss über den Untersuchungsgegenstand geben.
3. Attributwerte, welche falsch beobachtet oder gemessen wurden.

Beispiel

In diesem Sinne stellt jede Art der Verschmutzung auch Noise dar. Für die Einordnung als Noise ist es aber nicht von Bedeutung, ob der Datensatz grundfalsch, oder lediglich nicht von Interesse ist. Eine Anpassung an irrelevante Attribute etwa, ist unbedingt zu vermeiden. Zur Illustration soll folgendes vereinfachendes Beispiel dienen. Eine Bank will mittels KDD die Kreditwürdigkeit ihrer Kunden prüfen. Es erscheint naheliegend, die Umsätze der Kunden zu betrachten, um herauszufinden „wie gut sie mit Geld umgehen können“⁴. Die Umsatzdaten haben das folgende Format: (Kontonummer | Umsatz | Verwendungszweck | Neuer_Kontostand). Für einen menschlichen Betrachter ist es offensichtlich, dass die Kreditwürdigkeit eines Kunden nicht in sinnvoller Weise von der Kontonummer abhängen kann. Für ein DM-Verfahren ist dies jedoch nicht offensichtlich. Es ist möglich, dass ein DM-Verfahren ein rein zufälliges „Zusammenhangsmuster“ zwischen Kontonummer und Kreditwürdigkeit in den Daten entdeckt, welches sich naturgemäß nicht generalisieren lässt. Deshalb wäre in diesem Fall das Attribut **Kontonummer** als Noise zu betrachten und nicht zu berücksichtigen.

Statistischer Noise

Den bislang diskutierten Arten von Noise ist gemein, dass zumindest prinzipiell zwischen (Nutz-)Signal und Noise unterschieden werden kann, auch wenn die Unterscheidung im Einzelfall sehr aufwändig sein kann. Wenn dagegen Attributwerte von vornherein nicht richtig beobachtet oder gemessen werden können, und hier sind wieder auch Messfehler im weiteren Sinne gemeint, dann lag nie ein unverfälschtes Signal vor. In diesem Fall handelt es sich also weder um eine Verschmutzung im engeren Sinne, noch um reinen Noise, sondern die vorliegenden Daten bestehen aus einer Kombination von Signal und Noise. Es handelt sich also im Wortsinne um noisy data. Dennoch ist noisy data nicht per sé nutzlos. Typischerweise sind z.B. Messwerte durch Messfehler nicht in einem Maße verfälscht, das sie völlig unbrauchbar machen würde. Anschaulich wird dies anhand eines sehr verbreiteten Modells für Noise, dem Gauß'schen Noise. Hierbei gilt für reellwertige Objekte und Nachrichten:

⁴ Mögliche rechtliche Bedenken gegen ein solches Vorgehen seien für das Beispiel außen vor gelassen.

$$s_{i,j} = \mathbb{P}(m_j|z_i) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(\frac{-(m_j - z_i)^2}{2\sigma^2}\right) \quad (3.17)$$

Die Nachrichten folgen einer Normalverteilung mit dem tatsächlichen Objekt als Mittelwert und konstanter Standardabweichung (σ) (MacKay, 2003). Dargestellt veranschaulicht Abb. 7 verschiedene σ beispielhaft. Es ist erkennbar, dass der Informationsgehalt nicht völlig verloren ist, sondern die Daten in ihrer Gesamtheit durchaus dem richtigen Trend, hier einer Geraden, folgen. Eine solche Trendgerade lässt sich dann auch leicht per Regression bestimmen, hier beispielhaft für $\sigma = 0.25$. Grundsätzlich ist also festzuhalten, dass zwar einerseits die Datenbasis möglichst frei von allen Arten des Noise gehalten werden sollte, andererseits aber statistischer Noise in der Praxis so gut wie unvermeidbar ist und DM-Verfahren daher demgegenüber robust sein müssen, um trotzdem zu sinnvollen Modellen kommen zu können.

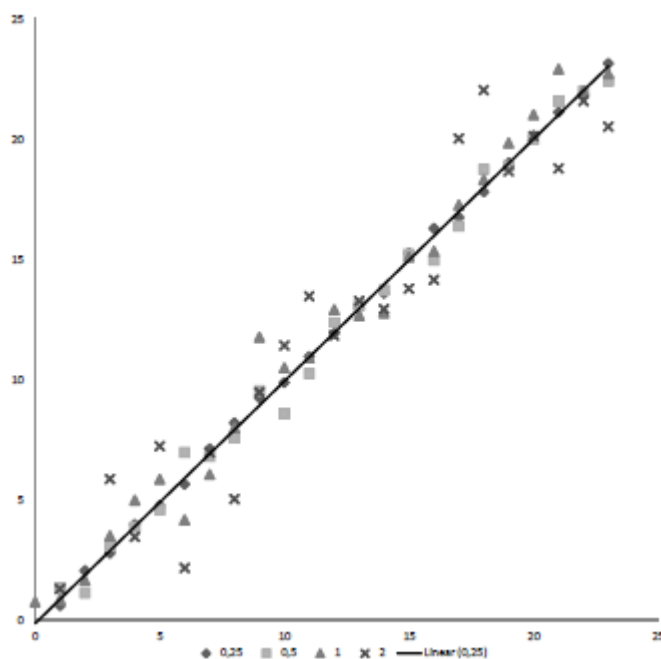


Abb. 7: Gauß'scher Noise mit $s = \{0.25, 0.5, 1.0, 2.0\}$ und Trendgerade

Ungültige bzw. fehlerhafte oder unzulässige Werte sind solche Werte, welche syntaktisch im gegebenen Datenmodell nicht zulässig sind, z. B. der Wert 2 für das Attribut Geschlecht wenn das Datenmodell die Kodierung (männlich = 0, weiblich = 1) vorsieht (Bramer, 2007). Ein weiteres Beispiel wäre die Eingabe einer vierstelligen Postleitzahl in Deutschland. Ein wesentliches Merkmal dieser Fehlerart ist, dass sie, im Gegensatz zu semantischen Fehlern, durch Konsistenzprüfungen automatisch erkannt werden kann (Mertens & Wiecek, 2000, S. 131). Dadurch ist es möglich, fehlerhafte Werte bis zu einem bestimmten Grad automatisch zu korrigieren. Verwendet das System z. B. als Datumsformat für das Geburtsdatum eines Kunden tt.mm.jjjj, die Eingabe liegt aber im Format t.mm.jj vor, so kann für Tag und Monat eine 0 sowie für Jahr eine 19 ergänzt werden (Bauer & Günzel, 2001, S. 88). Problematisch an dieser Vorgehensweise ist, dass auf diese Weise aus einem leicht zu erkennenden Syntaxfehler möglicherweise ein wesentlich schwerer zu erkennender semantischer Fehler wird. Etwa wenn der Benutzer sich schlicht vertippt hat (3.03.1955 statt 23.03.1955) oder wenn Kunden irgendwann auch nach 1999 geboren sein können (eine Reinkarnation des bekannten Millennium-Bugs). Besser ist

Unzulässige Werte

es daher, den Benutzer, falls möglich, direkt bei der Eingabe der Daten auf das Problem hinzuweisen.

Fehlende Werte

Eine weitere häufige Fehlerart sind Nullwerte bzw. fehlende oder unvollständige Werte. Insbesondere bei Umfragedaten und Daten zu den persönlichen Angaben eines Kunden bleiben manche Attribute leer, weil der Kunde keine Angabe machen möchte. Ein häufiges Beispiel ist das Einkommen eines Kunden. In diesem Fall sollte zumindest der „leere Eintrag“ standardisiert zur Auswahl gestellt werden, z. B. als „keine Angabe“. Es muss dann im Weiteren jedoch Sorge getragen werden, dass nicht fälschlicherweise ein, aller Wahrscheinlichkeit nach unsinniges, Muster bezüglich des leeren Eintrags gefunden wird. Die Aussage „Kunden mit dem Einkommen ‘keine Angabe’ kauften signifikant häufiger Produkt A als andere.“ ist z. B. wenig hilfreich. Wenn für ein Attribut kein Wert bekannt ist, bleiben prinzipiell wenig Möglichkeiten zu reagieren. Am aufwändigsten und gleichzeitig sichersten ist es, den wahren Wert des Attributs nachträglich herauszufinden, falls dies möglich ist. Im anderen Extrem können Instanzen mit fehlenden Attributwerten komplett ignoriert werden. Dies geht jedoch zu Lasten der verwertbaren Information und der Repräsentativität des Konzepts. Ein Mittelweg ist es, einen Ersatzwert zu bestimmen. Alle Möglichkeiten, einen solchen Ersatzwert anzugeben, führen aber zu einer Verzerrung der Daten. Die geringste Verzerrung wird erreicht, wenn jeweils derjenige Wert eingetragen wird, der, gegeben die anderen, vorhandenen Daten, am wahrscheinlichsten ist. Diesen Wert für alle Instanzen mit fehlenden Attributwerten zu ermitteln, ist aber ein DM-Problem für sich (Han & Kamber, 2006, S. 61f.).

Beispiel

Fehlende Werte sind häufig keine zufälligen Fehler. Beispielsweise können Kundenangaben fehlen, weil das Unternehmen nur wenig Kontakt zu diesem Kunden hat. Durch einen Ausschluss aller Kunden mit bestimmten fehlenden Merkmalsausprägungen wird dieses Kundensegment bei der Analyse nicht berücksichtigt. Möglicherweise wäre es jedoch gerade interessant, auch diese Kunden bei zukünftigen Entscheidungen und Analysen zu berücksichtigen. Ein weiterer Aspekt, der bei dieser Möglichkeit berücksichtigt werden muss, ist eine mögliche logische Ursache des fehlenden Attributwertes. So kann beispielsweise der Eintrag „Anzahl der Kinder“ bei kinderlosen Kunden fehlen oder aber das „Datum der Eheschließung“ bei unverheirateten Personen. Des Weiteren steigt bei größeren Datenbanken automatisch auch der Anteil von Instanzen mit fehlenden Attributwerten. Ein Ausschluss der betroffenen Attribute würde jedoch dazu führen, dass möglicherweise interessante Zusammenhänge nicht erkannt werden können (Hippner & Wilde, 2001).

Redundanz

Ein weiterer Fehler, der in der Datenhaltung häufig auftritt, ist die Redundanz der vorgehaltenen Daten. Dieser Fehler tritt meist dann auf, wenn unterschiedliche Datenbanken zur Haltung der gleichen Daten verwendet werden. In diesen Fällen sollten die Datenbanken miteinander abgeglichen und möglicherweise zusammengeführt werden (Bauer & Günzel, 2001, S. 91).

Unregelmäßigkeit

Die nicht-einheitliche Verwendung oder Unregelmäßigkeit von Attributeinträgen ist eine weitere Fehlerquelle. Beispielsweise betrage das Einkommen eines Kunden 5000 \$. Das entsprechende Attribut sei aber bezüglich seines Wertebereichs V_i in Tausend Euro skaliert. Trägt nun ein Mitarbeiter bei der Eingabe in das IS hier 5000 ein, so ergibt sich ein Fehler. Das Einkommen müsste für einen korrekten Eintrag zunächst von Dollar in Euro umgerechnet und durch 1000 dividiert werden (Müller & Freytag, 2003).

Tab. 3: Zuordnung der Fehlerarten zu Fehlerklassen

Fehlerart	Fehlerklasse		
	semantisch	syntaktisch	coverage
Verschmutzungen	•		
Noisy Data	•		
Unzulässige Werte		•	
Fehlende Werte			•
Unvollständige Werte			•
Redundanz	•		
Unregelmäßigkeiten		•	

Tab. 3 zeigt die Zuordnung von Fehlerarten zu Fehlerklassen. So vielfältig wie die Fehlerarten sind auch die Fehlerursachen und Abhilfemaßnahmen. Es können jedoch nicht alle Fehler im Nachhinein behoben werden. Deshalb sollte nach Möglichkeit Datenqualitätsproblemen aktiv vorgebeugt werden, indem häufige Fehlerursachen im IS behoben werden. Fehlerhafte Daten sollten möglichst früh im Datenverarbeitungsprozess korrigiert werden. Zum einen ist der Suchraum dann kleiner und der Fehler kann schneller gefunden werden, zum anderen muss der aufgetretene Fehler lediglich einmal direkt am Ursprung korrigiert werden, sodass der gleiche Fehler im Folgenden nicht wieder auftritt. Dies kann ein entscheidender Kostenfaktor sein (Apel, Brehme, Eberlein & Merighi, 2010). In vielen Unternehmen ist die Problematik der fehlerhaften Daten jedoch so groß, dass die Fehlerbehebung hinsichtlich verschiedener Kriterien priorisiert werden muss. Nach Zwirner (2008) gibt es acht verschiedene Beurteilungskriterien, welche im Folgenden dargestellt werden.

Beurteilung von Fehlern

Jedes Unternehmen muss individuell entscheiden, welche Daten eine hohe Bedeutung haben, d. h. wo sich Fehler in diesen Daten besonders negativ auswirken. Zu berücksichtigende Konsequenzen können beispielsweise Imageschäden, ein Akzeptanz- und Vertrauensverlust oder aber Fehlentscheidungen sein.

Bedeutung der Daten

Es ist ebenfalls wichtig, die Grundlage der Anforderungen an die Datenqualität zu betrachten, da sich hieraus unterschiedlich starke Konsequenzen ergeben können. Es lassen sich z. B. gesetzliche Anforderungen, aufsichtsrechtliche Anforderungen, strategische Entscheidungen sowie projektspezifische Anforderungen unterscheiden.

Anforderungen an die Datenqualität

Fachliche Fehler sind Fehler, welche die fachliche Richtigkeit der Daten betreffen. Alle zuvor genannten Fehlerarten können sich als fachliche Fehler äußern.

Natur der Fehlerursache

Technische Fehler betreffen meist die Logik in Datenbanken bzw. werden durch das technische IS verursacht. Fachliche Fehler müssen innerhalb der Daten behoben werden, bei technischen Fehlern ist in der Regel das technische IS an sich zu verbessern.

Je nach Fehlerart und -häufigkeit sind verschiedene Bereinigungsstypen sinnvoll. Generell lassen sich zwei Bereinigungsarten unterscheiden: die *manuelle* und die *maschinelle* Bereinigung. Die manuelle Bereinigung ist immer dann sinnvoll, wenn eine geringe Fehlerzahl vorliegt oder keine einfachen Regeln zur Fehlerbehebung vorhanden sind. Die maschi-

Art der möglichen Bereinigung

nelle Bereinigung bietet sich bei einer sehr großen Anzahl von Fehlern an, wenn Fehler nach spezifizierbaren Regeln automatisiert korrigiert werden können. Zusätzlich sind Mischformen der Bereinigung möglich.

- | | |
|--|---|
| Änderungshäufigkeit der Daten | Aus der Änderungshäufigkeit der Daten lässt sich schließen, ob die gesammelten Daten in kürzeren Zeitabständen bereinigt werden müssen oder ob die Zeitabstände größer sind. Bei relativ statischen Daten ist der Aufwand einer seltenen oder auch nur initialen Bereinigung eher vertretbar als bei dynamischen Daten. Mögliche Maßnahmen müssen darauf abgestimmt werden. |
| Anzahl der Datenfehler | Aus der Anzahl der Datenfehler lässt sich ableiten, welche Bereinigungsart oder welcher zeitliche Rhythmus sinnvoll ist. Generell lässt sich die Aussage treffen, dass eine hohe Anzahl an Datenfehlern ein größeres Problem darstellt, der positive Effekt einer Datenbereinigung jedoch auch größer ist. Trotzdem muss die Anzahl immer im Kontext gesehen werden, d. h. die Auswirkung der Fehler und entsprechend der Bereinigung muss immer mit betrachtet werden. |
| Maßnahmen zur Vermeidung neuer Fehler | In der Praxis wird der Fokus häufig auf die Bereinigung von bereits begangenen Fehlern gelegt. Es kann jedoch, gerade bei häufigen Fehlerarten, zielführender sein, Maßnahmen zur Vermeidung von Wiederholungen dieser Fehler einzuführen. |
| Aufwand für Maßnahmen | Bei jeder Datenbereinigung sollte der Aufwand dem gewonnenen Nutzen gegenübergestellt und abgewogen werden. Dies kann komplex sein, etwa wenn ein Fehler sich häufig wiederholt, der sich daraus ergebende Schaden jedoch relativ gering ist, wohingegen eine Bereinigung der Ursache einen großen Aufwand mit sich bringen würde. |
| Umgang mit Fehlern | Im Umgang mit Fehlern ist ein Abwägen aller zuvor genannten Kriterien erforderlich. Dem Entscheider stehen im Wesentlichen drei Möglichkeiten zur Verfügung, wie Fehler gehandhabt werden können. Bei Fehlern, die selten auftreten oder aber keine schwerwiegenden Konsequenzen zur Folge haben, kann das „ <i>Laissez Faire</i> “-Prinzip angewendet werden. Es werden keine Maßnahmen zur Bereinigung des Fehlers durchgeführt, da der Bereinigungsaufwand größer wäre als der Nutzen der Bereinigung. Die zweite Möglichkeit ist das <i>reaktive Vorgehen</i> . Dabei wird ein Fehler, wenn er entdeckt wird, bereinigt, jedoch werden keine Maßnahmen eingeführt, die das zukünftige Auftreten des gleichen Fehlers verhindern sollen. Es wird davon ausgegangen, dass, sofern der Fehler ein nächstes Mal auftritt, dieser erneut entdeckt und behoben werden kann. Die Lösung ist somit nicht nachhaltig und es besteht das Risiko, dass ein Fehler gar nicht erst erkannt wird. Die letzte Möglichkeit zum Umgang mit fehlerhaften Daten ist das <i>proaktive Vorgehen</i> . Bei diesem Vorgehen werden Fehler initial bereinigt und Maßnahmen zur künftigen Vermeidung und Überwachung der betreffenden Fehlerquellen eingeführt. Dies entspricht dem proaktiven DQM (vgl. Einheit 3), ist jedoch auch sehr aufwändig und daher nicht für jede Fehlerquelle wirtschaftlich sinnvoll (Zwirner, 2008). |

3.4 Projektion und Reduktion von Attributen

Attribute und Attributwerte

Nachdem im vorherigen Abschnitt 3.3 die Bereinigung von Fehlern auf der Ebene von Konzept und Instanzen im Vordergrund stand, soll nun eine Betrachtung auf der Ebene einzelner Attribute vorgenommen werden. Aufbauend auf den Definitionen in Abschnitt 3.2 soll dabei zunächst der Unterschied zwischen Attribut und Attributwert präzisiert werden. Zur Wiederholung: Ein Konzept X ist ein *zweidimensionaler Datenbestand* (3.14). Die *Zeilen* von X repräsentieren dabei einzelne Instanzen $x_i \in M$ (3.15), welche wiederum jeweils ein Objekt $o_j \in U$ abbilden. Die *Spalten* von X repräsentieren die *Werte* von jeweils einem Attribut in einem gegebenen Wertebereich V_i (3.16). Viele Anwendungen setzen voraus, dass $V_i \subseteq \mathbb{R}$, bzw. $M \subseteq \mathbb{R}^{N_Q}$. Dann ist x_i ein Vektor und wird als *Featurevektor* bezeichnet (Suarez-Alvarez, Pham, Prostov & Prostov, 2012). Über diese bereits dargestellten Zusammenhänge hinaus soll zur besseren Unterscheidung von Attributen und Attributwerten eine Menge der Attribute (Q) und eine Informationsfunktion (ψ) definiert werden, so dass gilt (Cios et al., 1998, S. 30):

$$Q = \{q_1, q_2, \dots, q_{N_Q}\} \quad (3.18)$$

$$V = \bigcup_{k \in \{1, \dots, N_Q\}} V_k \quad (3.19)$$

$$\psi: M \times Q \rightarrow V \quad (3.20)$$

$$\psi(x_i, q_j) = x_{i,j} \in V_j \quad (3.21)$$

Verkürzt bezeichne V_q mit $q \in Q$ den Wertebereich eines bestimmten Attributs, wo es nicht auf dessen Index ankommt und analog $x_{i,q}$ den Wert des Attributs q der Instanz x_i (i -te Zeile von X).

Für jedes Attribut $q \in Q$ kann ein Skalenniveau angegeben werden. Gemeinhin werden vier Niveaustufen unterschieden. Bei *nominal*/skalierten Attributen können die einzelnen Attributwerte zwar eindeutig und überschneidungsfrei unterschieden werden, andere Aussagen über die Beziehung der Attributwerte zueinander sind jedoch nicht möglich. Ein Beispiel wäre die Waagennummer von Gemüse im Supermarkt. *Ordinal*/skalierte Attribute erlauben es zusätzlich, die Attributwerte in eine Rangfolge zu bringen. So könnte etwa die Beliebtheit einer Gemüsesorte mit $V_{\text{Beliebtheit}} = \{\text{hoch}, \text{mittel}, \text{niedrig}\}$ erfasst werden. Zum Vergleich: Die Aussage, dass ein Gemüse in der Rangfolge der Beliebtheit vor einem anderen steht, hat eine sinnvolle Bedeutung. Die Aussage, dass ein Gemüse eine höhere Waagennummer als ein anderes hat, hat eine solche Bedeutung in aller Regel nicht. Die nächste Niveaustufe ist die *Intervallskala*. Auf dieser lassen sich zusätzlich zu den vorgenannten Eigenschaften auch Differenzen zwischen den einzelnen Attributwerten bilden. Beispielsweise lässt sich berechnen, dass das Verbrauchsdatum einer bestimmten Charge von Gemüse drei Tage später ist, als das eines anderen. Es lässt sich jedoch nicht analog berechnen, wieviel beliebter ein hochbeliebtes Gemüse im Vergleich zu einem mittelbeliebten ist. Eine *Verhältnisskala* zeichnet sich schließlich zusätzlich dadurch aus, dass es einen natürlichen Nullpunkt der Attributwerte gibt. Nur dann ist die Bildung von Verhältnissen sinnvoll definiert. Die Aussage „Eine Lieferung von vier Kisten Paprika ist doppelt so groß, wie eine Lieferung von zwei Kisten Paprika.“ ist sinnvoll. Die Aussage „Eine Lieferung, die bis zum 2. Mai verbraucht werden muss, ist doppelt so lange haltbar wie eine, die bis zum 1. Mai verbraucht werden muss.“ dagegen ist nicht sinnvoll (Yang, Webb & Wu, 2010).

Skalenniveau

Attribute welche nominal oder ordinal skaliert sind, haben *qualitativen* Charakter. Sie werden oft auch als *kategorische* Attribute bezeichnet, da die Attribut-

Qualitative und quantitative Attribute

werte diskrete Kategorien abbilden, auf denen keine arithmetischen Operationen zulässig sind. Attribute mit einer Intervall- oder Verhältnisskala sind *quantitativ* und natürlicherweise *numerisch*. Anders als bei qualitativen Attribut, kann der Wertebereich eines quantitativen Attribute entweder *diskret* oder *kontinuierlich* sein. Es ist immer möglich, unter Informationsverlust von einem höheren auf ein niedrigeres Skalenniveau zu wechseln. Der Vorgang quantitative Attribute unter möglichst geringem Informationsverlust in qualitative Attribute umzuwandeln wird – eigentlich irreführend – oft als *Diskretisierung* bezeichnet (Yang et al., 2010). Einige Verfahren, wie z. B. das EB-Verfahren (vgl. Abschnitt 4.3), erzeugen automatisch qualitative Attribute. Die meisten DM-Verfahren jedoch erwarten mindestens numerische Attribute auf einer Intervallskala. Darüber hinaus wird oft angenommen, die einzelnen Attribute seien jeweils normalverteilt. Es ist zwar ohne weiteres möglich, kategorische Attribute numerisch abzubilden, dadurch wird jedoch *kein* höheres Skalenniveau erreicht (Suarez-Alvarez et al., 2012).

Bedeutung des Skalenniveaus und der Verteilung

Um Einschätzen zu können, ob ein bestimmtes DM-Verfahren sinnvoll eingesetzt werden kann, muss demnach bekannt sein, welche Anforderungen an Skalenniveau und Verteilung der Attribute das entsprechende Verfahren hat, und ob die vorliegenden Attribute diese erfüllen. Während eine Nichterfüllung der formalen Voraussetzungen die theoretischen Eigenschaften der Verfahren zunichte macht, können in der Praxis oft dennoch gute Ergebnisse erzielt werden. Dies ist im Einzelfall zu prüfen. Insbesondere können viele negative Effekte durch eine geschickte Umkodierung der Attribute gemildert werden. Dies ist unter anderem Thema des vorliegenden Abschnitts.

Abschätzung der Verteilung

Die Vorstellung von statistischen Methoden zur Abschätzung der Verteilung einzelner Attribute würde an dieser Stelle zu weit führen. Stattdessen soll hier dem Vorschlag von Tukey (1977) gefolgt werden, der anregt, für einen ersten Überblick fünf einfache Kennzahlen heranzuziehen und sie in dem von ihm erfundenen *Boxplot* Format darzustellen. Die Kennzahlen haben zudem den Vorteil, dass ihre Bestimmung lediglich ein ordinales Skalenniveau voraussetzt. Sei X ein Konzept und $q \in Q$ ein Attribut; ohne Beschränkung der Allgemeinheit (o. B. d. A.) sei X gemäß der (ordinalen) Rangfolge von q sortiert, d. h. es gelte:

$$i < j \Rightarrow x_{i,q} \leq x_{j,q} \quad (3.22)$$

Mit dieser Sortierung und $0 \leq p \leq 1$ sei das p -Perzentil von q definiert als

$$x_{[p \cdot (N_X + 1)], q} \quad (3.23)$$

Das p -Perzentil von q ist somit derjenige Attributwert, für den der Anteil von Instanzen, bei denen q den gleichen oder einen geringeren Wert bezüglich der Rangfolge einnimmt, p beträgt (Han & Kamber, 2006, S. 54).

In Anlehnung an Tukey (1977, Kap. 2) geben, unter Beibehaltung der Sortierung, fünf Attributwerte einen Überblick über q :

- $x_{1,q}$ entspricht dem *Minimum* von q über alle Zeilen von X .
- $x_{N_X,q}$ entspricht dem *Maximum* von q über alle Zeilen von X .
- Das 0.5-Perzentil entspricht dem *Median* von q .

- Das 0.25-Perzentil, auch *erstes Quartil* genannt.
- Das 0.75-Perzentil, auch *drittes Quartil* genannt.

Je zentraler der Median zwischen dem ersten und dem dritten Quartil liegt, desto symmetrischer die Verteilung von q .

In Fortführung der eingangs genannten Beispiele aus dem Alltag soll die Problematik eines unzureichenden Skalenniveaus illustriert werden. Es werden verschiedene Gemüsesorten betrachtet, dies könnte z. B. bei einer BI-Anwendung im Lebensmitteleinzelhandel eine Rolle spielen. Für das Beispiel sei

Motivierendes Beispiel

$$V_{\text{Gemüse}} = \{\text{Paprika}, \text{Tomate}, \text{Gurke}, \text{Kopfsalat}\} \quad (3.24)$$

d. h. im betrachteten U existieren nur Paprika, Tomate, Gurke und Kopfsalat als mögliche Werte eines Attributs Gemüse. Wenn ein DM-Verfahren numerische Attribute voraussetzt, ist zunächst nicht eindeutig, wie Gemüse numerisch zu repräsentieren ist. Es gäbe die Option es einfach fortlaufend zu nummerieren, oder z. B. vorhandene Waagen- oder Artikelnummern zu verwenden. Diese Nummerierung wäre recht willkürlich und könnte etwa so aussehen:

- (rote) Paprika (53)
- Tomate (54)
- Gurke (55)
- Kopfsalat (56)

Nun wird das Problem offenkundig: wenn das Verfahren nicht nur eine Nominalskala, sondern eine Intervallskala voraussetzt und eine Differenz zwischen den Werten bildet, wäre ohne sachlichen Grund die Differenz einer Paprika und einer Tomate kleiner als die einer Gurke und einer Tomate.

Tab. 4: Beispiel für Eins-aus-N Kodierung

Attribut	q_1	q_2	q_3	q_4
(rote) Paprika	1	0	0	0
Tomate	0	1	0	0
Gurke	0	0	1	0
Kopfsalat	0	0	0	1

Wenn nichts weiter über ein nominal skaliertes Attribut bekannt ist, so ist es sinnvoll, davon auszugehen, dass alle Ausprägungen voneinander unabhängig sind. Wenn die möglichen Attributwerte in ihrer Anzahl überschaubar sind, so kann eine unabhängige Kodierung dadurch erreicht werden, dass für jeden Attributwert ein neues Attribut eingeführt wird, welches genau dann den Wert 1 hat wenn das ursprüngliche Attribut den Wert

Eins-aus-N Kodierung

hatte, der diesem neuen Attribut zugeordnet ist, und 0 sonst. Für N Attributwerte werden also ebenfalls N Ersatzattribute benötigt, von denen aber immer nur eines von 0 abweicht. Daher der Name Eins-aus- N Kodierung (Pyle, 1999, Kap. 6.1.1). Die Kodierung im o. g. Beispiel ist in Tab. 4 angegeben, wobei $q_1 \dots q_4$ die Ersatzattribute bezeichnen. Dabei spielt die Reihenfolge, in der die Ersatzattribute den ursprünglichen Attributwerten zugeordnet werden, keine Rolle, solange sie fix bleibt.

n -aus- N Kodierung Ein wesentlicher Nachteil der Eins-aus- N Kodierung ist, dass dadurch potentiell sehr viele Ersatzattribute entstehen können, welche zudem in jeder Instanz größtenteils den Wert 0 haben. Dies ist weder in Hinblick auf Speicherplatzbedarf, noch in Hinblick auf den Rechenaufwand der verschiedenen DM-Algorithmen effizient. Häufig ist es möglich, mit zusätzlichem Wissen über U die Anzahl der benötigten Ersatzattribute geringer als die Anzahl der ursprünglichen Attributwerte zu wählen. Von N Ersatzattributen können dann beliebig viele (n) von 0 abweichen, so ergibt sich der Name n -aus- N Kodierung. Dieses Vorgehen ist vor allem dann sinnvoll, wenn dadurch wesentlich weniger Ersatzattribute benötigt werden und diese zusätzlich sinnvolle Information tragen (Pyle, 1999, Kap. 6.1.2). Die Anwendung auf das obige Beispiel ist daher eher als Illustration denn als realistisches Beispiel zu verstehen. Die Grundidee ist es, binäre Merkmale zu finden, welche die verschiedenen Attributwerte voneinander differenzieren. Die Gemüse im Beispiel liegen in zwei Farben vor: rot und grün. Darüber hinaus lassen sich Gurke und Paprika eher als bissfest bezeichnen, während Tomate und Kopfsalat eher weich sind. Die sich somit ergebene Kodierung zeigt Tab. 5 In der Praxis sollte sichergestellt sein, dass die gewählte Kodierung sachlich zur betrachteten Fragestellung gehört. So sollte ein Analyst z. B. zusätzlich überprüfen, ob Kunden ihr Gemüse tatsächlich aufgrund der genannten Attribute auswählen.

Tab. 5: Beispiel für n -aus- N Kodierung

Attribut	rot	bissfest
(rote) Paprika	1	1
Tomate	1	0
Gurke	0	1
Kopfsalat	0	0

Dummy-Kodierung Die Dummy-Kodierung, sowie auch die im nächsten Absatz vorgestellte Effekt-Kodierung, haben ihren Ursprung in der Regressions- und Varianzanalyse und finden insbesondere dort Anwendung, wo mit Test- und Kontrollgruppen gearbeitet wird, z. B. in der Medizin oder in der Bildungsforschung (Alkharusi, 2012). Die Dummy-Kodierung ist der Eins-aus- N Kodierung konzeptionell sehr ähnlich, und hat ihren Namen daher, dass die oben sogenannten Ersatzattribute quasi als Attrappen (engl. Dummies) stellvertretend für das ursprüngliche Attribut stehen. Der wesentliche Unterschied zur Eins-aus- N Kodierung besteht darin, dass für N Attributwerte nur $N - 1$ Dummyattribute eingeführt werden und einer der ursprünglichen Attributwerte dadurch repräsentiert wird, dass alle Dummyattribute den Wert 0 annehmen. Crone, Lessmann und Stahlbock (2006) sprechen daher auch von $N - 1$ Kodierung. Die Dummy-Kodie-

rung bietet sich vor allem dann an, wenn es einen ausgezeichneten Attributwert gibt, im Verhältnis zu dem alle anderen Attributwerte betrachtet werden sollen. Dies liegt bei Existenz einer Referenzgruppe naturgemäß vor (Alkharusi, 2012).

Im ständigen Beispiel dieses Abschnittes gibt es einen solchen ausgezeichneten Wert zunächst nicht. Es muss je nach Anwendungsfall entschieden werden, ob die Verwendung der Dummy-Kodierung möglich ist. Bei einer Untersuchung, bei welcher die Beliebtheit des Gemüses eine Rolle spielt, ließe sich beispielsweise argumentieren, dass die Tomate als beliebtestes Gemüse in Deutschland einen geeigneten Bezugspunkt für die anderen Gemüse darstellt. Es ergäbe sich somit die Kodierung wie in Tab. 6.

Tab. 6: Beispiel für Dummy-Kodierung

Attribut	q_1	q_2	q_3
(rote) Paprika	1	0	0
Tomate	0	0	0
Gurke	0	1	0
Kopfsalat	0	0	1

Die Effekt-Kodierung ist nahezu identisch mit der Dummy-Kodierung und unterliegt den gleichen Voraussetzungen. Ihr Name rührt daher, dass sie im Rahmen der Regressionsanalyse den Effekt der einzelnen Attributwerte auf die abhängige Variable sichtbar macht (Alkharusi, 2012). Der einzige Unterschied zwischen Effekt-Kodierung und Dummy-Kodierung ist, dass alle Ersatzattribute für den Referenzwert des ursprünglichen Attributs mit -1 anstelle von 0 kodiert werden. Dies hat zur Folge, dass bei Gleichverteilung der verschiedenen Attributwerte alle Ersatzattribute in der Summe über alle Instanzen 0 ergeben und somit auch der Erwartungswert (μ) 0 ist. Dies kann auch außerhalb der Regressionsanalyse eine wünschenswerte Eigenschaft sein. Ein Beispiel für die Effekt-Kodierung zeigt Tab. 7.

Effekt-Kodierung

Tab. 7: Beispiel für Effekt-Kodierung

Attribut	q_1	q_2	q_3
(rote) Paprika	1	0	0
Tomate	-1	-1	-1
Gurke	0	1	0
Kopfsalat	0	0	1

Neben nominal skalierten Attributen fallen unter die Klasse der kategorischen Attribute auch solche, die ordinal skaliert sind. Für diese Attribute ist, anders als bei nominal skalierten Attributen, eine Kodierung in fortlaufenden dezimalen Nummern, eher vertretbar. Bestehen bleibt das Problem, dass diese numerische Kodierung suggeriert die Differenzen zwischen jeweils zwei benachbarten Werten seien stets gleich, was bekanntlich eine Intervallskala erfordern würde. Daher empfiehlt es sich, wenn die Anzahl verschiedener möglicher Attributwerte gering ist, ähnlich wie bei der Eins-aus-N Kodierung für jeden

Unäre Kodierung ordinalisierbarer Attribute

Attributwert ein Ersatzattribut einzuführen, dabei aber die Kodierung so zu wählen, dass die Attributvektoren betragsmäßig größer werden. Dies entspricht konzeptionell einer unären Kodierung, manchmal auch bildhaft Thermometer Kodierung genannt (Crone et al., 2006). Ein Beispiel für die unäre Kodierung zeigt Tab. 8.

Tab. 8: Beispiel für unäre Kodierung

Attribut	q_1	q_2	q_3
niedrig	1	0	0
mittel	1	1	0
hoch	1	1	1

Kodierung numerischer Attribute

Auch Attribute welche ohnehin schon quantitativ-numerisch skaliert sind können trotzdem unterschiedliche Skalen verwenden. Dies führt bei vielen DM-Verfahren dazu, dass Attribute mit, absolut gesehen, großen Werten solche mit kleinen Werten dominieren. Um dieses Problem zu umgehen, gibt es zwei Verfahren, Standardisierung und Normalisierung, welche nun in Anlehnung an Larose (2005, S. 36f.) beschrieben werden sollen.

Standardisierung

Wenn begründet angenommen werden kann, dass ein Attribut $q \in Q$ zwar annähernd normalverteilt aber nicht standardnormalverteilt ist, dann empfiehlt es sich das Attribut so zu standardisieren, dass $\mu = 0$ und $\sigma = 1$ wird. Dies erfolgt nach der Formel:

$$x'_{i,q} = \frac{x_{i,q} - \mu}{\sigma} \quad (3.25)$$

Häufig wird davon ausgegangen, dass X einer zufälligen Stichprobe von Nachrichten entspricht. Dann kann μ durch das *Stichprobenmittel* ($\bar{x}_q$) geschätzt werden:

$$\bar{x}_q = \frac{1}{N_X} \sum_{i=1}^{N_X} x_{i,q} \quad (3.26)$$

$$\mathbb{E}(\bar{x}_q) = \mu \quad (3.27)$$

Entsprechend wird σ als Wurzel aus der Stichprobenvarianz (s^2) geschätzt:

$$s^2 = \frac{1}{N_X - 1} \sum_{i=1}^{N_X} (x_{i,q} - \bar{x}_q)^2 \quad (3.28)$$

Normalisierung

Wenn keine Normalverteilungsannahme getroffen werden kann, empfiehlt sich eine Normalisierung. Dabei wird der Wertebereich eines Attributs auf ein gewünschtes Intervall, i. d. R. $[0, 1]$, verkleinert, indem die folgende Formel angewendet wird:

$$x'_{i,q} = \frac{x_{i,q} - \text{minimum}(q)}{\text{maximum}(q) - \text{minimum}(q)} \quad (3.29)$$

Auch wenn eine Standardisierung oder Normalisierung der Daten in der Praxis unumgänglich ist und häufig zu akzeptablen Ergebnissen führt, sollten sich Analysten und Entscheider dennoch

darüber im Klaren sein, dass es sich, je nach Art des Attributs, in den meisten Fällen um eine gänzlich arbiträre Operation handelt.

Das Attribut, welches normiert werden soll, sei das Einkommen. Der maximale Wert über alle Instanzen in X betrage 10000, der minimale Wert liege bei 500.

Beispiel

Für eine konkrete Instanz x_k sei $x_{k,\text{Einkommen}} = 3500$. Es muss dann gerechnet werden:

$$x'_{k,\text{Einkommen}} = \frac{3500 - 500}{10000 - 500} = \frac{3000}{9500} \approx 0.32 \quad (3.30)$$

Die Auswahl von Attributen bzw. das Verwerfen von Attributen, also eine Attributsreduktion, anhand von bestimmten Kriterien ist in vielen Fällen sinnvoll.

Attributauswahl

Durch die Selektion von Attributen können bessere Ergebnisse erzielt werden, denn irrelevante Attribute können zu Verzerrungen und Verschiebungen bei den Ergebnissen des DM führen. Zudem führt eine geringere zu bearbeitende Datenmenge natürlich auch zu kürzeren Rechenzeiten (Freitas, 2002, Kap. 4). Da die Attributauswahl in einige DM-Verfahren bereits integriert ist, in anderen Fällen aber auch gänzlich eigene DM-Verfahren erfordert, soll hier nicht umfassend darauf eingegangen werden. Domingos (2012) weist darauf hin, dass eine korrekte Attributauswahl eine „schwarze Kunst“ sei, die zu großen Teilen Domänenspezifisch ist und keinen klaren Regeln folgt.

Zwei grundsätzliche Vorgehensweisen zur Attributauswahl sind der Filteransatz und der Wrapperansatz. Der Filteransatz basiert auf einer einfachen Funktionsweise.

Filter- und Wrapperansatz

Unabhängig vom verwendeten DM-Algorithmus wird versucht, die relevanten Attribute auszuwählen und die übrigen Attribute herauszufiltern. Dieser Ansatz ist jedoch problembehaftet. Beispielsweise enthalte X zu einzelnen Patienten verschiedene medizinische Daten, eine Diagnose und die Krankenversicherungsnummer des Patienten. Eine Analyse wird ergeben, dass auch die Diagnose variiert, wenn die Versicherungsnummer variiert wird. Demnach scheint die Versicherungsnummer relevant für die Diagnose zu sein. Schlimmer noch: Wenn zu jedem Patienten nur der letzte Krankheitsfall betrachtet wird, reicht die Versicherungsnummer alleine bereits aus, um die Diagnose eindeutig zu bestimmen. Ein fachlicher Zusammenhang existiert jedoch nicht. Der Wrapperansatz berücksichtigt daher auch den jeweiligen DM-Algorithmus. Der Algorithmus wird als Blackbox betrachtet und wiederholt auf einen kleinen Teil von X angewendet, wobei die Attributmenge $Q' \subset Q$ variiert wird. Dasjenige Q' mit der besten Performance auf diesem kleinen Teil von X wird dann für das weitere DM verwendet (Kohavi & John, 1997). Das Problem des Wrapper Ansatzes ist jedoch, dass die gemessene Performance ggf. nicht aussagekräftig ist (vgl. S. 62). Alternative Verfahren sind Lineare Diskriminanzanalyse (LDA) und Hauptkomponentenanalyse (PCA) (vgl. Tab. 10).

4 Phase II: Data Mining

Begriffsdefinition und Rolle im KDD-Prozess

In Abschnitt 2.1 wurde bereits die Begriffsüberschneidung zwischen DM und KDD dargestellt. Gemäß der Definition des KDD-Prozesses in Abschnitt 2.2 umfasst DM die zweite Meta-Phase des KDD-Prozesses und damit die zentralen Prozessschritte. DM bedeutet aus dem Englischen übersetzt etwa „Schürfen nach Daten“. Han und Kamber (2006, S. 5) weisen darauf hin, dass der Begriff eigentlich irreführend ist, da nicht nach Daten „geschürft“ wird. Dies würde suggerieren, dass ein besonderes Interesse an den Daten oder dem „Abbau“ neuer Daten vorhanden wäre, tatsächlich gilt das Interesse des KDD-Prozesses – der somit weniger irreführend benannt ist – der Gewinnung von Wissen und nicht von Daten. Bei der Betrachtung von DM als Schritt im KDD-Prozess ist es vielmehr naheliegend, eine Definition von DM zu wählen, die mit der Definition von KDD (vgl. Abschnitt 2.1) kompatibel ist. Eine solche prozessorientierte Definition liefert Chattamvelli (2009):

„Data Mining is the process of extracting hitherto unknown and potentially useful patterns, trends, anomalies and rules from stored historical data for business promotion, decision making and classification.“ (Chattamvelli, 2009, S. 13)

So verstanden, fügt sich DM nahtlos in den KDD-Prozess ein. Es handelt sich im Wesentlichen um eine Konkretisierung der KDD-Definition.

Inhalte des Kapitels

Im vorliegenden Kapitel stehen, ausgehend vom KDD-Prozess, zunächst die Grundaufgaben im Vordergrund, welche mittels DM-Algorithmen gelöst werden können. In einem nächsten Schritt, wird dargestellt, wie eine begründete Entscheidung für einen DM-Algorithmus getroffen werden kann. Schließlich wird die Ausführung verschiedener grundlegender DM-Algorithmen, das DM im engeren Sinne, genauer beleuchtet.

4.1 Auswahl der Aufgabe

Abhängigkeit vom Erkenntnisziel

Welche Grundaufgabe des DM vorliegt bzw. ausgewählt werden sollte ist offensichtlich abhängig vom Erkenntnisziel des KDD-Projekts. Erste Anhaltspunkte darüber, welche Erkenntniszielen welchen Aufgaben entsprechen, geben die Anwendungsbeispiele zu Beginn dieser Einheit in Abschnitt 1.1. Die Auswahl der Aufgabe steht im prototypischen KDD-Prozess, welcher dieser Einheit zugrunde liegt, auf den ersten Blick an einer ungewöhnlich späten Stelle. Der Vorteil dieser Herangehensweise ist jedoch, dass der Analyst sich zunächst, ohne Rücksicht auf ein bestimmtes Verfahren, nach sachlichen Kriterien vergegenwärtigt, welche Daten und Attribute vorliegen, in welchem Verhältnis diese zum zugrundeliegenden Entscheidungsproblem stehen, was die Attribute durch welche Kodierung abbilden und wie das Verhältnis zwischen den Attributen ist. Fragestellungen dieser Art sind, neben der technischen Aufbereitung, das ordnende Prinzip des Kapitel 3. Somit sollte zum Zeitpunkt der Auswahl der Aufgabe schon viel Domänenwissen über das Erkenntnisziel implizit in das Konzept bzw. dessen Kodierung eingeflossen sein. Dem gegenüber steht der Nachteil, dass ggf. zu einem vorherigen Schritt zurückgekehrt werden muss, um aufgabenspezifische Vorbereitungen zu treffen. Daher ist der KDD-Prozess grundsätzlich nicht als strikt linear zu verstehen.

In der Literatur herrscht weitgehende Einigkeit über die Grundaufgaben des DM, welche sich aus den Grundaufgaben des KDD direkt ergeben (vgl. S. 8).

Grundaufgaben von DM

Die folgende Einteilung findet sich so oder ähnlich z. B. in (Linoff & Berry, 2011, S. 85f.), (Chattamvelli, 2009, S. 15) oder (Han & Kamber, 2006, S. 21ff.). Demnach sind die vier Grundaufgaben, welche durch DM-Methoden gelöst werden können:

- Assoziationsanalyse
- Clusteranalyse
- Klassifizierung
- Approximation, Vorhersage oder Regression

Bei der Assoziationsanalyse, oder auch Affinitätsgruppierung, ist das Ziel die Überprüfung, welche Attributwerte eines Attributs innerhalb einer Instanz häufig in Verbindung mit welchen Attributwerten eines anderen Attributs auftreten. Der häufigste Anwendungsfall ist die Analyse des Kaufverhaltens, etwa von Supermarktkunden. Das Ziel der Analyse ist es, herauszufinden, welche Artikel ein Kunde zusammen mit einem oder mehreren anderen Artikeln kauft (Linoff & Berry, 2011, Kap. 15). Etwas formaler ausgedrückt geht es darum, logische Abhängigkeiten zwischen den einzelnen Attributen einer Instanz bzw. den zwischen den Eigenschaften des zugrundeliegenden Objekts zu erkennen. Die per Assoziationsanalyse erzeugten Modelle werden daher häufig in der Form logischer „wenn-dann“ Regeln notiert (Chattamvelli, 2009, S. 165f.). Eine resultierende Regel könnte dann z. B. die Form Kunde kauft Chips $\Rightarrow$ Kunde kauft Cola haben.

Assoziationsanalyse

Das Ziel der Clusteranalyse ist es, eine heterogene Grundgesamtheit in mehrere verschiedene, in sich homogene Gruppen, sogenannte Cluster, zu unterteilen.

Clusteranalyse

Dabei sind die verschiedenen Cluster vorab nicht definiert und ergeben sich erst im Laufe der Anwendung (Han & Kamber, 2006, S. 26). Das resultierende Modell besteht dabei sowohl aus den Clustern als auch aus der Zuordnung von Datensätzen zu Clustern. Clusteranalyse kann z. B. angewendet werden, wenn ein Unternehmen seine Kunden in verschiedene Käufertypen unterteilen möchte aber die Struktur der Käufertypen nicht im Vorhinein kennt.

Ziel der Klassifizierung ist es, eine Zuordnungsvorschrift (Funktion) zu finden, welche Instanzen aufgrund ihrer jeweiligen Attributwerte zu Klassen zuordnet.

Klassifizierung

Dazu muss eine Menge *diskreter* Klassen vorab in ihrer Art und *endlichen* Anzahl bekannt sein, *und* es muss bekannt sein, zu welcher Klasse in dieser Menge die einzelnen Instanzen in X jeweils gehören. Die resultierende Zuordnungsvorschrift kann dann verwendet werden, um *neue* Instanzen, deren Klasse noch nicht bekannt ist, einer Klasse zuzuordnen (Hastie et al., 2001, Kap. 2). Formal setzt dies voraus, dass sich Q in zwei disjunkte Teilmengen zerlegen lässt, d. h. $Q = Q_f \cup Q_g$. Ziel ist es dann, für eine gegebene Instanz anhand der Attribute in Q_f die Attribute in Q_g zu bestimmen. Letztere werden daher auch *Zielattribute* (engl. Target) oder *Klassenattribute* genannt. Häufig ist Q_g einelementig, d. h. $Q_g = \{q_g\}$ (Linoff & Berry, 2011, S. 96). Nach obigen Ausführungen muss V_{q_g} diskret und endlich sein. Typischerweise sollte auch V_{q_g} nur wenige Elemente haben, häufig sogar nur zwei. Dies wird als *binäre Klassifizierung* bezeichnet.

Abgrenzung von Clusteranalyse und Klassifizierung

Sowohl bei der Clusteranalyse als auch bei der Klassifizierung geht es letztlich um die Zuordnung von Instanzen zu einzelnen Gruppen, nämlich Clustern bzw. Klassen. Bei der Clusteranalyse sind die Daten bekannt und auch eine Zuordnungsvorschrift ist prinzipiell bekannt, nämlich „Ordne die Daten so zu, dass die Cluster in sich möglichst homogen und untereinander möglichst heterogen sind“. Nicht bekannt ist dagegen die Zielgröße, d. h. Semantik und Anzahl der Cluster sowie die „tatsächliche“ (im Sinne einer korrekten Beschreibung von U) Zuordnung der einzelnen Instanzen zu diesen Clustern. Bei der Klassifizierung dagegen sind Semantik und Anzahl der Klassen sowie die tatsächliche Zuordnung für *einige* Instanzen bekannt. Die Zuordnungsvorschrift ist dagegen unbekannt, d. h. reine Ähnlichkeit ist an dieser Stelle, anders als bei der Clusteranalyse, *kein* Argument. Ziel ist es, eine Zuordnungsvorschrift zu finden, welche auch auf Instanzen angewendet werden kann, deren Klasse noch nicht bekannt ist.

Approximation

Diese letzte Grundaufgabe wird in der Literatur unterschiedlich benannt, wobei keine der Bezeichnungen vollständig präzise ist und jede eine andere Sicht auf denselben Sachverhalt ausdrückt. Es soll im Folgenden vor allem der Begriff der Approximation verwendet werden. Die Darstellung der verschiedenen Sichten orientiert sich an (Han & Kamber, 2006, S. 285ff.) und den bereits in diesem Abschnitt vorgestellten Überlegungen. Approximation kann als Verallgemeinerung der Klassifizierung auf unendlich viele, oft kontinuierliche, Klassen verstanden werden. Diese Interpretation fasst die Attributwerte einer Instanz für Attribute in Q_f als vektorielle unabhängige Variable (Featurevektor) auf, hier kurz x_{i,Q_f} genannt. Die abhängige Variable dagegen ist der Vektor der Attribute in Q_g , hier kurz x_{i,Q_g} . Annahme ist nun, Noise außen vor gelassen, dass ein funktionaler Zusammenhang besteht, sodass gilt:

$$\forall i: x_{i,Q_g} = h(x_{i,Q_f}) \quad (4.1)$$

Die Funktion $h(\cdot)$ ist unbekannt und soll *approximiert* werden. Beispielsweise kann ein Unternehmen versuchen, die Kundenbindungsdauer zu approximieren, indem verschiedene Einflussfaktoren wie Alter und Geschlecht eines Kunden mit unterschiedlicher Gewichtung auf das Zielattribut Kundenbindungsdauer einwirken.

Vorhersage oder Regression

Der Begriff der Vorhersage als alternative Bezeichnung für Approximation ist dagegen mehrdeutig. Der Vorgang der Klassifizierung kann z. B. ebenfalls als Vorhersage interpretiert werden: aufgrund der Attributwerte der Attribute in Q_f wird eine Vorhersage über den unbekannten Wert des Klassenattributs in Q_g getroffen. Tatsächlich ist der Begriff Vorhersage im Sinne der Approximation ganz ähnlich zu verstehen, nur dass nicht eine diskrete nominale Klasse vorhergesagt wird, sondern ein numerischer Wert. Vorhersage ist hier also als Kurzform von *numerischer* Vorhersage gemeint. Regressionsanalyse (vgl. Abschnitt 4.4) ist ein häufig benutztes Werkzeug, um die Grundaufgabe der Approximation in der Praxis zu lösen. Deshalb wird auch der Begriff Regression teilweise synonym mit dem Begriff Approximation verwendet (Han & Kamber, 2006, S. 286).

4.2 Auswahl des Algorithmus

DM-Methoden sind häufig entweder statistische Methoden oder Methoden aus dem Bereich maschinelles Lernen (ML), wobei die Grenze zwischen diesen beiden Gebieten fließend ist (Witten et al., 2011, S. 5 & S. 29). Die zunehmende Konvergenz der beiden Gebiete äußert sich auch darin, dass etablierte ML-Algorithmen statistisch in aller Regel sehr gut untersucht sind. Die statistische Lerntheorie bildet die formale Grundlage für die Untersuchung von ML-Algorithmen (Bousquet, Boucheron & Lugosi, 2004). Viele Erkenntnisse daraus sind in der Praxis schon in die bekannten DM-Algorithmen integriert. Statistische Lerntheorie und ML bilden dennoch den theoretischen Hintergrund für die Auswahl eines konkreten Algorithmus. Die Diskussion in diesem Abschnitt ist von solchen Erwägungen geleitet.

Abschnitt 3.3 liefert eine Einführung in die parallel zu berücksichtigenden Modelle der Datengenerierung. Vorgreifend wurde festgehalten, dass DM-Algorithmen es ermöglichen sollen, zu einem gegebenen Konzept X ein angenommenes Modell der Datengenerierung zu bestimmen (vgl. S. 26). Die klassische statistische Perspektive auf diesen Sachverhalt zeichnet sich dadurch aus, dass zunächst in einem informellen Verfahren eine Modellklasse $\mathcal{H}$ bestimmt wird in der das ideale Modell der Datengenerierung mutmaßlich liegt und dann methodisch nach demjenigen konkreten Modell $h \in \mathcal{H}$ gesucht wird, welches X bestmöglich abbildet. Häufig unterscheiden sich die einzelnen Modelle in $\mathcal{H}$ dabei nur durch einen Modellparameter (θ). (Hosking, Pednault & Sudan, 1997; Fonseca & Fieller, 2006).

Statistische Perspektive

Während ML versucht, diesen Vorgang zu automatisieren, trachtet die statistische Lerntheorie danach, diesen Vorgang zu formalisieren (Bousquet et al., 2004).

Gemäß der Definition zu Beginn dieses Kapitels (vgl. S. 40). besteht der Input von DM-Algorithmen aus „historischen“ Daten, also Daten, die in der Vergangenheit entstanden sind. Der Output gemäß Definition sind „Muster, Trends, Anomalien und Regeln“. Witten et al. (2011, S. 39) dagegen sprechen davon, dass ausgehend vom Konzept eine „Beschreibung des Konzepts“ erlernt werden soll. Der statistischen Perspektive folgend, soll hier vereinfachend davon gesprochen werden, dass der Output letztlich die Form eines Modells $h \in \mathcal{H}$ annimmt. Wie im vorherigen Abschnitt dargestellt, kann es sich bei h z. B. um eine Funktion oder um Regeln handeln. Das automatisierte Schließen von Daten auf Modelle wird in der konkreten Durchführung im Rahmen des ML häufig als *Training* bezeichnet. So gewonnene Modelle können dann verwendet werden um Vorhersagen zu treffen. Es geht also beim DM stets darum, aus *vergangenheitsbezogenen* Daten Wissen zu extrahieren, dessen Kenntnis *gegenwärtige* und *zukünftige* Entscheidungsprozesse unterstützen soll, auch wenn dieser Grundsatz ist zwar in den verschiedenen Grundaufgaben unterschiedlich offensichtlich ist.

Modellbildung bzw Training

Intuitiv scheint es sinnvoll, alle Daten in X für das Training zu verwenden. Tatsächlich ist dies jedoch *nicht* das optimale Vorgehen. Unabhängig davon, wie die Performance von DM-Algorithmen gemessen wird, ist die Performance auf historischen Daten nicht die interessante Größe. Ein Grund dafür ist z. B. der Umstand, dass sich alle Instanzen in X durch Nachschlagen in der zugrundeliegenden Tabelle trivial korrekt klassifizieren lassen, ohne dass dies einen Erkenntnisgewinn brächte. Ein weiterer Grund ist, dass sich U verändern könnte, nachdem die Daten erhoben wurden. Von Interesse ist also die Performance auf Daten, welche zum Zeitpunkt des Trainings nicht zur Verfügung standen. Die Fähigkeit von ML-Algorithmen, aus

Generalisierung

bekannten Daten Modelle für unbekannte Daten zu erzeugen, wird Generalisierung genannt (Domingos, 2012; Wolpert, 1996).

Trainings- und Testdaten

In Begriffen aus Abschnitt 3.3 kann die Forderung nach Generalisierung damit begründet werden, dass KDD an einem angenommenen Modell von U interessiert ist und nicht an einem Modell von X . Da die Performance einerseits in der Praxis logischerweise nur auf Daten getestet werden kann, welche auch vorliegen, andererseits für aussagekräftige Tests Daten erforderlich sind, die nicht im Training verwendet wurden, muss X aufgeteilt werden (Domingos, 2012). Derjenige Teil von X der für das Training verwendet wird, bildet die sogenannten Trainingsdaten, der verbleibende Teil entsprechend die Testdaten (Chattamvelli, 2009, S. 13). Da das Training unbedingt *unabhängig* von den Testdaten sein muss, ist die Aufteilung vorzunehmen, *bevor* ein Algorithmus ausgewählt wird. Die Testdaten dürfen erst wieder zur Evaluation desjenigen Algorithmus verwendet werden, welcher *unwiderruflich* ausgewählt wurde. Dies ist Inhalt von Kapitel 5.

Überwachtes und unüberwachtes Lernen

Ein wichtiges Kriterium zur Auswahl eines Algorithmus ergibt sich aus der zu lösenden Grundaufgabe. ML-Verfahren gliedern sich in überwachtes und unüberwachtes Lernen. *Überwachtes Lernen* liegt dann vor, wenn die zu lernende Zielgröße innerhalb der Trainingsdaten bekannt ist, das Lernergebnis also in Hinblick auf den Fehler im Vergleich zur Zielgröße überwacht werden kann. Dies ist gemäß Abschnitt 4.1 bei Klassifizierung und Approximation der Fall. Der Erfolg der jeweiligen DM-Algorithmen kann während des Trainings *formal* überwacht werden, indem h auf die Attribute Q_f der einzelnen Trainingsinstanzen angewendet und das Ergebnis mit den tatsächlichen Werten der Attribute in Q_g verglichen wird. Bei *unüberwachtem Lernen*, z. B. der Clusteranalyse, ist dagegen über die Strukturen im Datenbestand im Vorhinein nichts sicher bekannt (Hastie et al., 2001, Kap. 2 & Kap. 14).

Fachliche Voraussetzung für überwachtes Lernen

Die Möglichkeit, ein Verfahren zu überwachen, impliziert nicht, dass es bezüglich des Erkenntnisziels einen Beitrag leistet. Der Einsatz überwachter Verfahren bietet sich vor allem dann an, wenn die Attribute in Q_g prinzipbedingt erst zu einem *späteren Zeitpunkt* erhoben werden können als die in Q_f . Die wirtschaftliche Situation eines Kunden wird z. B. bei der Kreditvergabe erhoben. Ob der Kredit dagegen vollständig zurückgezahlt wurde, wird erst zum Ende der Laufzeit bekannt. In so gelagerten Fällen liefert h schon zum Zeitpunkt der Messung von Q_f eine *Vorhersage* für das spätere Q_g . Durch die feste zeitliche Abfolge ist außerdem klar, dass das spätere Ereignis nicht ursächlich für das frühere sein kann. Mehr Vorsicht ist dagegen geboten, wenn die Erhebung der Attribute in Q_f und Q_g gleichzeitig erfolgt. So könnten Bankkunden mit einem geringen Kontostand auf dem Tagesgeldkonto z. B. mit erhöhter Wahrscheinlichkeit ein Depotkonto führen. Der Ursache-Wirkungszusammenhang wäre aber umgekehrt zu vermuten: Der geringe Kontostand beim Tagesgeld besteht, *weil* es ein Depotkonto gibt, welches potentiell mehr Rendite verspricht. Die mögliche Schlussfolgerung, nun allen Kunden mit niedrigem Kontostand gezielt ein Depotkonto anzubieten, wäre daher fatal (Linoff & Berry, 2011, S. 158f.).

Komplexe Modellklassen

Wie zu Beginn dieses Abschnitts beschrieben, besteht der klassische statistische Ansatz im Wesentlichen darin, zunächst ein H festzulegen und dann dasjenige θ zu finden, mit dem H die Daten in X bestmöglich abbildet. Die Entscheidung für ein H fällt dabei oft auf Basis dessen, was die Erfahrung des Analysten oder die Domäne naheliegend erscheinen lassen. Grundsätzlich ist diese Vorgehensweise auch für das DM empfehlenswert und

einfache statistische Modelle sollten, wann immer möglich, als Referenzpunkt für den Vergleich von Algorithmen herangezogen werden. ML-Algorithmen gehen über den klassischen statistischen Ansatz hinaus, indem sie auch solche H zulassen, die keine Entsprechung in einem einfachen statistischen Modell haben. Dabei kommen in der Regel mehrere θ in Betracht, sodass zusätzlich das Problem zu lösen ist, Modelle zu vergleichen, welche sich nicht nur durch ein θ voneinander unterscheiden (Hosking et al., 1997).

Obiger Argumentation folgend, ist die Beschaffenheit der Modellklasse, die ein gegebener ML-Algorithmus repräsentieren kann, ein wichtiges Merkmal, um verschiedene ML-Algorithmen miteinander zu vergleichen. Das Ziel einer solchen Repräsentation ist es, Hypothesen über Zusammenhänge in den Daten in eine maschinenlesbare Form zu bringen. In diesem Zusammenhang wird H daher auch *Hypothesenraum* genannt. Mögliche Formen der Repräsentation sind z. B. einzelne Instanzen, Hyperebenen, EBs und KNN (Domingos, 2012). Schwerer zu beurteilen, aber oft nicht minder wichtig, ist die Frage, wie gut die verschiedenen maschinenlesbaren Modelle von Menschen gelesen und verstanden werden können (Jammer-negg, Luptáčík, Nakhaeizadeh & Schnabl, 1998).

Repräsentation von Modellklassen

Aus den bisherigen Ausführungen ergibt sich ein fundamentales Problem für die Auswahl eines geeigneten DM-Algorithmus. Unabhängig davon, welche Vergleichskriterien bei der Auswahl zugrunde gelegt werden, können diese nur auf vergangenheitsbezogenen Daten gemessen werden. Die Testdaten kommen hierfür nicht in Frage, da diese nur zur Evaluierung eines bereits ausgewählten Algorithmus verwendet werden dürfen. Auch die Trainingsdaten kommen nicht ohne weiteres in Frage, da eine Validierung mithilfe der Trainingsdaten jede gelernte „Eigenheit“ der Trainingsdaten bestätigen wird. Tatsächlich besteht aber das Risiko, auch solche Eigenheiten der Trainingsdaten zu lernen, welche sich nicht generalisieren lassen. Letzteres wird als *Overfitting* bezeichnet (Domingos, 2012). Um dennoch schon während der Auswahl- und Trainingsphase einen Eindruck von der Performance der DM-Algorithmen zu bekommen, bieten sich zwei Auswege aus diesem Dilemma an: Es können die Trainingsdaten erneut unterteilt werden. Der abgetrennte Teil, die *Validierungsdaten*, wird dann zur Validierung im Training verwendet (Chattamvelli, 2009, S. 13). Eine Alternative, die häufig im DM zum Einsatz kommt, ist die *Kreuzvalidierung* (Domingos, 2012; Hosking et al., 1997). Dabei werden die Trainingsdaten in k gleich große und disjunkte Teile $X_1 \dots X_k$ geteilt. Für alle $i \in \{1 \dots k\}$ wird die Performance des Algorithmus auf X_i gemessen, wobei nur die *anderen* $(k - 1)$ Teile zum Training verwendet werden. Die Ergebnisse dieser k -vielen Messungen werden gemittelt.

Validierung im Training

Die Vorstellung, was konkret unter einer „großen“ Datenmenge zu verstehen ist, ändert sich mit fortschreitender Entwicklung (vgl. S. 2). Dennoch war und ist es zu jeder Zeit ein Ziel von DM, mit großen Datenvolumina umgehen zu können (Hosking et al., 1997; Lusti, 2002; Klein et al., 2013). Die Komplexität der Datensätze rührt dabei nicht nur von der reinen Anzahl der Instanzen, sondern auch vom gemeinsamen Auftreten kategorischer und kontinuierlicher Attribute (vgl. S. 33 sowie der hohen Dimensionalität der Daten, d. h. der Höhe von N_Q (Hosking et al., 1997; Domingos, 2012). Vor diesem Hintergrund muss die Ergebnisqualität einzelner DM-Algorithmen ins Verhältnis zu deren Laufzeit gesetzt werden (Jammer-negg et al., 1998). Wenn die Menge der verfügbaren Daten relativ zur verfügbaren Rechenkapazität hoch ist, muss zudem zwischen dem Einsatz einfacher Algorithmen auf einer großen Datenmenge und dem Einsatz komplexer Algorithmen auf einer kleineren Auswahl der Daten abgewogen werden (Domingos, 2012).

Einsatz auf großen Datenmengen

Occam's Razor

Unter dem Namen Occam's Razor ist ein traditionelles Prinzip der Wissenschaft zu verstehen, nach dem einfache Theorien bei gleicher Erklärungskraft gegenüber komplexen Theorien zu bevorzugen sind. Auf DM bezogen: Sei $h_1, h_2 \in H$. Wenn h_1 und h_2 U gleich gut erklären, dann ist das einfachere Modell zu bevorzugen. Die Erklärungskraft bezüglich U ist aber nicht messbar. Häufig findet sich daher eine Variante, die sinngemäß besagt: Wenn h_1 und h_2 X gleich gut erklären, dann ist davon auszugehen, dass das einfachere Modell U besser erklärt. Die zweite Aussage ist in dieser Tragweite theoretisch nicht haltbar. Es gibt empirische Beispiele, in denen eine Vereinfachung des Modells tatsächlich zu besserer Generalisierung führt – etwa Pruning im EB-Verfahren (vgl. Abschnitt 4.3) – jedoch lassen sich ebenso relevante Gegenbeispiele konstruieren. In der Praxis sollte daher, wie schon erwähnt, die Bevorzugung einfacher Modelle zusätzlich durch „handfeste“ Erwägungen wie Laufzeitkomplexität der Algorithmen und Interpretierbarkeit der Ergebnisse geleitet werden (Domingos, 1999).

No Free Lunch Theorem

Alle zuvor genannten Kriterien können gemeinsam als Anhaltspunkte für die Auswahl eines DM-Algorithmus dienen. In der Praxis wird dies häufig zu akzeptablen Ergebnissen führen. Es ist jedoch ein zentrales Ergebnis der statistischen Lerntheorie, dass auch ein solches Vorgehen keine Garantien für Erfolg geben kann. Unter dem Namen „No free lunch⁵ theorem“, konnte Wolpert (1996), umgangssprachlich formuliert, die folgenden Aussagen beweisen:

- Es gibt keinen allgemein besten ML-Algorithmus. Für je zwei Algorithmen A, B, lassen sich genauso viele Fälle angeben, in denen A besser generalisiert, wie solche, in denen B besser generalisiert. Ein Vergleich ist daher nur möglich, wenn die Phänomene, welche auftreten können, a priori begrenzt sind.
- Es gibt kein allgemeines Verfahren zur Auswahl von ML-Algorithmen. Die erste Aussage gilt auch dann, wenn A ein Metaalgorithmus ist, der per Kreuzvalidierung aus mehreren Algorithmen denjenigen mit der *höchsten* Performance auf den Trainingsdaten wählt und B denjenigen Algorithmus mit der *niedrigsten* Performance auf den Trainingsdaten wählt.
- Es gibt keinen festen Zusammenhang zwischen Trainings- und Testperformance.
- Aus einer guten Trainingsperformance eines Modells mit formal geringer Komplexität (Occam's Razor) folgt *nicht* automatisch eine gute Testperformance.

Es folgt daraus, dass es beim DM immer notwendig ist, Annahmen über die zu erwartenden Phänomene und den Zusammenhang zwischen historischen und zukünftigen Daten zu treffen. Solche Annahmen können nie aus den Daten selbst gewonnen werden, sondern sie erfordern vom Entscheider zusätzliches Wissen über die Art und Struktur der Daten sowie die zu lösende Aufgabenstellung (Bousquet et al., 2004).

Übersicht über die vorgestellten Algorithmen

Wu et al. (2008) haben ein „Top 10“ der DM-Algorithmen aufgestellt, welche von der Forschungsgemeinschaft als die einflussreichsten angesehen werden. Es handelt sich dabei um Algorithmen mit einem breiten Einsatzgebiet, welche

⁵ Zu Deutsch etwa „man bekommt nichts geschenkt“.

in der Praxis gute Ergebnisse liefern. Viele der Algorithmen sind erstaunlich einfach und daher auch didaktisch gut als Beispiele geeignet. Tab. 9 und Tab. 10 geben zunächst einen Überblick über lineare Verfahren der klassischen Statistik.

Zunächst wird Ihnen ein Überblick über die „Top 10“ der DM-Algorithmen sowie einige zusätzliche Algorithmen gegeben. Alle hier und im Folgenden genannten Algorithmen eignen sich als Referenzpunkt zur Beurteilung komplexerer oder domänenspezifischer Verfahren. Ausgewählte Algorithmen und Verfahren werden in den Abschnitten 4.3–4.8 dann genauer vorgestellt.

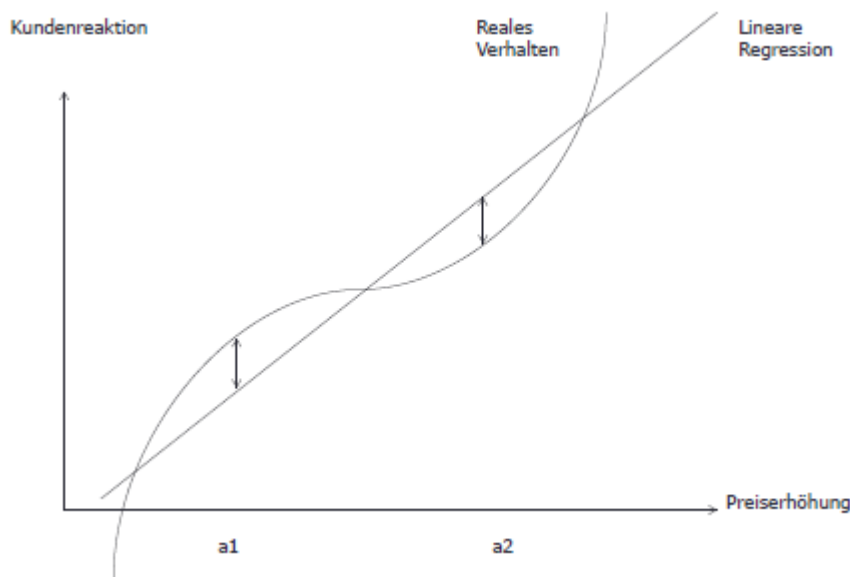


Abb. 8: Beispiel für die lineare Regression des nicht-linearen Zusammenhangs zwischen Preiserhöhung und Kundenreaktion.

Quelle: in Anlehnung an (Krahl et al., 1998)

Tab. 9: Übersicht zu Regressionsverfahren

Algorithmen:	Lineare Regression, logistische Regression
Anwendung:	statistische Modellierung
Repräsentation:	Koeffizientenvektor
Modellkomplexität:	linear
Optimierungskriterium:	Schätzung von $\mathbb{P}(h X)$ (least squares, log-likelihood)
Interpretierbarkeit:	einfache Modelle, jedoch in hohen Dimensionen nicht anschaulich

Regressionsverfahren kommen aus der klassischen Statistik bzw. den Anfängen der elektronischen Datenverarbeitung. Es sollen hier nur solche Vertreter berücksichtigt werden, bei denen H auf lineare Modelle beschränkt ist. Dies können z. B. lineare Funktionen zur Approximation oder eine lineare Trennung (Gerade, Ebene, Hyperebene) zwischen den Instanzen zweier Klassen sein. Eine solche Funktion oder Hyperebene lässt sich durch einen Vektor von Koeffizienten repräsentieren. Da die Modelle so einfach sind, ist die Gefahr von Overfitting als extrem gering einzuschätzen. Lineare Regressionsverfahren stellen somit kaum Anforderungen an die Beschaffenheit der Daten. Da allerdings die wenigsten Zusammenhänge in der Praxis auch wirklich linear sein werden, ist es oft notwendig, die Daten vor Anwendung eines linearen Verfahren einer nicht-linearen Transformation zu unterziehen. Auch die Auswahl der Attribute muss im Vorfeld, ggf. auf Basis anderer

statistischer Verfahren, getroffen werden. Regressionsverfahren folgen der statistischen Perspektive und versuchen, die Wahrscheinlichkeit des Modells gegeben die Daten $P(h|X)$ zu maximieren (Hastie et al., 2001, Kap. 3–5). Lineare Modelle lassen sich aufgrund ihrer geringen Komplexität häufig gut interpretieren. Oft können die Koeffizienten als Gewichte verstanden werden, welche die relative Bedeutung des jeweiligen Attributs repräsentieren. Der Versuch, das Ergebnis graphisch anschaulich darzustellen muss jedoch in hohen Dimensionen scheitern. Didaktische Beispiele beschränken sich daher oft auf zwei Dimensionen. Abb. 8 zeigt ein solches Beispiel zu den Grenzen des Verfahrens. Die Stärke der Kundenreaktion soll in Abhängigkeit von der Höhe einer Preissteigerung approximiert werden. In der Realität liege kein linearer Zusammenhang vor. So führen beispielsweise kleinere Preiserhöhungen zu relativ geringer Kundenreaktion wohingegen höhere Preissteigerungen zu einer überproportionalen Kundenreaktion führen würden. Bei einer Preiserhöhung in Höhe von a_1 liegt die mittels linearer Regression geschätzte Stärke der Kundenreaktion deutlich unterhalb des realen Wertes. Bei einer Preiserhöhung in Höhe von a_2 ist der reale Wert deutlich unterhalb des geschätzten Wertes.

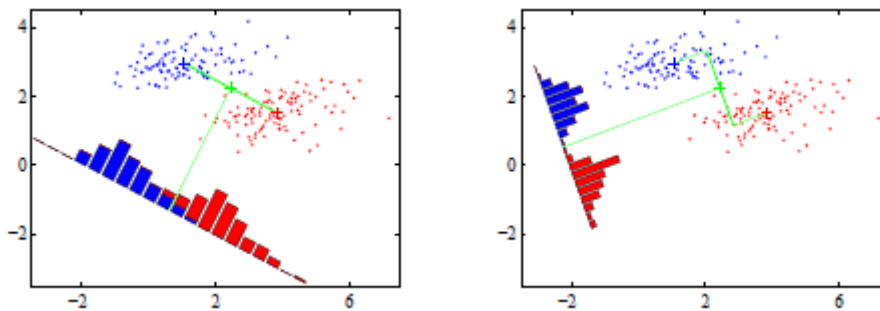


Abb. 9: Graphische Darstellung der LDA: Featurevektoren werden aus einem hochdimensionalen Raum (hier: zweidimensional) in einen niedrigdimensionalen Raum (hier: eindimensional) projiziert. Das Histogramm stellt dabei die Anzahl der Instanzen dar, deren Bild in den entsprechenden Abschnitt der Geraden fällt. Die Instanzen fallen im Beispiel in zwei Klassen. Die linke Abb. zeigt die Projektion welche nur die Varianz zwischen den beiden Klassen maximiert. In der rechten Abb. wurde zugleich die Varianz innerhalb der jeweiligen Klasse minimiert. Es kommt zu weniger Überlappungen und die Klassen lassen sich somit insgesamt besser trennen. Quelle: entnommen aus (Bishop, 2006, Kap. 4)

Tab. 10: Übersicht zu Projektionsverfahren

Algorithmen:	PCA, LDA
Anwendung:	Dimensionsreduktion
Repräsentation:	Projektionsmatrix
Modellkomplexität:	lineare Abbildung
Optimierungskriterium:	Varianz
Interpretierbarkeit:	Linearkombinationen der Attribute werden sichtbar

Projektionsverfahren sind, wie Regressionsverfahren, Methoden der klassischen Statistik. Sie adressieren das Problem der Dimensionsreduktion, d. h. sie haben das Ziel, N_Q zu reduzieren. Auch hier sind lineare Verfahren aufgrund ihrer geringen Komplexität besonders beliebt. Wie für viele andere Verfahren auch, gilt für PCA und LDA, dass ihre Anwendung nur unter bestimmten Bedingungen theoretisch begründbar ist, aber in der Praxis auch dann gute Ergebnisse erzielt werden können, wenn diese Bedingungen verletzt sind. Als minimale Anforderung soll hier davon ausgegangen werden, dass es sich bei den Instanzen x_i , als Zeilen von X , um Vektoren quantitativer Attribute handelt. Es gilt nun, eine lineare Abbildung von $\mathbb{R}^{N_Q}$ nach $\mathbb{R}^{N_Q'}$ zu finden, für die gilt:

$N_{Q'} < N_Q$. Eine solche Abbildung wird durch Multiplikation mit einer Matrix $\mathbf{W} \in \mathbb{R}^{N_Q \times N_{Q'}}$ geleistet:

$$\mathbf{x}'_i = (\mathbf{x}_i - \boldsymbol{\mu})\mathbf{W}$$

o. B. d. A. kann angenommen werden, dass $\mathbf{X}$ mittelwertfrei gemacht wurde, und Mittelwertvektor $\boldsymbol{\mu}$ somit der Nullvektor ist. Die offene Frage ist dann, wie $\mathbf{W}$ zu bestimmen ist. Die PCA geht hierbei davon aus, dass die ursprünglichen Daten möglichst gut repräsentiert werden sollen. Dies ist dann der Fall, wenn die Varianz in den Daten bestmöglich erhalten bleibt bzw. äquivalent dazu, der quadratische Fehler der Rücktransformation minimiert wird. Es handelt sich um ein unüberwachtes Verfahren. Die LDA dagegen ist ein überwachtes Verfahren. Die Grundidee ist hier, die Daten so zu projizieren, dass die *Klassen* im Bildraum *Cluster* bilden, d.h. dass im Bildraum die Varianz zwischen den Klassen möglichst groß, aber innerhalb der Klassen möglichst klein wird (Abb. 9). Im Grenzfall $N_{Q'} = 1$ ist LDA gleichzeitig ein einfacher und effektiver Klassifizierungsalgorithmus, wenn die Attribute näherungsweise normalverteilt sind (Runkler, 2010, S. 37f u. 91f). Die einzelnen Attribute im Bildraum ($q' \in Q'$) sind Linearkombinationen der ursprünglichen Attribute. Eine Analyse dieser Linearkombinationen kann in einigen Fällen aufzeigen, welche der ursprünglichen Attribute konzeptionell verwandt sind.

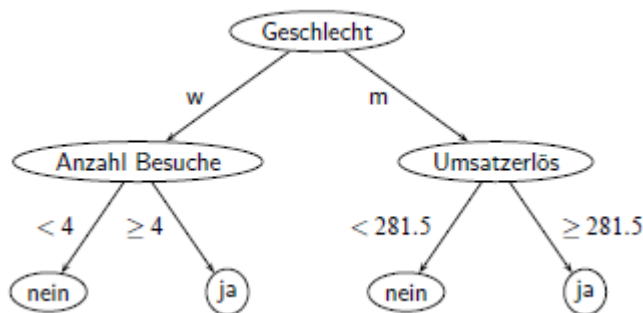


Abb. 10: Beispielhafter EB: Kunden des US-Einzelhändlers HomeStuff werden danach klassifiziert, ob sie „best customers“ sind. Dazu werden die Attribute Geschlecht, Anzahl der Besuche und Umsatzerlös der jeweiligen Kunden herangezogen.

Quelle: in Anlehnung an (Ville, 2006)

Tab. 11: Übersicht zu EB-Verfahren

Algorithmen:	Classification and Regression Tree (CART), C4.5
Anwendung:	überwachtes Lernen
Repräsentation:	EB
Modellkomplexität:	parameterfrei
Optimierungskriterium:	lokal (Gini-Koeffizient, Informationsgewinn, ...)
Interpretierbarkeit:	gut für Menschen lesbar, jedoch abhängig von der Baumgröße

Die Bezeichnung „EB-Verfahren“ leitet sich von der Repräsentation des Modells als EB ab. EBs lassen sich leicht visualisieren, was zumindest kleine Bäume gut für Menschen lesbar macht (vgl. Abb. 10). EB-Verfahren stellen nur geringe Anforderungen an die Aufbereitung der Daten (vgl. Abschnitt 3.3). Transformationen der Attribute sind nicht erforderlich und auch die Selektion relevanter Attribute ist bereits Teil dieser Verfahren. Auch gegenüber Ausreißern und fehlenden

Werten sind EB-Verfahren robust (Shmueli, Patel & Bruce, 2010, S. 187f.). EBs sind nicht-linear und parameterfrei. Das bedeutet, dass EBs prinzipiell geeignet sind, um komplexe Zusammenhänge zu modellieren. Sie können in ihrer Komplexität mit den Anforderungen des betrachteten Konzepts wachsen. Die wesentliche Einschränkung besteht darin, dass die gängigen Algorithmen die Daten nur lokal und univariat betrachten (Giuffrida, Chu & Hanssens, 2000). Im Beispiel (Abb. 10) werden zunächst die gesamten Trainingsdaten betrachtet. Die beste Einteilung der Kunden ist an dieser Stelle nach dem Attribut Geschlecht. Diese Entscheidung fällt aber lokal. Ob dies auch global zum optimalen EB führt, kann nicht garantiert werden. Das EB-Verfahren kann Auskunft darüber geben, dass in Abhängigkeit von der Unterteilung nach einem Attribut weitere Unterteilungen nach unterschiedlichen Attributen sinnvoll sind. Dies ist jedoch sorgfältig von der *gleichzeitigen* Berücksichtigung einer Attributkombination zu unterscheiden, welche von EB-Verfahren nicht geleistet wird. Diese Grenzen sind jedoch nicht per sé als Schwäche des EB-Verfahrens zu bewerten. Es folgt aus dem No free lunch Theorem, dass H begrenzt sein muss, um überhaupt Generalisierung zu ermöglichen. Zudem wirken sich die genannten Grenzen äußerst günstig auf die Laufzeitanforderungen aus und machen den Einsatz der betreffenden Algorithmen überhaupt erst praktisch sinnvoll. Es ist also in erster Linie zu beurteilen, ob die Einschränkungen in jeweiligen Domäne ein Problem darstellen.

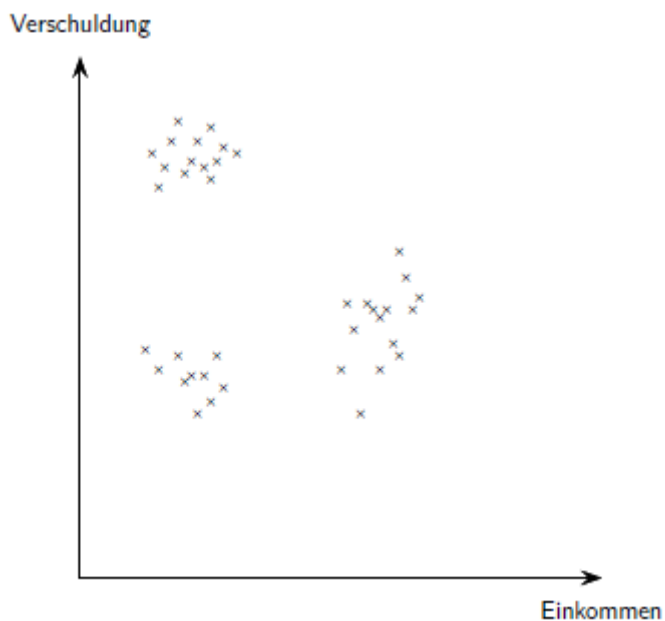


Abb. 11: Fiktives Beispiel für Clusteranalyse: Dargestellt sind Einkommen und Verschuldung von Haushalten. Diese fallen in drei Cluster. Bei mehr Dimensionen versagt die Anschauung, das Prinzip aber bleibt gleich.

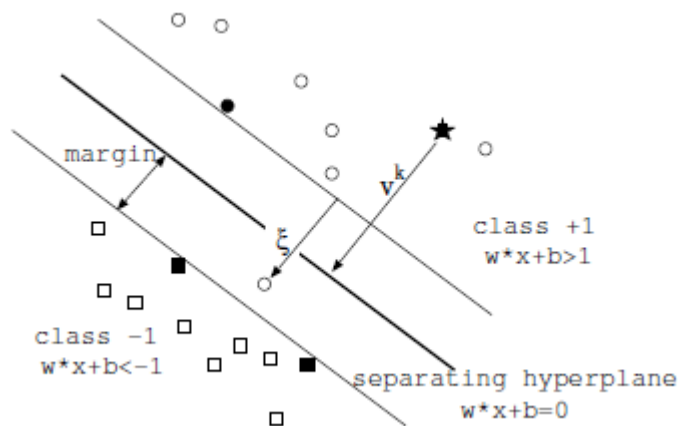


Abb. 12: Soft-Margin SVM: Instanzen zweier Klassen, hier als Quadrate und Kreise abgebildet, werden durch eine Hyperebene (vgl. Abschnitt 4.7) getrennt. Die Klassifizierung neuer Instanzen erfolgt dann je nachdem, auf welcher Seite der Trennebene sie liegen. Dargestellt ist der zweidimensionale Fall (Gerade), die SVM kann aber adaptiv hochdimensionale Hyperebenen erzeugen. Optimierungskriterium ist dabei grundsätzlich der Margin, d. h. der Sicherheitsabstand zwischen der Trennebene und der nächstgelegenen Instanz. Zulässig ist jedoch auch Schlupf (ξ), d. h. einige Trainingsinstanzen können auf der falschen Seite des Margin liegen, wenn dadurch insgesamt ein besseres Ergebnis zu erwarten ist.

Quelle: entnommen aus (Leopold & Kindermann, 2006)

Tab. 12: Übersicht zu Clusteringverfahren

Algorithmen:	<i>k</i> -Means, Expectation–Maximisation (EM), hierarchical Clustering
Anwendung:	Clusteranalyse (unüberwachtes Lernen)
Repräsentation:	Partitionierung oder hierarchische Ordnung von X
Modellkomplexität:	Clusterformen vom Algorithmus abhängig
Optimierungskriterium:	im Vorfeld wählbare Distanz- oder Dichtefunktion
Interpretierbarkeit:	Ergebnisse sind leicht verständlich, ihre Konsequenzen aber oft unklar

Clusteringverfahren sind nach ihrem Anwendungsgebiet, der Einteilung von Instanzen in Cluster, benannt. Dies ist z. B. im Marketing bei der Segmentierung von Kunden anhand ihrer Merkmale wichtig. Abb. 11 zeigt ein einfaches Beispiel. Da es sich um unüberwachtes Lernen handelt, ist die „korrekte“ Einteilung in Cluster nicht bekannt, die jeweiligen Algorithmen versuchen heuristisch, eine Einteilung abzuleiten. Voraussetzung dafür ist die Angabe einer Distanzfunktion oder Dichtefunktion, anhand derer sich abschätzen lässt, welche Instanzen zum selben Cluster gehören. Zu den Herausforderungen bei der Anwendung von Clusteringverfahren gehört es, eine solche Funktion passend zur Domäne und Zielsetzung auszuwählen (Ester & Sander, 2000, Kap. 4). Neben der Auswahl einer Distanz- oder Dichtefunktion hängt der Erfolg eines Clusteringverfahrens maßgeblich von der Auswahl der richtigen Attribute ab. Clusteringverfahren unterscheiden sich zudem z. T. gravierend in ihren Anforderungen an Rechenleistung und Speicherplatz und ihrer Robustheit gegen Ausreißer (Shmueli et al., 2010, S. 295). Die wesentliche Limitation für $\mathcal{H}$ ist die mögliche Form der Cluster. Der *k*-means Algorithmus kann z. B. nur sphärische Cluster entdecken.

Im einfachsten Fall ist das Ergebnis jeder Clusteranalyse eine Zuordnung von Instanzen zu Clustern. Dies ist konzeptionell leicht verständlich, jedoch ist es Aufgabe des Entscheiders, zu interpretieren.

tieren, was es bedeutet, dass genau diese Instanzen zusammen einen Cluster bilden. Diese Konsequenzen zu erkennen kann, je nach Semantik der Instanzen, problematisch sein. Ein weiteres Problem ist allein die Annahme, dass jedes Datenobjekt genau einem Cluster zugeordnet werden kann. Dies trifft in der Praxis nicht immer zu. Eine Erweiterung der klassischen Verfahren ist es, den Instanzen, statt eines festen Clusters, Wahrscheinlichkeiten für die Zugehörigkeit zu einem Cluster zuzuweisen. Dieser Ansatz wird als Fuzzy- Clustering bezeichnet (Grabmeier, 2001).

Tab. 13: Übersicht zu SC

Algorithmen:	SVM, PageRank, KNN, Evolutionäre Strategie (ES)
Anwendung:	ML (sowohl überwacht als auch unüberwacht)
Repräsentation:	verschieden, oft biologisch inspiriert (neuronales Netz, Genom, ...)
Modellkomplexität:	adaptiv
Optimierungskriterium:	verschieden (z. B. Margin, Fitness)
Interpretierbarkeit:	häufig schwer nachvollziehbar

SC ist ein Sammelbegriff für verschiedene Techniken des ML. Dabei ist die Abgrenzung zu anderen Bereichen nicht vollständig trennscharf. SC-Algorithmen zeichnen sich durch mehrere Eigenschaften aus, die nun kurz diskutiert werden sollen. Häufig sind SC-Verfahren eine Hybridisierung aus einem ML-Algorithmus wie KNN oder ES und einem probabilistischen Ansatz wie Fuzzy Logik (FL). SC-Verfahren ähneln eher Heuristiken als Optimierungsverfahren, da keine optimale Lösung angestrebt wird. Es geht bei diesem Prinzip nicht nur um Rechenzeiteinsparung, sondern auch darum anzuerkennen, dass eine perfekte Lösung auf fehlerbehafteten Inputs keinen echten Vorteil bringt. Auch ist häufig nicht bekannt, wie die Optimalität einer Lösung zu messen wäre. SC-Verfahren zeichnen sich weiterhin dadurch aus, dass sie implizit und adaptiv ein Modell aufbauen, während z. B. lineare Modelle in ihrer Struktur fest vorgegeben sind (Magdalena, 2010).

Viele einflussreiche DM-Algorithmen können dem SC zugeordnet werden. Die *soft-margin SVM* erlaubt explizit Schlupf, um im Gegenzug die Generalisierungsfähigkeit zu verbessern und Overfitting zu verkleinern (vgl. Abb. 12). Der *Page-Rank*-Algorithmus bestimmt ein Ranking von Webseiten, dessen Qualität immer subjektiv sein wird, d. h. nicht „hart“ bestimmt werden kann (Wu et al., 2008). ES sind aus der Analogie zur Natur, genauer zur Evolutionstheorie, entstanden. Dabei entspricht ein Individuum, repräsentiert durch sein Genom, im übertragenen Sinne einer Lösungsmöglichkeit. Die Bewertung der Individuen erfolgt mittels einer sogenannten Fitnessfunktion, welche die Güte der Lösung misst. Die ES besteht aus drei Schritten: Selektion, Rekombination, und Mutation. Diese werden so lange wiederholt, bis keine Verbesserung des Fitnesswertes mehr möglich ist (Linoff & Berry, 2011, Kap. 11).

Tab. 14: Übersicht zu Rule Induction

Algorithmen:	Apriori
Anwendung:	Assoziationsanalyse
Repräsentation:	wenn-dann Regeln
Modellkomplexität:	theoretisch exponentiell viele Regelkandidaten
Optimierungskriterium:	Häufigkeit der Itemkombinationen
Interpretierbarkeit:	Menge der Regeln oft unüberschaubar groß

Die Assoziationsanalyse beschäftigt sich allgemein mit der Frage „was passt zu was?“ (Shmueli et al., 2010, S. 263) oder anders formuliert mit „gemeinsam auftretenden Dingen“ (Bollinger, 1996).

Sachverhalte dieser Art werden häufig durch wenn-dann Regeln repräsentiert (vgl. Abschnitt 4.1). Der Begriff Assoziationsanalyse wird oft auch synonym mit dem wichtigsten Anwendungsgebiet, der Warenkorbanalyse, verwendet (Bollinger, 1996; Shmueli et al., 2010). Die „Dinge“ um die es geht, z. B. Produkte in einem Supermarkt, werden in der Literatur englisch mit Items bezeichnet. Im Rahmen dieser Einheit kann angenommen werden, dass zu jedem Item ein Attribut in Q gehört, dessen Wert 1 entspricht, wenn das Item „auftritt“, also z. B. gekauft wird.

Bei Verfahren zur Rule Induction gilt es nun, aus den Daten automatisiert signifikante Regeln zu extrahieren. Das Hauptproblem dabei ist, dass es für N_Q viele Items 2^{N_Q} viele Kombinationen gibt, in denen diese auftreten können. Diesen Raum vollständig zu durchsuchen ist praktisch nicht umsetzbar. Der *Apriori*-Algorithmus kann dieses Problem nicht prinzipiell umgehen, durch geschickte Berücksichtigung nur derjenigen Itemkombinationen, welche sehr häufig auftreten, lassen sich aber viele Praxisfälle in vertretbarer Zeit lösen (Wu et al., 2008). Durch den Apriori-Algorithmus werden interessante Zusammenhänge entdeckt. Jedoch ist die Anzahl der entdeckten Regeln aufgrund der Vielzahl der Kombinationsmöglichkeiten häufig sehr groß. Dadurch entsteht ein hoher Interpretationsaufwand, weil sämtliche Regeln zunächst gesichtet werden müssen, um abschließend für jede einzelne Regel zu entscheiden, ob diese entweder wichtig ist, oder vorab bekannt war, oder aber trotz der Regel kein kausaler Zusammenhang erkennbar ist. Im letzteren Fall können zwei Datensätze korreliert sein, ohne dass es dafür eine sachliche Begründung gibt. Ein weiteres Problem sind externe Einflussfaktoren. Diese können dazu führen, dass bestimmte Regeln nicht mehr interpretiert werden können. So können im Supermarktbeispiel das Wetter, die Uhrzeit, der Wochentag oder die Jahreszeit als externe Faktoren bei der Interpretation der gefundenen Zusammenhänge wichtig sein (Bollinger, 1996).

Tab. 15: Eager Learning, Lazy Learning und Ensemble Learning

Algorithmen:	Naive Bayes, k -Nearest Neighbour, AdaBoost
Anwendung:	überwachtes Lernen
Repräsentation:	modellbasiert, instanzbasiert oder gemischt
Modellkomplexität:	möglichst gering
Optimierungskriterium:	Trainingsfehler
Interpretierbarkeit:	intuitiv verständlich aber ggf. geringer Erkenntnisgewinn

Eager („eifriges“) Learning bezeichnet ML-Verfahren, die aus den Trainingsdaten sofort und ohne Umschweife ein Klassifizierungsmodell erstellen, welches dann verwendet werden kann, um neue Instanzen zu klassifizieren. Ein Beispiel für ein solche Modelle sind EBs (Tab. 11). Prototypisch für diese Klasse von Verfahren ist der *Naive Bayes*-Algorithmus. Demgegenüber bezeichnet lazy („träges“) Learning ML-Verfahren, welche den Generalisierungsvorgang aufschieben, bis sie tatsächlich mit neuen Instanzen konfrontiert werden, welche es zu klassifizieren gilt. Diese Verfahren bilden *kein* Modell von X sondern speichern die Trainingsinstanzen nur, ggf. in aufbereiteter Form, zur späteren Verwendung ab. Die Verfahren sind also nicht modellbasiert sondern instanzbasiert. Ein klassisches Beispiel dafür ist der k -Nearest Neighbour Algorithmus. Da in diesem Fall kein echtes Modell gebildet wird, ist der potentielle abstrakte Erkenntnisgewinn gering (Han & Kamber, 2006, S. 347).

Modellbasierte Verfahren bilden ein Modell für alle Instanzen und benötigen hohen Rechenaufwand bei der Modellerzeugung. Instanzbasierte Verfahren verwenden gewissermaßen jede Instanz einzeln als Modell und betreiben dafür hohen Aufwand bei der Klassifizierung neuer Instanzen. Ensemble Learning kann als Kompromiss zwischen diesen Ansätzen verstanden werden.

Der *AdaBoost*-Algorithmus kombiniert ein „Ensemble“ von sehr einfachen, unaufwändigen Modellen mit einer instanzbasierten Gewichtung. Somit werden die Einzelmodelle im Ensemble quasi auf wechselnde Untermengen der Instanzen zugeschnitten. Der Algorithmus hat attraktive theoretische Eigenschaften und liefert in der Praxis oft gute Ergebnisse. Die Attributauswahl ist als Nebeneffekt schon in den Algorithmus integriert, die Generalisierungsfähigkeit ist in der Regel hoch, das Overfitting gering (Wu et al., 2008).

4.3 Entscheidungsbaumverfahren

Klassifizierungs- und Regressionsbäume

Methoden zur Erzeugung von EBs, die der Klassifizierung und Approximation dienen, wurden, in den 1980er Jahren entwickelt. Bahnbrechend war dabei eine Monographie von Breiman, Friedman, Olshen und Stone (1984). Noch heute wird mit der Abkürzung CART die Implementierung der dort vorgestellten Ideen gemeint. Quinlan (1986) stellte den ID3 Algorithmus vor, einen Vorläufer des C4.5 Algorithmus, welcher heute zu den einflussreichsten und meist benutzten DM-Algorithmen gehört. Mögliche Anwendungsgebiete reichen von der medizinischen Diagnostik über die Vorhersage von Kundenverhalten bis hin zur Meteorologie (Shmueli et al., 2010, S. 164).

Formaler Rahmen

Für diese Einheit soll der Fokus auf EBs zur Klassifizierung gelegt werden. Gemäß der bisherigen Ausführungen muss also ein Konzept X vorliegen. Für spätere Verwendung sei die Indexmenge der Trainingsinstanzen (I_{Tr}) gegeben durch $(I_{Tr}) \subset \{1, \dots, N_X\}$ und die Menge der Trainingsdaten (X_{Tr}) durch $(X_{Tr}) = \{x_i | i \in I_{Tr}\}$. Die Instanzen $x_j \in X_{Tr}$ seien repräsentiert durch einen Vektor von Attributen aus Q mit dem jeweiligen Wertebereich V_i in qualitativer oder quantitativer Skalierung. Es gebe ein besonderes Attribut $q_g \in Q$, wobei V_{q_g} qualitativ skaliert ist und x_{i,q_g} die Klasse von x_i bezeichnet. Da der Baum auch auf andere als die Trainingsinstanzen angewendet werden soll, reicht es nicht X_{Tr} zu betrachten. Als Featureraum (X) sei daher pessimistisch die Menge aller möglichen Instanzen bezeichnet. Unter leichter Anpassung der Notation gelte:

$$X = V_1 \times \dots \times V_{q_g-1} \times V_{q_g+1} \times \dots \times V_{N_Q} \quad (4.2)$$

$$Y = V_{q_g} \quad (4.3)$$

$$X_{Tr} \subset X \times Y \quad (4.4)$$

Zentrale Idee

Ziel ist es, aus X_{Tr} einen EB abzuleiten, der gegeben $x \in X$ die zugehörige Klasse $y \in Y$ vorhersagt. Sei $Y = \{C_1, \dots, C_C\}$. Die Klassifizierungsaufgabe kann erfüllt werden, indem eine disjunkte Zerlegung $X = B_1 \cup \dots \cup B_C$ angegeben wird, sodass $h(x) = C_i \Leftrightarrow x \in B_i$. Das heißt: X wird in Partitionen aufgeteilt und die Klasse von $x \in X$ wird mit C_i angegeben, wenn x in der Partition B_i liegt. Per Definition ist dies gleichzeitig eine Partitionierung von X_{Tr} . Ein

EB ist die graphische Repräsentation einer solchen Partitionierung. Die verschiedenen EB-Verfahren unterscheiden sich darin, wie sie X_{Tr} verwenden um eine Partitionierung zu erstellen (Sutton, 2005).

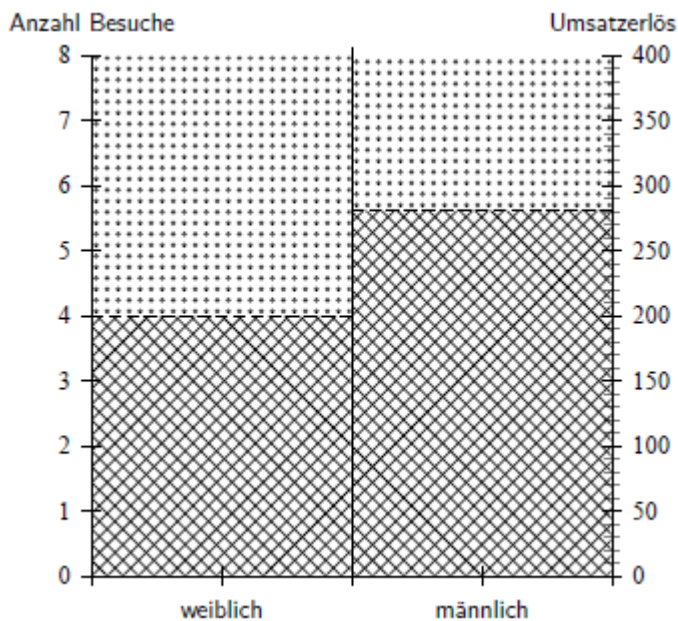


Abb. 13: Beispiel für Partitionierung durch einen EB

Abb. 13 zeigt die Partitionierung, die durch den beispielhaften EB, welcher in **Beispiel** Abb. 10 dargestellt ist, induziert wird. X wird zunächst nach dem Geschlecht in zwei Hälften geteilt. Die jeweiligen Hälften dann nach der Anzahl der Besuche bzw. dem Umsatzerlös. Die schraffierte Fläche wird als *nein* klassifiziert, die gepunktete Fläche als *ja*. Der Baum selbst besteht aus Knoten und Kanten. Der Knoten ohne Vorgänger heißt *Wurzel*, Knoten ohne Nachfolger heißen *Blätter*. Alternativ können die Blätter auch als *Endknoten* und die anderen Knoten als *innere Knoten* bezeichnet werden. An jedem inneren Knoten wird entschieden, nach welchem Attribut X weiter aufgeteilt werden soll. Die Kanten geben an, wie die Aufteilung vorgenommen wird. Endknoten geben an, zu welchem B_i die jeweilige Teilpartition gehört. Im Beispiel gibt es vier Endknoten und zwei Partitionen. Mit Nummerierung der Attribute in der Reihenfolge $Q_f = \{\text{Geschlecht, Besuche, Umsatz}\}$ ist:

$$t_1 = \{x_i \in \mathcal{X} | x_{i,1} = w \wedge x_{i,2} < 4\} \quad (4.5)$$

$$t_2 = \{x_i \in \mathcal{X} | x_{i,1} = w \wedge x_{i,2} \geq 4\} \quad (4.6)$$

$$t_3 = \{x_i \in \mathcal{X} | x_{i,1} = m \wedge x_{i,3} < 281.5\} \quad (4.7)$$

$$t_4 = \{x_i \in \mathcal{X} | x_{i,1} = m \wedge x_{i,3} \geq 281.5\} \quad (4.8)$$

$$B_{\text{ja}} = t_2 \cup t_4 \quad (4.9)$$

$$B_{\text{nein}} = t_1 \cup t_3 \quad (4.10)$$

Das Vorgehen, zunächst an der Wurzel mit ganz X zu beginnen und dann sukzessive weiter aufzuteilen wird als **Top-Down Vorgehen** Top-Down Induction of Decision Trees (TDIDT) bezeichnet (Quinlan, 1986). Alg. 1 ist ein generischer Algorithmus für TDIDT. Ein EB ist formal ein Tupel $EB = (T, K, \text{label})$ mit Knotenmenge (T) und Kantenmenge (K). *label* sei eine Liste mit Beschriftungen für die Elemente des EB. Der Algorithmus geht so vor, dass parallel zur Aufspaltung des EB auch eine Partitionierung von X erzeugt wird. Dies erledigt die Hilfsfunktion

Partition. Außerdem ist es häufig nötig, zu einer Partition die darin liegenden Trainingsinstanzen zu kennen. Dies erledigt `TrainingInstances`. Die Funktion `SplitTree` wird solange rekursiv aufgerufen, bis ein Abbruchkriterium erreicht ist.

Algorithmus 1: Generischer TDIDT-Algorithmus

Quelle: in Anlehnung an (Rokach & Maimon, 2010)

```

Input :  $\mathcal{X}, \mathcal{Y}, X_{Tr}, V, Q_f, q_g$ 
Output :  $EB = (T, K, label)$ 
Function TDIDT():
     $T \leftarrow \{t_0\}$ 
     $K \leftarrow \emptyset$ 
    SplitTree( $\mathcal{X}, t_0$ )
    PruneTree( $T$ )
    return ( $T, K, label$ )

Function SplitTree( $\mathcal{T}, t$ ):
    if StoppingCriterion( $\mathcal{T}$ ) then
         $tr \leftarrow \text{TrainingInstances}(\mathcal{T})$ 
         $label(t) \leftarrow \arg\max_{C \in \mathcal{Y}} \{|\{x_i \in tr | x_{i,q_g} = C\}|\}$ 
    else
         $q^*, \mathcal{V}_q^* \leftarrow \arg\max_{q \in Q_f, \mathcal{V}_q \in \text{SplitAttribute}(\mathcal{T}, q)} \{\text{Gain}(\mathcal{T}, q, \mathcal{V}_q)\}$ 
         $label(t) \leftarrow q^*$ 
        foreach  $v \in \mathcal{V}_q^*$  do
             $T \leftarrow T \cup \{t_v\}$ 
             $k \leftarrow (t, t_v)$ 
             $label(k) \leftarrow v$ 
             $K \leftarrow K \cup \{k\}$ 
            SplitTree(Partition( $\mathcal{T}, q^*, v$ ),  $t_v$ )

Function TrainingInstances( $\mathcal{T}$ ):
    return  $\{x_i \in X_{Tr} | x_{i,q_f} \in \mathcal{T}\}$ 

Function Partition( $\mathcal{T}, q, v$ ):
    return  $\{x \in \mathcal{T} | x_q \in v\}$ 

```

Spezialisierungen von TDIDT

Viele EB-Verfahren sind Spezialisierungen des generischen TDIDT-Algorithmus. Bestandteile einer solchen Spezialisierung sind laut Petersohn (2005, Kap. 5):

- **Attributauswahl zur Aufspaltung eines Knotens (Gain):** An jedem inneren Knoten muss zunächst das Attribut bestimmt werden, anhand dessen X dann unterteilt wird. Die Auswahl des Attributes wird mittels eines Entropiemaßes getroffen. Mögliche Kandidaten für ein solches Maß sind die Reinheit ("Purity") der entstehenden Partitionen, der Gini-Koeffizient oder der Informationsgewinn.
- **Aufspaltung des Knotens (SplitAttribute):**
Je nach Verfahren können Knoten in zwei oder mehrere Unterknoten aufgespaltet werden (Split). Unproblematisch ist es, wenn der Wertebereich des Attributs genau so viele Elemente hat, wie es Unterknoten geben kann, z. B. das Attribut Geschlecht in männlich/weiblich. Bei quantitativen Attributen ist dies in der Regel nicht möglich. Im Beispiel wurde der Wertebereich des Attributs Umsatz etwa in zwei Intervalle geteilt.
- **Abbruchkriterium zur Endknotenbestimmung (StoppingCriterion):**

Attributauswahl und Aufspaltung werden so lange wiederholt, bis sich eine weitere Unterteilung von X nicht mehr „lohnt“, d. h. bis die erwartete Verbesserung bezüglich des Entropiemaßes nur noch marginal ist. Dann wird aus dem betrachteten Knoten ein Endknoten. Dies ist z. B. trivial der Fall, wenn alle Trainingsinstanzen $x_i \in X_{Tr}$, welche in der aktuellen Partition liegen, der selben Klasse C_i angehören. Die aktuelle Partition wird dann Teil von B_i . Ein solcher Endknoten heißt *reiner* Knoten. Befinden sich dagegen in der aktuellen Partition noch Trainingsinstanzen verschiedener Klassen, so wird sie demjenigen B zugeordnet, welches zur Klasse der *Mehrheit dieser Trainingsinstanzen* gehört. Ein solcher Endknoten heißt *unreiner* Knoten. Der Abbruch an unreinen Knoten scheint unintuitiv, kann sich aber lohnen um Overfitting (vgl. S. 45) zu verhindern. Dies wird dann als *early Stopping* bezeichnet.

- **Post-Pruning (PruneTree):**
Trotz early Stopping ist der entstandene EB auf das konkrete X_{Tr} zugeschnitten. Möglicherweise lässt sich die Generalisierungsleistung des Baumes durch *Pruning* (Zurückschneiden) erhöhen. Dabei wird die Anzahl der Endknoten, so weit wie möglich reduziert, indem einzelne Teilbäume insgesamt einer Klasse zugeordnet werden.

Beispiel: Einfaches EB-Verfahren Zur Einführung soll zunächst ein sehr einfaches EB-Verfahren gezeigt werden welches das Vorgehen illustriert, aber für die Praxis zu anfällig für Overfitting ist. Gemäß obiger Liste werden die folgenden Elemente des Verfahrens festgelegt:

Purity Als extrem simples Maß für den Gewinn bei Aufspaltung eines Knotens soll hier die Abnahme von Unreinheit eines Knotens verwendet werden. Die Reinheit eines Knotens sei als der maximale Anteil einer Klasse an den Instanzen der betreffenden Partition der Trainingsdaten definiert. Dieser Anteil ist maximal 1, wenn alle Datensätze zur selben Klasse gehören und für $|Y| = c$ viele Klassen minimal $\frac{1}{c}$, wenn alle Klassen über die Trainingsinstanzen gleich verteilt sind. Die Unreinheit oder *Impurity* sei nun entsprechend definiert als $1 - \text{Purity}$ mit Minimum 0 und Maximum $\frac{c-1}{c}$. Der Gewinn ist also die Impurity des ursprünglichen Knotens abzüglich der gewichteten Impurity der Kindknoten (Alg. 2). Dieses Maß lässt in der Praxis einiges zu wünschen übrig, hat aber den Vorteil, dass es durch einfaches Nachzählen der Trainingsinstanzen bestimmt werden kann.

Algorithmus 2: Funktion zur Bestimmung des PurityGain

Function PurityGain($\mathcal{T}, q, \mathcal{V}$):

```

  gs ← 0
  foreach v ∈ V do
    weight ←  $\frac{|\text{TrainingInstances}(\text{Partition}(\mathcal{T}, q, v))|}{|\text{TrainingInstances}(\mathcal{T})|}$ 
    gs ← gs + weight · Impurity(Partition( $\mathcal{T}, q, v$ ))
  return Impurity( $\mathcal{T}$ ) - gs

```

Function Impurity($\mathcal{T}$):

```

  tr ← TrainingInstances( $\mathcal{T}$ )
  return  $1 - \max_{C \in \mathcal{Y}} \left\{ \frac{|\{x_i \in tr \mid x_{i, \text{agg}} = C\}|}{|tr|} \right\}$ 

```

Aufspaltung der Attribute Qualitative Attribute sollen in ihre möglichen Werte aufgespalten werden. Quantitative Attribute am arithmetischen Mittel der Trainingsinstanzen (Alg. 3). Die Aufspaltung soll bis zu einer Impurity von 0 fortgesetzt werden. Early Stopping ist nicht vorgesehen. Auch auf den Schritt des Post-Prunings soll an dieser Stelle vollständig verzichtet werden.

Algorithmus 3: Aufspaltung quantitativer Attribute am Mittelwert

```

Function SplitAttributeMean( $\mathcal{T}, q$ ):
  if  $q$  ist qualitativ then
    return  $\{\{v\} | v \in V_q\}$ 
  else
     $tr \leftarrow \text{TrainingInstances}(\mathcal{T})$ 
     $m \leftarrow \frac{1}{|tr|} \sum_{x_i \in tr} x_{i,q}$ 
    return  $\{\{[\min(V_q), m]\}, \{[m, \max(V_q)]\}\}$ 

```

Tab. 16: Medizinische Beispieldaten für das EB-Verfahren

ID	Geschlecht	Alter	Blutdruck	Medikament
1	männlich	70	Hoch	A
2	männlich	37	Hoch	A
3	männlich	39	Mittel	B
4	männlich	24	Niedrig	B
5	männlich	59	Mittel	A
6	männlich	21	Niedrig	B
7	weiblich	52	Mittel	A
8	weiblich	35	Mittel	B
9	weiblich	32	Hoch	A
10	weiblich	27	Mittel	B
11	weiblich	49	Mittel	A
12	weiblich	33	Niedrig	B

Beispieldaten Als Trainingsdaten liegen von insgesamt zwölf Personen Informationen über deren Geschlecht, das Alter, den Blutdruck und ein verabreichtes Medikament, entweder A oder B, vor. Ziel der Klassifikation ist es, neuen Patienten ein geeignetes Medikament zu verabreichen. Die Datenbasis kann Tab. 16 entnommen werden. Das Verfahren beginnt mit den gesamten Trainingsdaten, d. h. im ersten Schritt besteht der Baum nur aus einem einzigen Knoten. Die Partitionierung der Trainingsdaten soll hier einfach durch die Menge der ID-Nummern in der jeweiligen Partition illustriert werden, d. h. es gilt $t_1 = \{1, \dots, 12\}$. Im Beispiel haben 6 von 12 Datensätzen die Klasse „Medikament A“ und 6 von 12 Datensätzen die Klasse „Medikament B“ der Anteil ist also jeweils $\frac{1}{2}$ dies ist gleichzeitig Purity und Impurity von t_1 welches somit maximal unrein ist:

(1)

Schritt 1

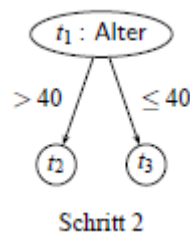
Bestimmung des ersten Attributs Um herauszufinden, welche Aufspaltung vorzunehmen ist, wird die gewichtete Impurity der Kindknoten für jede mögliche Aufspaltung einzeln berechnet:

Es lässt sich sehr leicht nachzählen, dass durch die Aufspaltung nach dem Attribut Geschlecht an dieser Stelle nichts zu gewinnen wäre. Die Datenbasis zerfällt in 6 männliche und 6 weibliche Personen, die Gewichtung wäre also jeweils $\frac{1}{2}$. Innerhalb der so gewonnenen Partitionen werden sowohl das Medikament A als auch das Medikament B jeweils 3 mal eingenommen, die Impurity in jeder Partition wäre somit ebenfalls $\frac{1}{2}$ und damit auch das gewichtete Mittel der Impurity aller Kindknoten. Dies war aber auch schon die Impurity der Datenbasis.

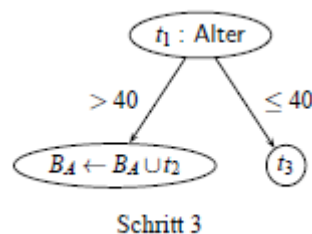
Interessanter stellt sich die Situation schon beim **Alter** dar. Das Durchschnittsalter beträgt rund 40 Jahre. Nur 4 Personen sind überhaupt älter als 40 Jahre – ihr Anteil bzw. Gewichtungsfaktor ist somit $\frac{1}{3}$ – und sie nehmen alle das Medikament A. Die Impurity dieser Partition ist somit 0. Der Anteil der jüngeren ist $\frac{2}{3}$, von denen 2 das Medikament A aber 6 das Medikament B nehmen. Die Impurity dieser Partition ist also $1 - \frac{6}{8} = \frac{1}{4}$ und die gewichtete Gesamt-Impurity ist $\frac{1}{3} * 0 + \frac{2}{3} * \frac{1}{4} = \frac{1}{6}$.

Für das Attribut **Blutdruck** ist die Situation wiederum recht übersichtlich. Jeweils 3 Personen haben hohen oder niedrigen Blutdruck, der Anteil ist also jeweils $\frac{1}{4}$. Dabei nehmen innerhalb der jeweiligen Partitionen alle dasselbe Medikament, die Impurity ist also jeweils 0. Dagegen haben die übrigen 6 Personen einen mittleren Blutdruck und jeweils 3 davon nehmen dasselbe Medikament. Die Impurity ist also für diese Partition erneut $\frac{1}{2}$. Somit ist die gesamte Impurity für diese Aufspaltung $\frac{1}{4} * 0 + \frac{1}{4} * 0 + \frac{1}{2} * \frac{1}{2} = \frac{1}{4}$.

Durchführung der Aufspaltung Aufgrund der verschiedenen Impuritywerte bringt eine Aufspaltung nach dem Attribut **Alter** im ersten Schritt den größten Gewinn. t_1 wird also in $t_2 = \{1,5,7,11\}$ und $t_3 = t_1 \setminus t_2$ aufgeteilt. Nach dieser Aufspaltung stellt sich die Situation wie unter Schritt 2 verzeichnet dar:



Überprüfung des Abbruchkriteriums Für jeden neu entstandenen Knoten muss nun überprüft werden, ob das Abbruchkriterium für diesen Knoten erfüllt ist. Für die Partition t_2 der über 40-jährigen, ist die Impurity wie oben bestimmt 0, also so, wie das Abbruchkriterium es vorsieht. Somit gehört dieser Knoten zur Partition B_A des Medikaments A.



Erneute Anwendung des Verfahrens Für die Partition t_3 der unter 40-jährigen ist die Impurity mit $\frac{1}{4}$ größer als 0 und das Verfahren muss erneut angewendet werden. Da sich die Trainingsinstanzen in dieser Partition von denen in t_1 unterscheiden, muss die Impurity für alle Attribute erneut bestimmt werden:

Der Anteil der beiden Geschlechter an t_3 ist erneut $\frac{1}{2}$. Sowohl bei den Männern als auch bei den Frauen nehmen jeweils $\frac{1}{4}$ der Personen Medikament A. Der Anteil von Medikament B beträgt jeweils $\frac{3}{4}$. Die Impurity hier ist also $\frac{1}{4}$ und die gewichtete Gesamtimpurity für das Attribut Geschlecht ist $\frac{1}{2} * \frac{1}{4} + \frac{1}{2} * \frac{1}{4} = \frac{1}{4}$.

Für das Alter hat sich der neue Mittelwert von 31 Jahren eingestellt. $\frac{3}{8}$ der Patienten in t_3 sind jünger als 31 Jahre, diese nehmen ausnahmslos Medikament B ein. Die übrigen $\frac{5}{8}$ weisen eine Impurity von $\frac{2}{5}$ auf, sodass sich insgesamt eine Impurity von $\frac{1}{4}$ ergibt.

Dagegen ist eine Aufspaltung nach dem Blutdruck an dieser Stelle besonders erfolgreich, da nur reine Knoten übrig bleiben und die gesamte Impurity somit 0 beträgt. Alle entstehenden Partitionen $t_4 = \{4, 6, 12\}$, $t_5 = \{3, 8, 10\}$ und $t_6 = \{2, 9\}$ sind also Endknoten. Der endgültige Baum hat damit die in nachstehender Abb. 14 dargestellte Struktur.

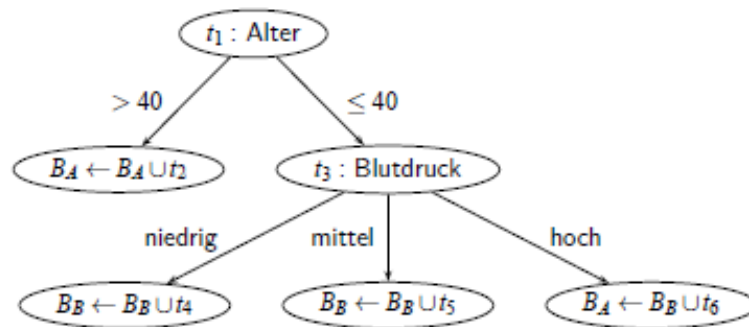


Abb. 14: Klassifikationsbaum nach Beendigung des Beispielverfahrens

Abschließende Überlegungen Der ermittelte Baum kann verwendet werden, um neuen Instanzen mit unbekannter Klasse eine Klasse zuzuweisen, indem an jeder Verzweigung demjenigen Ast gefolgt wird, welcher für die gegebene Instanz zutrifft, bis ein Endknoten erreicht wurde. Die Tatsache, dass alle Knoten am Ende reine Knoten waren, lässt jedoch nicht darauf schließen, dass auch alle zukünftigen Datensätze richtig klassifiziert werden. Im Gegenteil handelt es sich eher um ein Anzeichen dafür, dass ein Fall von Overfitting vorliegt, in diesem Fall natürlich begünstigt durch die extrem kleine Datenbasis. Frühzeitiges Abbrechen bei Impuritywerten > 0 sowie nachträgliches Pruning kann die Generalisierungsleistung eines EB erhöhen.

Einfache und dennoch praxisrelevante Spezialisierungen des TDIDT-Algorithmus sind CART (Breiman et al., 1984) und C4.5 (Quinlan, 1993). CART verwendet den Gini-Index (Alg. 4) zur Attributauswahl und *Cost-Complexity Pruning*, welches hier nicht weiter diskutiert werden soll. Ein early Stopping ist nicht vorgesehen, wie auch im einfachen Beispielverfahren wird bei einer Impurity von 0 gestoppt. Die wohl wesentlichste Eigenschaft von CART im Vergleich zum generischen TDIDT ist, dass alle Aufteilungen bei CART binär sind (Wu et al., 2008). Breiman et al. (1984) begründen dies mit einer besseren Performance. Es erlaubt zudem, qualitative und quantitative Attribute im Wesentlichen gleich zu behandeln. Um binäre Partitionen zu ermöglichen, muss die *SplitAttribute* Funktion so definiert werden, dass sie stets binäre Aufteilungen zurückgibt. Prinzipiell gibt es verschiedene Möglichkeiten, den Wertebereich V_q eines Attributs $q \in Q$ binär aufzuteilen. Anders als im einfachen Beispiel, soll *SplitAttributeBinary* (Alg. 5)

CART

daher mehrere mögliche Splits zurückgeben, die dann bezüglich GiniGain gegeneinander abgewogen werden. Allerdings müssen potentiell sehr viele Werte verglichen werden, was viel Rechenzeit benötigt. EBs haben daher ihre größte Stärke dort, wo bei den meisten Attributen von vornherein nur wenige und nur qualitative Werte zugelassen sind. Da Splits sich immer aus den Trainingsdaten ergeben, sind bei Attributen mit vielen möglichen Werten nur die Werte von Interesse, die auch wirklich in X_{Tr} vorkommen. Diese werden durch **Values** bestimmt.

Algorithmus 4: Gini-Index zur Attributauswahl

Quelle: in Anlehnung an (Rokach & Maimon, 2010)

```

Function GiniGain( $\mathcal{T}, q, \mathcal{V}$ ):
     $gs \leftarrow 0$ 
    foreach  $v \in \mathcal{V}$  do
         $weight \leftarrow \frac{|\text{TrainingInstances}(\text{Partition}(\mathcal{T}, q, v))|}{|\text{TrainingInstances}(\mathcal{T})|}$ 
         $gs \leftarrow gs + weight \cdot \text{Gini}(\text{Partition}(\mathcal{T}, q, v))$ 
    return  $\text{Gini}(\mathcal{T}) - gs$ 

Function Gini( $\mathcal{T}$ ):
     $tr \leftarrow \text{TrainingInstances}(\mathcal{T})$ 
    return  $1 - \sum_{C \in \mathcal{Y}} \left( \frac{|\{x_i \in tr \mid x_{i,q} = C\}|}{|tr|} \right)^2$ 

```

Der C4.5-Algorithmus basiert nicht auf dem Gini-Index sondern auf der informationstheoretischen Entropie. InformationGain entspricht dabei GiniGain, nur dass der Gini-Index durch die Entropie ersetzt wurde. InformationGain bevorzugt allerdings unter den qualitativen Attributen solche mit vielen möglichen Werten. Um dies zu vermeiden sollte stattdessen GainRatio maximiert werden. Die Aufspaltung der Attribute (Alg. 7) erfolgt binär für quantitative Attribute und in alle möglichen Werte für qualitative Attribute. Der C4.5-Algorithmus implementiert early Stopping, falls die Anzahl der Trainingsinstanzen an einem Knoten unter eine vom Anwender bestimmte Schwelle fällt. Der verwendete Pruning-Mechanismus ist *Error-Based Pruning* (Wu et al., 2008).

C4.5

Algorithmus 5: Binäre Aufspaltung von Attributen

```

Function SplitAttributeBinary( $\mathcal{T}, q$ ):
     $\mathcal{V}_{bin} \leftarrow \emptyset$ 
    foreach  $v \in \text{Values}(q)$  do
        if  $q$  ist qualitativ then
             $\mathcal{V}_{bin} \leftarrow \mathcal{V}_{bin} \cup \{ \{v\}, V_q \setminus \{v\} \}$ 
        else
             $\mathcal{V}_{bin} \leftarrow \mathcal{V}_{bin} \cup \{ [\min(V_q), v], [v, \max(V_q)] \}$ 
    return  $\mathcal{V}_{bin}$ 

Function Values( $\mathcal{T}, q$ ):
    return  $\{ v \in V_q \mid \exists x_i \in \text{TrainingInstances}(\mathcal{T}) : x_{i,q} = v \}$ 

```

Algorithmus 6: InformationGain zur Attributauswahl
 Quelle: in Anlehnung an (Rokach & Maimon, 2010)

```

Function GainRatio( $\mathcal{T}, q, \mathcal{V}$ ):
  return  $\frac{\text{InformationGain}(\mathcal{T}, q, \mathcal{V})}{\text{Entropy}(\mathcal{T}, q, \mathcal{V})}$ 
Function InformationGain( $\mathcal{T}, q, \mathcal{V}$ ):
   $gs \leftarrow 0$ 
  foreach  $v \in \mathcal{V}$  do
     $weight \leftarrow \frac{|\text{TrainingInstances}(\text{Partition}(\mathcal{T}, q, v))|}{|\text{TrainingInstances}(\mathcal{T})|}$ 
     $gs \leftarrow gs + weight \cdot \text{Entropy}(\text{Partition}(\mathcal{T}, q, v), q_g, \{V_{q_g}\})$ 
  return  $\text{Entropy}(\mathcal{T}, q_g, \{V_{q_g}\}) - gs$ 
Function Entropy( $\mathcal{T}, q, \mathcal{V}$ ):
   $tr \leftarrow \text{TrainingInstances}(\mathcal{T})$ 
  return  $-\sum_{v \in \mathcal{V}} \left( \frac{|\{x_i \in tr | x_{i,q} = v\}|}{|tr|} \right) \cdot \log_2 \left( \frac{|\{x_i \in tr | x_{i,q} = v\}|}{|tr|} \right)$ 

```

Algorithmus 7: Gemischte Aufspaltung von Attributen

```

Function SplitAttributeMixed( $\mathcal{T}, q$ ):
  if  $q$  ist qualitativ then
    return  $\{\{v\} | v \in V_q\}$ 
  else
     $\mathcal{V}_{bin} \leftarrow \emptyset$ 
    foreach  $v \in \text{Values}(q)$  do
       $\mathcal{V}_{bin} \leftarrow \mathcal{V}_{bin} \cup \{\{[\min(V_q), v]\}, \{[v, \max(V_q)]\}\}$ 
    return  $\mathcal{V}_{bin}$ 

```

4.4 Regressionsanalyse

Zentrale Idee Regressionsverfahren dienen der statistischen Modellierung von Klassifizierungs- und Approximationsproblemen (vgl. Tab. 9). Es gilt weiterhin der formale Rahmen, der in Abschnitt 4.3 vorgestellt wurde. Die Modellierung erfolgt allerdings hier nicht als EB sondern als Funktion $h(\cdot) \in H$. Gesucht ist also für Approximation eine Funktion

$$h: X \rightarrow Y \quad (4.11)$$

$$x_{i,q_f} \mapsto x_{i,q_g} \quad (4.12)$$

Für binäre Klassifizierung mit $Y = \{-1, +1\}$ kann das Vorzeichen $\text{sgn}(h(\cdot))$ verwendet werden. Die Basis für eine Regressionsanalyse bildet das Regressionsmodell, welches H bestimmt. In der Theorie gibt es viele verschiedene Regressionsmodelle. Von besonderer praktischer Bedeutung sind aber die lineare Regression und die logistische Regression.

Lineare Regression Bei der linearen Regression ist H die Menge der linearen Funktionen über X , d.h. es gilt:

$$x_{i,q_g} = h(x_{i,q_f}) = \theta_0 + \sum_{q \in Q_f} \theta_q x_{i,q} + \varepsilon_i \quad (4.13)$$

Dabei heißt $x_{i,qg}$ *Regressand* und $x_{i,q}$ *Regressor*. Die Parameter des Modells sind wie zuvor als θ notiert, welches hier einen Koeffizientenvektor bildet. Die Abweichung ε_i zwischen der Vorhersage der linearen Funktion und dem tatsächlichen Wert des Regressanden wird als *Residuum* bezeichnet (Eckstein, 2014).

Im einfachsten Fall gibt es nur einen Regressor. Dies wird daher auch als Einfachregression bezeichnet. Abb. 15 zeigt ein solches Beispiel. Die sogenannte Freihandmethode – das Einzeichnen einer möglichst passenden Gerade in eine Punktwolke – ist offensichtlich unwissenschaftlich. Ein wissenschaftlicher, da objektiv nachvollziehbarer Ansatz ist es dagegen die Summe der horizontalen Abstandsquadrate zu minimieren:

Methode der kleinsten Quadrate

$$\sum_{i=1}^{N_x} \varepsilon_i^2 \stackrel{!}{\rightarrow} \min \quad (4.14)$$

für Einfachregression ist dies durch Umformen von (4.13) identisch mit:

$$\sum_{i=1}^{N_x} (x_{i,qg} - (\theta_0 + \theta_1 x_{i,qf}))^2 \stackrel{!}{\rightarrow} \min \quad (4.15)$$

Minimieren entspricht dem Nullsetzen der partiellen Ableitungen:

$$\frac{\partial}{\partial \theta_0} = -2 \sum_{i=1}^{N_x} (x_{i,qg} - (\theta_0 + \theta_1 x_{i,qf})) \stackrel{!}{=} 0 \quad (4.16)$$

$$\frac{\partial}{\partial \theta_1} = -2 \sum_{i=1}^{N_x} (x_{i,qg} - (\theta_0 + \theta_1 x_{i,qf})) * x_{i,qf} \stackrel{!}{=} 0 \quad (4.17)$$

Und durch einfaches Umformen und Einsetzen ergibt sich (vgl. (3.26)):

$$\theta_0 = \overline{x_{i,qg}} - \theta_1 \overline{x_{i,qf}} \quad (4.18)$$

$$\theta_1 = \frac{N_x \sum_{i=1}^{N_x} x_{i,qf} x_{i,qg} - (\sum_{i=1}^{N_x} x_{i,qf})(\sum_{i=1}^{N_x} x_{i,qg})}{N_x \sum_{i=1}^{N_x} x_{i,qf}^2 - (\sum_{i=1}^{N_x} x_{i,qf})^2} = \frac{cov(x_{i,qf}, x_{i,qg})}{cov(x_{i,qf}, x_{i,qf})} \quad (4.19)$$

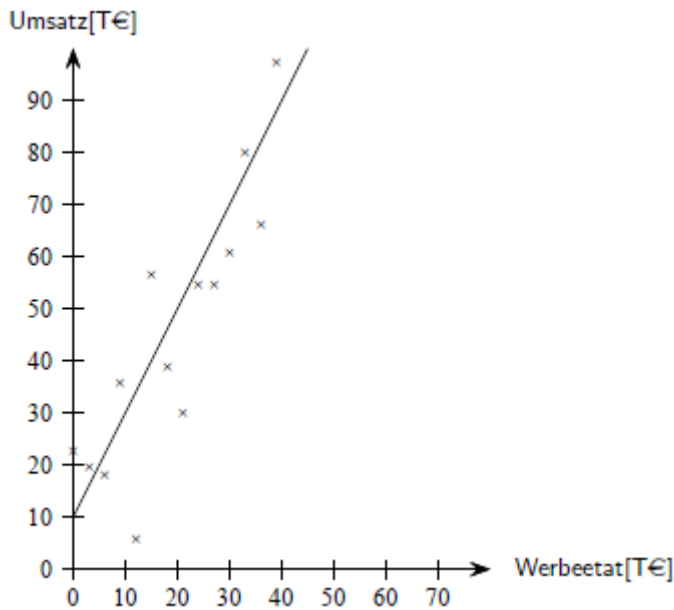


Abb. 15: Fiktives Beispiel für lineare Regression: Ein Startup-Unternehmen fährt Monat für Monat schrittweise seinen Werbeetat hoch und beobachtet dabei die im Diagramm abgetragenen Umsätze. Die Regressionsgerade wird dabei so durch die Punktwolke gelegt, dass sie die Daten möglichst gut wiedergibt. Jedoch ist es nicht möglich, dass alle Punkte auf einer Geraden liegen. Die vertikalen Abweichungen bilden die Residuen und entsprechen den Größen, die bei der Schätzung nicht berücksichtigt wurden bzw. nicht messbar sind, z. B. das Image des Unternehmens. Die geschätzte Gerade hat die Funktion $h(x) = 10 + 2x$. Dies lässt sich so interpretieren, dass für jeden zusätzlichen Euro Werbeetat zwei Euro an Umsatz generiert werden. Der Umsatz ohne die Durchführung von Werbemaßnahmen beträgt dabei 10.000e. Mit Hilfe der geschätzten Regressionsgeraden lassen sich für zukünftige Werbemaßnahmen die Umsätze prognostizieren. Möchte das Unternehmen beispielsweise 5.000e für Werbemaßnahmen ausgeben, so liegt der erwartete Umsatz bei $10 + 2 \cdot 5 = 20\text{T€}$.

Das analytische Vorgehen lässt sich auf mehr als einen Regressor – die multivariate Regression – verallgemeinern, dazu muss jedoch eine Matrix invertiert werden (Guckelsberger & Unger, 1999, Kap. 6).

Logistische Regression

Die Grundidee bei der logistischen Regression ist es, eine lineare Entscheidungsfunktion für den Zweiklassenfall zu definieren. Sinnvollerweise sollte die Funktion $h(\cdot)$ dabei folgende Anforderungen erfüllen:

$$h: \mathcal{X} \rightarrow \mathbb{R} \quad (4.20)$$

$$h(x_{i,q_f}) > 0 \Leftrightarrow \mathbb{P}(x_{i,q_g} = +1 | x_{i,q_f}) > \mathbb{P}(x_{i,q_g} = -1 | x_{i,q_f}) \quad (4.21)$$

Im Zweiklassenfall handelt es sich um Gegenwahrscheinlichkeiten und die Anforderungen werden erfüllt durch

$$h(x_{i,q_f}) = \log \left(\frac{\mathbb{P}(x_{i,q_g} = +1 | x_{i,q_f})}{1 - \mathbb{P}(x_{i,q_g} = +1 | x_{i,q_f})} \right) \quad (4.22)$$

Diese Funktion soll nun durch eine lineare Funktion genähert werden, d. h. es gilt (für Einfachregression):

$$\log \left(\frac{\mathbb{P}(x_{i,q_g} = +1 | x_{i,q_f})}{1 - \mathbb{P}(x_{i,q_g} = +1 | x_{i,q_f})} \right) = \theta_0 + \theta_1 x_{i,q_f} \quad (4.23)$$

Dies hat eine Lösung:

$$\mathbb{P}(x_{i,q_g} = +1 | x_{i,q_f}) = \frac{1}{1 + e^{-(\theta_0 + \theta_1 x_{i,q_f})}} \quad (4.24)$$

Bei der Lösung handelt es sich um die logistische Funktion. Diese gibt der logistischen Regression ihren Namen. Anders als bei linearer Regression kann θ nicht durch die Methode der kleinsten Quadrate berechnet werden, sondern muss mit einem maximum Likelihood Ansatz geschätzt werden (Linoff & Berry, 2011, S. 229–231).

4.5 Clusteranalyse

Ziel der Clusteranalyse ist es, Instanzen gemäß objektiv nachvollziehbarer Kriterien zu gruppieren, ohne dass eine solche Gruppierung im Vorhinein bekannt wäre. Es handelt sich somit um unüberwachte Verfahren. Umgangssprachlich formuliert ist das häufigst genannte Clusteringkriterium, „dass Objekte [Instanzen] im gleichen Cluster möglichst ähnlich und Objekte aus verschiedenen Clustern möglichst unähnlich zueinander sind“ (Ester & Sander, 2000, S. 45). Zu klären ist nun also, was „ähnlich“ bzw. „unähnlich“ formal bedeutet, und wie eine Gruppierung nach diesem Kriterium algorithmisch erfolgen kann.

Zentrale Idee

Wieder gelte der formale Rahmen aus Abschnitt 4.3. Der Feature Raum X sei zudem *metrisch*, d. h. es gibt eine Distanzfunktion (*dist*) mit folgenden Eigenschaften (Chattamvelli, 2009, S. 195):

Formaler Rahmen

$$dist: \mathcal{X} \times \mathcal{X} \rightarrow \mathbb{R} \quad (4.25)$$

$$dist(x, x) = 0 \quad (4.26)$$

$$dist(x, y) \geq 0 \quad (\text{nicht-Negativität}) \quad (4.27)$$

$$dist(x, y) = dist(y, x) \quad (\text{Symmetrie}) \quad (4.28)$$

$$dist(x, y) \leq dist(x, z) + dist(z, y) \quad (\text{Transitivität}) \quad (4.29)$$

Distanzfunktionen können zur Bestimmung der Ähnlichkeit bzw. Unähnlichkeit zwischen Instanzen verwendet werden. Über den reellen Zahlen gibt es verschiedene Distanzfunktionen. Die im DM häufigst verwendeten sind:

Anwendung von Distanzmaßen

$$dist(x_i, x_j) = \sqrt{\sum_{q \in Q} (x_{i,q} - x_{j,q})^2} \quad \text{Euklidische Distanz} \quad (4.30)$$

$$dist(x_i, x_j) = \sum_{q \in Q} |x_{i,q} - x_{j,q}| \quad \text{Manhattan-Distanz} \quad (4.31)$$

Beide Distanzmaße setzen quantitative Attribute voraus. Bei qualitativen, insbesondere binären Attributen sind theoretisch andere Distanzmaße wie der Jaccard-Koeffizient oder der M-Koeffizient einzusetzen (Grimmer & Mucha, 1998). In der Praxis sind Konzepte mit ausschließlich qualitativen Attributen jedoch selten sodass hier nur der Fall $X \subseteq R_{N_Q}$ betrachtet werden soll. Allgemein muss überprüft werden, ob das verwendete Distanzmaß für das untersuchte Problem sachlich begründbar ist. Oft ist es komfortabel, die paarweisen Distanzen zwischen den Instanzen als Distanzmatrix (**D**) zu schreiben, sodass gilt:

$$D_{i,j} = \text{dist}(x_i, x_j) \quad (4.32)$$

Wegen der Symmetrie der Distanzfunktion ist es ausreichend, die obere oder untere Dreiecksmatrix anzugeben.

Partitionierende Verfahren

Da es sich bei der Clusteranalyse um unüberwachtes Lernen handelt, sei $Q_f = Q$ und $Q_g = \emptyset$. Das heißt es gibt keine Zielattribute. Abgesehen davon seien X und X_{Tr} definiert wie in Abschnitt 4.3. Gesucht ist wieder eine Partitionierung $X = B_1 \cup \dots \cup B_C$. Allerdings gibt es, anders als bei der Klassifizierung, kein leitendes Zielattribut, sondern es soll gelten:

1. Jede Partition enthält mindestens eine Instanz ($\forall B_i \exists x \in X_{Tr} : x \in B_i$).
2. Die Partitionierung ist disjunkt.
3. Die paarweise Distanz innerhalb der Cluster soll tendenziell minimiert werden ($\forall x_i, x_j \in B_i : \text{dist}(x_i, x_j) \xrightarrow{!} \min$).
4. Die paarweise Distanz zwischen zwei Clustern soll tendenziell maximiert werden ($\forall x_i \in B_i, x_j \in B_j : \text{dist}(x_i, x_j) \xrightarrow{!} \max$).

Bezüglich der beiden letztgenannten Punkte sind häufig Kompromisse nötig. Partitionierende Verfahren bestimmen zunächst eine initiale Partitionierung und „verschieben“⁶ dann Trainingsinstanzen iterativ zwischen den Partitionen bis keine Verbesserung mehr erzielt werden kann. Ein Spezialfall partitionierender Verfahren sind dichtebasierte Verfahren, die eine Dichtefunktion statt einer Distanzfunktion zugrunde legen (Han & Kamber, 2006, S. 398f).

⁶ Formal entspricht dies einer Änderung der Partitions Grenzen

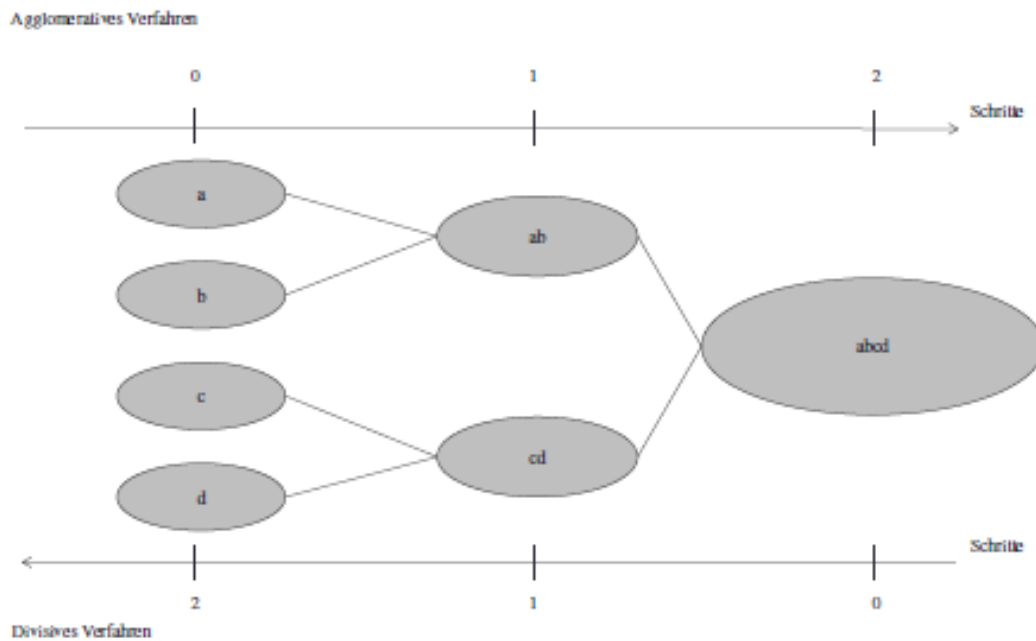


Abb. 16: Vergleich zwischen agglomerativem und divisivem Clustering

Hierarchische Verfahren generieren eine Hierarchie der Trainingsinstanzen, welche sich dann als Clustering interpretieren lässt. Da hierarchische Verfahren auf X_{Tr} operieren und nicht auf X , ändert sich das Clustering i. d. R., wenn neue Instanzen hinzukommen. Hierarchische Verfahren bilden Cluster, indem sie Gruppen von Trainingsinstanzen entweder aufspalten oder zusammenfügen. Ersterer Fall wird als divisives, letzterer Fall als agglomeratives Clusteringverfahren bezeichnet. Bei den divisiven Verfahren bildet X_{Tr} zu Beginn ein einziges Cluster. Mit jedem Schritt wird dieses Cluster unterteilt, bis letztlich die Anzahl der Cluster der Anzahl der Instanzen entspricht, d.h. dass es für jede Instanz ein eigenes Cluster gibt. Agglomerative Verfahren agieren gegensätzlich. Hier liegt zu Beginn in jedem Cluster genau eine Instanz. Über den Zeitverlauf werden diese schrittweise zusammengefügt, bis nach dem letzten Schritt genau ein Cluster übrig bleibt, in dem alle Instanzen liegen (Han & Kamber, 2006, S. 399). Abb. 16 zeigt das Vorgehen schematisch. Abb. 17 zeigt die verschiedenen Verfahren im Überblick.

Hierarchische Verfahren

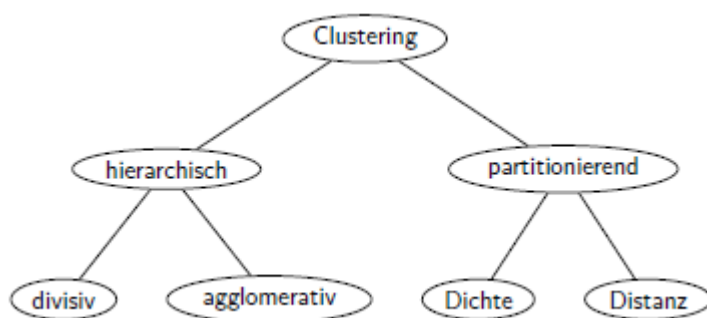


Abb. 17: Übersicht über Typen von Clusteringverfahren

Wie erwähnt, ist es das Ziel von hierarchischen Clusteringverfahren, eine Hierarchie der Trainingsinstanzen zu erzeugen. Anschaulich kann eine solche Hierarchie als Baum verstanden werden, wobei der Wurzelknoten alle Instanzen repräsentiert, und jeder Endknoten eine individuelle Instanz. Jeder innere Knoten steht für eine Untermenge des jeweils übergeordneten Knotens. Konzeptionell ähnelt dies einem EB, allerdings

Hierarchisch-agglomeratives Clustering

mit einem entscheidenden Unterschied: Während ein EB den Raum X partitioniert, werden beim hierarchischen Clustering nur die Trainingsinstanzen in eine Hierarchie gebracht. Die Instanzen, die durch einen bestimmten Knoten repräsentiert werden, können als einzelner Cluster interpretiert werden. Für die Beschreibung des hierarchisch-agglomerativen Clusterings soll der Begriff Cluster in der Bedeutung verwendet werden, die der zuvor genannten Anschauung entspricht. Das hierarchisch-agglomerative Clustering ist ein Algorithmus, dass einen solchen Baum erzeugt. Anders als die zuvor vorgestellten TDIDT-Algorithmen, arbeitet dieser Algorithmus bottom-up, d. h. er beginnt an den Endknoten mit der Baumkonstruktion. Dazu der Algorithmus umgangssprachlich in einer Version nach Backhaus, Erichson, Plinke und Weiber (2006, S. 514f.):

1. Beginne mit der Partitionierung, bei der jede Instanz einem Cluster entspricht.
2. Berechne die Distanzmatrix für alle Cluster.
3. Finde die beiden Cluster mit der geringsten Distanz zueinander, d. h. den minimalen, von 0 verschiedenen Wert in der Distanzmatrix.
4. Fasse diese beiden Cluster zu einem Cluster zusammen.
5. Wenn es noch mehr als einen Cluster gibt, wiederhole ab Schritt 2.

Berechnung der Distanzmatrix

Zur Durchführung und effizienten Implementierung von Schritt 2 ist zu klären, wie die Distanz zwischen Clustern, im Gegensatz zur Distanz zwischen einzelnen Instanzen, zu berechnen ist, und wie die Distanzmatrix in einer folgenden Iteration aus der Distanzmatrix der vorherigen Iteration berechnet werden kann. Zur zweiten Frage zunächst folgende Beobachtungen:

- Die Distanzmatrix sei o. B. d. A. untere Dreiecksmatrix, d. h. es gelte für von 0 verschieden Einträge stets $j < i$.
- Wenn ein Cluster i mit einem Cluster j zusammengefasst wird, so bleiben alle Einträge außer den Zeilen und Spalten i und j unverändert. Es ändert sich nur die Distanz zwischen dem neuen Cluster und allen anderen.
- Ebenfalls o. B. d. A. können also die i -te Zeile und Spalte der Matrix gelöscht werden und die neuen Einträge in die j -te Zeile bzw. Spalte geschrieben werden.

Seien i^*, j^* mit $j^* < i^*$ die Indizes der Cluster die zusammengefügt werden. Für die Distanzmatrix in der Iteration $t + 1$, $D^{(t+1)}$ ergeben sich die Einträge aus den Einträgen der vorherigen Iteration, $D^{(t)}$, nach der in (4.33) dargestellten Fallunterscheidung. Die Fallunterscheidung sieht kompliziert aus, letztlich geht es aber nur darum, die Indizes unter Beibehaltung der Dreiecksform korrekt zu übertragen. Die dargestellten Fälle fallen ihrerseits in drei Kategorien:

- $D_{ij}^{(t+1)}$ ist nach Definition der Nulleintrag, wenn $i = j$ oder $D_{ij}^{(t+1)}$ nicht Teil der unteren Dreiecksmatrix ist.
- $D_{ij}^{(t+1)}$ ist ein Eintrag von $D^{(t)}$, wenn weder i^* noch j^* berührt sind.

- $D_{i,j}^{(t+1)}$ lässt sich direkt aus den Einträgen von $D^{(t)}$ für i^* und j^* berechnen, wenn keiner der vorgenannten Fälle zutrifft.

Die Begründung für die Formel in der dritten Kategorie wird sogleich nachgereicht.

$$D_{i,j}^{(t+1)} = \begin{cases} D_{i,j}^{(t)} & i < j^* \wedge j < i \\ \alpha D_{i^*,j}^{(t)} + \beta D_{j^*,j}^{(t)} + \gamma D_{i^*,j^*}^{(t)} + \delta |D_{i^*,j}^{(t)} - D_{j^*,j}^{(t)}| & i = j^* \wedge j < i \\ D_{i,j}^{(t)} & j^* < i < i^* \wedge j < j^* \\ \alpha D_{i,j^*}^{(t)} + \beta D_{i^*,i}^{(t)} + \gamma D_{i^*,j^*}^{(t)} + \delta |D_{i,j^*}^{(t)} - D_{i^*,i}^{(t)}| & j^* < i < i^* \wedge j = j^* \\ D_{i,j}^{(t)} & j^* < i < i^* \wedge j^* < j < i \\ D_{i+1,j}^{(t)} & i^* \leq i \wedge j < j^* \\ \alpha D_{i+1,j^*}^{(t)} + \beta D_{i+1,j^*}^{(t)} + \gamma D_{i^*,j^*}^{(t)} + \delta |D_{i+1,j^*}^{(t)} - D_{i+1,j^*}^{(t)}| & i^* \leq i \wedge j = j^* \\ D_{i+1,j}^{(t)} & i^* \leq i \wedge j^* < j < i^* \\ D_{i+1,j+1}^{(t)} & i^* \leq i \wedge i^* \leq j < i \\ 0 & \text{sonst} \end{cases} \quad (4.33)$$

Um die Distanz zwischen zwei Clustern zu bestimmen, gibt es verschiedene Ansätze. Beim *single-linkage* Verfahren entspricht die Distanz zwischen zwei Clustern der minimalen paarweisen Entfernung zweier Instanzen, die sich in jeweils einem der beiden Cluster befinden. Gegensätzlich hierzu ist der *complete-linkage* Ansatz. Es wird die Entfernung der beiden Instanzen aus den beiden Clustern bestimmt, die maximal ist. Das *average-linkage* Verfahren wählt das Mittel der paarweisen Entfernungen. Das *centroid* Verfahren verwendet den Abstand der Mittelpunkte (Zentroiden, vgl. (4.35)) der jeweiligen Cluster. Das Verfahren nach *Ward* minimiert die Varianz innerhalb des neu entstehen Clusters. Das Verfahren nach Ward erzielt dabei oft die besten Ergebnisse, hat aber gleichzeitig die höchsten theoretischen Anforderungen. Wenn Zweifel an der metrischen Skalierung aller Attribute besteht sollte auf eines der drei zuerst genannten Verfahren zurückgegriffen werden. Alle genannten Verfahren können, trotz ihrer unterschiedlichen Motivation, durch Wahl von Faktoren $\alpha, \beta, \gamma, \delta$ auf die selbe, in (4.33) genannte, Formel zurückgeführt werden. Die entsprechenden Faktoren sind in Tab. 17 dargestellt, wobei B_{i^*} und B_{j^*} die zusammenzufügenden Cluster seien und B_c der Cluster, zu dem die Distanz des neuen Clusters berechnet werden soll (Backhaus et al., 2006, Kap. 8.2.2.2).

Distanz zwischen Clustern

Tab. 17: Distanzberechnung bei agglomerativem Clustering.
In Anlehnung an: (Backhaus et al., 2006, S. 517)

Verfahren	Faktor			
	α	β	γ	δ
Single	0,5	0,5	0	-0,5
Complete	0,5	0,5	0	0,5
Average	$\frac{ B_i }{ B_i + B_{j^*} }$	$\frac{ B_{j^*} }{ B_i + B_{j^*} }$	0	0
Centroid	$\frac{ B_i }{ B_i + B_{j^*} }$	$\frac{ B_{j^*} }{ B_i + B_{j^*} }$	$-\frac{ B_i \cdot B_{j^*} }{(B_i + B_{j^*})^2}$	0
Ward	$\frac{ B_i + B_c }{ B_i + B_{j^*} + B_c }$	$\frac{ B_{j^*} + B_c }{ B_i + B_{j^*} + B_c }$	$-\frac{ B_c }{ B_i + B_{j^*} + B_c }$	0

Dendrogramm

Das Zusammenfügen der Cluster kann graphisch durch sogenannte Dendrogramme abgebildet werden. Ein Dendrogramm entspricht der eingangs erwähnten Interpretation einer Hierarchie der Trainingsinstanzen als Baum. Dabei werden die Blätter des Baumes, also die einzelnen Instanzen, linksbündig dargestellt und die Kanten waagrecht eingezeichnet, wobei jedoch die Länge der Kanten proportional zur Distanz der jeweiligen Cluster gewählt wird.

Beispiel

Ein Supermarkt möchte seine Kunden mittels Clusteranalyse segmentieren. Zu jedem Kunden liegen Informationen über seine Markentreue, seine Innovationsbereitschaft sowie der Bereitschaft, erhöhte Preise zu zahlen, vor. Die drei Attribute sind durch ganze Zahlen im Intervall von eins bis sechs angegeben. Für das Beispiel soll – ggf. etwas realitätsfremd – angenommen werden, die Daten seien bei Erhebung intervallskaliert und eine weitere Transformation nicht erforderlich. Die Beispieldaten können Tab. 18 entnommen werden.

Tab. 18: Kundenmerkmale als Beispieldaten für hierarchisches Clustering

Kunde	Bereitschaft für		
	Markentreue	Innovation	Zahlung
A	5	3	4
B	3	2	4
C	3	1	1
D	2	6	1
E	1	6	2

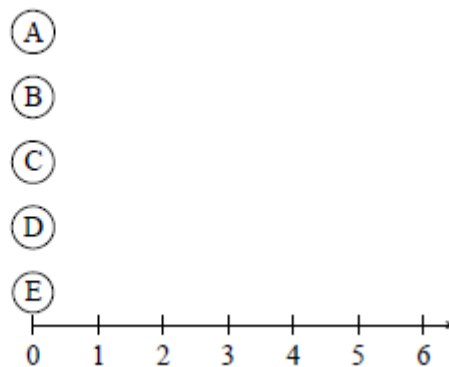
Konkretisierung des Verfahrens

Es soll an dieser Stelle beispielhaft das hierarchisch-agglomerative Clustering durchgeführt werden. Dabei können die verwendete Distanzmetrik und das verwendete Linkage-Verfahren unabhängig voneinander gewählt werden. Da es sich bei den gegebenen Werten nach Annahme um intervallskalierte Daten handelt, ist die Verwendung der Manhattan-Distanz hier sinnvoll. Auch sollte ein Linkage-Verfahren mit geringen Anforderungen gewählt werden, im Beispiel wird singlelinkage verwendet.

Initiale Distanzmatrix

Mittels der Manhattan-Distanz werden paarweise alle Entfernungen der Instanzen untereinander berechnet. So ist die Entfernung zwischen Kunde A und Kunde B z. B. gegeben durch $|5 - 3| + |3 - 2| + |4 - 4| = 3$. Auf alle Paare angewendet ergibt sich die Distanzmatrix $D^{(0)}$ mit dem zugehörigen unvollständigen Dendrogramm:

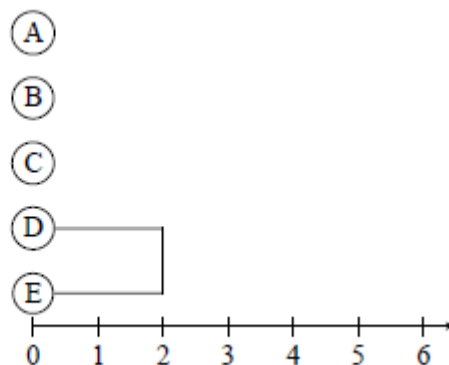
	A	B	C	D	E
A					
B	3				
C	7	4			
D	9	8	6		
E	9	8	8	2	



Unabhängig von Distanzmetrik und Linkage-Verfahren werden immer die beiden Cluster zusammengefasst, für welche der Eintrag in der Distanzmatrix minimal ist. In diesem Fall sind dies die Kunden D und E mit der Distanz 2. Die Distanzen zwischen den meisten Clustern bleiben unverändert. Neu ist nur die Distanz zwischen dem soeben gebildeten Cluster $\{D, E\}$ und allen anderen Clustern. Durch Anwendung der zuvor beschriebenen Formel ergibt sich, dass in diesem Fall nur die letzte Zeile bzw. Spalte der Matrix neu berechnet werden müssen. Die Berechnung für den letzten Eintrag lautet dabei z. B. $0,5 \cdot 6 + 0,5 \cdot 8 - 0,5 \cdot |6 - 8| = 6$. Dies ist erwartungsgemäß das Minimum der Einträge 6 und 8. Es ergibt sich eine neue Distanzmatrix und ein neuer Knoten im Dendrogramm in Höhe 2, da dies die Distanz zwischen den zusammengefassten Clustern war.

Cluster zusammenfassen

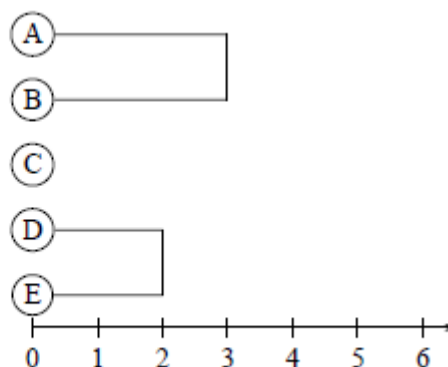
	A	B	C	D,E
A				
B	3			
C	7	4		
D,E	9	8	6	



Für die neue Distanzmatrix ist die kleinste Distanz diejenige zwischen A und B sodass diese Cluster ganz analog zum ersten Schritt zusammengefasst werden und sich wiederum eine neue Distanzmatrix ergibt:

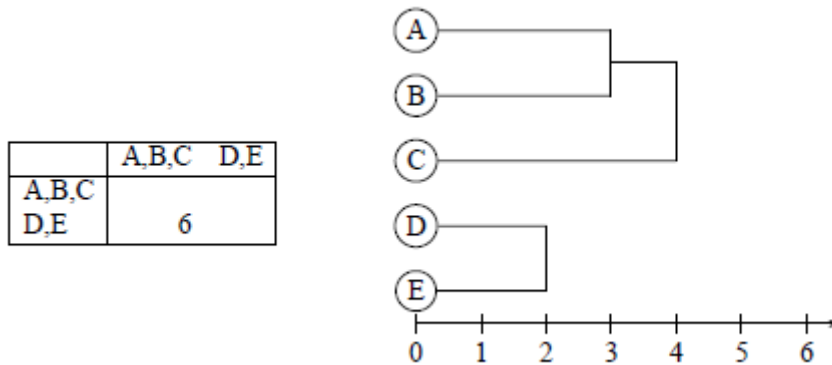
Erneutes Zusammenfassen

	A,B	C	D,E
A,B			
C	4		
D,E	8	6	



Abbruchbedingung

Der Vorgang wird fortgesetzt, bis entweder die gewünschte Anzahl an Clustern erreicht wurde, oder aber nur noch ein Cluster übrig ist, der alle Datensätze enthält. Aufgrund der extrem kleinen Datenbasis ist dieser Punkt im Beispiel natürlich schnell erreicht. Die letzte Distanzmatrix ist:



Interpretation

Fünf Kunden in Segmente aufteilen zu wollen, ist offensichtlich nicht besonders sinnvoll. In realistischeren Beispielen mit größeren Datenmengen stellt sich zunächst die Frage, bei wie vielen Clustern davon auszugehen ist, dass hier die „natürliche“ Segmentierung erreicht wurde. Diese Frage lässt sich nicht allgemein beantworten, sondern ist für jede Entscheidungssituation neu zu analysieren. Im vorliegenden Fall könnte z. B. bottom-up argumentiert werden, dass die Distanzen zwischen den Clustern erst gleichmäßig in Einerschritten ansteigen, dann aber im letzten Schritt „sprunghaft“ (sofern eben bei dieser kleinen Datenmenge möglich) direkt um zwei Einheiten wächst. Backhaus et al. (2006, S. 536) bezeichnen dies als „Elbow“-Kriterium. Ein solcher „Ellenbogen“ in der Entwicklung des Distanzmaßes könnte ein Indiz dafür sein, dass es sich tatsächlich um zwei Kundensegmente $\{A, B, C\}$ und $\{D, E\}$ handelt und nicht etwa nur um fünf Kunden desselben Segments. Andererseits merken Backhaus et al. (2006, S. 536) an, dass ein solcher Sprung in der letzten Zusammenfassung zu einem Cluster immer auftritt. Aussagekräftiger sind daher Sprünge, die bereits bei größeren Clusterzahlen auftreten. Ein solcher Sprung ist in diesem Beispiel nicht zu verzeichnen. Alternativ könnte top-down eine Vermutung darüber getroffen werden, wie viele Kundensegmente das Unternehmen bedient, und dann analysiert werden, wann und wie sich diese Anzahl von Clustern einstellt. Ein einfaches graphisches Hilfsmittel zur Interpretation der Ergebnisse des Clusterings ist das vollständige Dendrogramm (Abb. 18). Dabei werden zwei Cluster gerade an der Stelle der Skala graphisch, d. h. als Baumstruktur, zusammengefasst, welche ihrer Distanz zueinander entspricht. Im Beispiel also an den Stellen 2, 3, 4 und 6.

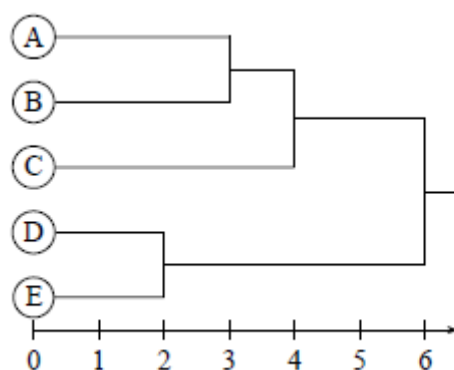


Abb. 18: Beispiel für ein Dendrogramm

Als Herleitung für den k -means Algorithmus, sei folgende Idee, in Anlehnung an (Ester & Sander, 2000, S. 51f.), vorgestellt: Jeder Cluster kann durch den Zentroid der darin liegenden Trainingsinstanzen repräsentiert werden. Sei $X = B_1 \cup \dots \cup B_k$ eine Partitionierung von X . Diese Partitionierung induziert ein Clustering $C_1, \dots, C_k$ von X_{Tr} mit:

Zentroidbasierte Partitionierung

$$C_i = \text{TrainingInstances}(B_i) \quad (\text{vgl. Alg. 1}) \quad (4.34)$$

Der Zentroid eines Clusters C_i , μ_{C_i} sei definiert als Mittelwertvektor der enthaltenen Trainingsinstanzen, d.h.

$$\mu_{C_i} = (\bar{x}_{C_i,1}, \dots, \bar{x}_{C_i,N_Q}) \quad \text{mit:} \quad (4.35)$$

$$\bar{x}_{C_i,j} = \frac{1}{|C_i|} \sum_{x_c \in C_i} x_{c,j} \quad (\text{vgl. (3.26)}) \quad (4.36)$$

Umgekehrt induzieren die Zentroide eine Partitionierung. Ein wichtiges Prinzip beim Clustering ist es, die Varianz innerhalb der Cluster möglichst gering zu halten. Dies entspricht der zentralen Idee der Clusteringverfahren, es wurde aber auch bei der Diskussion der LDA schon anschaulich (vgl. Abb. 9). Die Varianz innerhalb eines Clusters ist gegeben durch

$$\text{Var}(C_i) = \frac{1}{|C_i|} \sum_{x_c \in C_i} \text{dist}^2(x_c, \mu_{C_i}) \quad (4.37)$$

Offensichtlich ist die Varianz über alle Cluster dann minimal, wenn jede Instanz demjenigen Cluster zugeordnet wird, für das ihr Abstand zum Zentroiden des Clusters minimal ist, d.h.

$$B_i = \left\{ x_c \in \mathcal{X} \mid C_i = \arg \min_C \{ \text{dist}(x_c, \mu_C) \} \right\} \quad (4.38)$$

Nicht jede Partitionierung von X erfüllt die zuvor genannten Bedingungen. Eine Partitionierung, für die (4.34) und (4.38) *gleichzeitig* gelten, sei daher als *stabile* Partitionierung bezüglich der Zentroiden der Trainingsinstanzen bezeichnet. Für gegebenes X_{Tr} ist die stabile Partitionierung zudem nicht unbedingt eindeutig, es kann also mehrere Partitionierungen geben, welche die genannten Bedingungen erfüllen. Anders ausgedrückt: Das in (4.38) konstruierte Minimum ist ggf. ein lokales Minimum bezüglich der Gesamtheit aller möglichen Partitionierungen.

Stabile Partitionierung

Algorithmus 8: Clustering durch Varianzminimierung

Quelle: in Anlehnung an (Ester & Sander, 2000, S. 53)

Input : $\mathcal{X}, \text{dist}, X_{Tr}, k$
Output : $(B_1, \dots, B_k)$
Function VarMin():
 $t \leftarrow 0$
 foreach $i \in \{1, \dots, k\}$ **do**
 $\mu_{C_i}^{(0)} \leftarrow x_i \in X_{Tr}$
 repeat
 $t \leftarrow t + 1$
 foreach $i \in \{1, \dots, k\}$ **do**
 $B_i^{(t)} \leftarrow \{x_c \in \mathcal{X} \mid C_i = \text{argmin}_C \{ \text{dist}(x_c, \mu_C^{(t-1)}) \} \}$
 $C_i^{(t)} \leftarrow \text{TrainingInstances}(B_i^{(t)})$
 $\mu_{C_i}^{(t)} \leftarrow (\bar{x}_{C_i^{(t)},1}, \dots, \bar{x}_{C_i^{(t)},N_Q})$
 until $\forall i \in \{1, \dots, k\} : \mu_{C_i}^{(t)} = \mu_{C_i}^{(t-1)}$
 return $B_1^{(t)}, \dots, B_k^{(t)}$

Varianzminimieren- des Partitionie- rungsverfahrens

Gesucht ist also, gemäß obiger Herleitung, ein Algorithmus, welcher automatisch eine stabile Partitionierung erzeugt. Der allgemeine Algorithmus, der dies tut, sei hier als varianzminimierender Algorithmus bezeichnet. Häufig wird Alg.

8 auch als k -means Algorithmus bezeichnet, obwohl dies streng genommen falsch ist, da k -means i. e. S. eine Variante darstellt. Zwei Eigenschaften sind jedoch in jedem Fall grundlegend: Erstens, die Anzahl der Cluster k muss dem Algorithmus als Parameter vorgegeben werden. Zweitens, der Algorithmus wird B und C solange abwechselnd aktualisieren, bis die Partitionierung stabil ist. Die Initialisierung der Cluster bzw. der Zentroiden ist dabei zufällig. Es ist jedoch naheliegend, zu Anfang Zentroide vorzugeben, die auch tatsächlich in X_{Tr} vorkommen, um abseitige Werte zu vermeiden. Unter der Annahme, dass die Indizierung der Trainingsinstanzen ohnehin zufällig ist, können einfach die ersten k Instanzen verwendet werden.

Algorithmus 9: k-means Clustering

Quelle: in Anlehnung an: (Ester & Sander, 2000, S. 54)

Input : $\mathcal{X}, dist, X_{Tr}, I_{Tr}, k$
Output : $(B_1, \dots, B_k)$
Function KMeans():

```

  Initialisiere Cluster zufällig und bestimme deren Zentroide
  repeat
    stable  $\leftarrow$  true
    foreach  $l \in I_{Tr}$  do
       $i^* \leftarrow i \in \{1, \dots, k\} : x_l \in C_i$ 
       $j^* \leftarrow i^*$ 
      foreach  $j \in \{1, \dots, k\}$  do
        if  $dist(x_l, \mu_{C_j}) < dist(x_l, \mu_{C_{j^*}})$  then
           $j^* \leftarrow j$ 
      if  $i^* \neq j^*$  then
        stable  $\leftarrow$  false
         $C_{i^*} \leftarrow C_{i^*} \setminus \{x_l\}$ 
         $C_{j^*} \leftarrow C_{j^*} \cup \{x_l\}$ 
         $\mu_{C_{i^*}} \leftarrow \frac{1}{|C_{i^*}|} ((|C_{i^*}| + 1) \mu_{C_{i^*}} - x_l)$ 
         $\mu_{C_{j^*}} \leftarrow \frac{1}{|C_{j^*}|} ((|C_{j^*}| - 1) \mu_{C_{j^*}} + x_l)$ 
    until stable
    foreach  $i \in \{1, \dots, k\}$  do
       $B_i \leftarrow \{x_c \in \mathcal{X} | C_i = \arg \min_C \{dist(x_c, \mu_C)\}\}$ 
  return  $B_1, \dots, B_k$ 

```

Der soeben vorgestellte Algorithmus wurde nach didaktischen Kriterien präsentiert. Er ist daher nur als prinzipielles Vorgehen zu verstehen. Der k -means Algorithmus i. e. S. bezeichnet eine geschickte Implementierung dieses Prinzips für die Praxis. Das Ziel des Algorithmus ist es, eine stabile Partitionierung zu finden. Es erscheint also naheliegend, gezielt nach solchen Instanzen zu suchen, welche das Stabilitätskriterium verletzen. Gesucht sind also x^* mit der Eigenschaft $x^* \in C_i$ und $\exists j \neq i : dist(x^*, \mu_{C_j}) < dist(x^*, \mu_{C_i})$. Ist ein solches x^* einmal gefunden, so kann die Situation unmittelbar verbessert werden, indem x^* aus C_i entfernt und zu C_j hinzugefügt wird. B ist für dieses Update nicht erforderlich. Es ändern sich dadurch nur μ_{C_i} und μ_{C_j} , diese können aber direkt neu berechnet werden. Eine strikte Diskretisierung der einzelnen Iterationen ist also auch nicht erforderlich. Zu beachten ist nur, dass der Algorithmus dadurch sensitiv für die Reihenfolge der Trainingsinstanzen wird. Der resultierende Algorithmus ist in Alg. 9 dargestellt.

| **k-means Algorithmus** |

4.6 Apriori-Algorithmus

Das Ziel des Apriori-Algorithmus ist die Erkennung von „wenn-dann“ Regeln, also das Auftreten eines oder mehrerer Items als *Konsequenz* aus dem Auftreten eines oder mehrerer anderer Items. Um dies untersuchen zu können, muss X Information darüber enthalten, welche Items gemeinsam aufgetreten sind. Eine einzelne Instanz könnte z. B. ein Einkauf in einem Supermarkt sein (vgl. Tab. 14). So könnte beispielsweise eine Analyse von Kundeneinkäufen in einem Supermarkt ergeben, dass wenn ein Kunde Bier kauft, er dann auch Chips kauft. Diese Regel würde dann als $Bier \rightarrow Chips$ dargesellt. Allgemein: Seien I_a und I_b Mengen von Items. Dann hat eine Regel die Form

| **Zielsetzung** |

$$I_a \rightarrow I_b \quad (4.39)$$

und der Regelkopf I_a heißt Bedingung (Prämisse) und der Regelrumpf I_b heißt Konsequenz (Konklusion). Die einzelnen Items in den Mengen sind dabei mit einem logischen *und* verknüpft, d. h. die Regel besagt: „Wenn alle Items in I_a auftreten, dann treten auch alle Items in I_b auf.“ (Shmueli et al., 2010, Kap. 13.3).

Notation | Jedes mögliche Item entspreche genau einem Attribut in Q mit $V_q = \{0,1\} \forall q \in Q$. Für eine gegebene Instanz x_i gelte $x_{i,q} = 1$, wenn das zu $q \in Q$ gehörige Item in x_i auftritt und $x_{i,q} = 0$, wenn nicht. Mengen von Items, sogenannte Itemsets, können nun als Untermengen von Q notiert werden. Um von Instanzen zu Itemsets zu kommen bedarf es einer Hilfsfunktion

$$Q_+ : \mathcal{X} \rightarrow \mathcal{P}(Q) \quad (4.40)$$

$$x_i \mapsto \{q \in Q \mid x_{i,q} = 1\} \quad (4.41)$$

Support | Seien $I_a, I_b \subset Q$ Itemsets mit $I_a \cap I_b \neq \emptyset$. Eine Instanz x_i *erfüllt* die Regel $I_a \rightarrow I_b$ wenn gilt: $I_a \cup I_b \subseteq Q_+(x_i)$. Diejenigen Instanzen, welche eine Regel erfüllen, „unterstützen“ diese Regel. Die Menge dieser Instanzen heißt daher *Supportmenge*. Wieder bietet sich eine Hilfsfunktion an, um von Itemsets zurück zu Instanzen zu kommen:

$$X_{Sup} : \mathcal{P}(Q) \rightarrow \mathcal{X} \quad (4.42)$$

$$I \mapsto \{x_i \in X_{Tr} \mid I \subseteq Q_+(x_i)\} \quad (4.43)$$

Der *Support* einer Regel ist nun definiert als der Anteil der Supportmenge dieser Regel an der Trainingsmenge, d.h.

$$support(I_a \rightarrow I_b) = \frac{|X_{Sup}(I_a \cup I_b)|}{|X_{Tr}|} \quad (4.44)$$

Offensichtlich gibt es sehr viele mögliche Regeln, sog. Regelkandidaten, und nicht alle sind gleich sinnvoll. Ein Parameter des Apriori-Algorithmus ist daher der *Mindestsupport*, der angibt, ab welchem Supportwert Regelkandidaten überhaupt berücksichtigt werden sollen (Bollinger, 1996).

Konfidenz | Nur weil zwei Itemsets I_a, I_b häufig gemeinsam auftreten, wie es durch den Support gemessen wird, bedeutet dies noch nicht, dass I_b immer dann auftritt, wenn auch I_a auftritt, wie es die Regel $I_a \rightarrow I_b$ besagen würde. Die gemeinsamen Auftreten müssen also ins Verhältnis zu *allen* Auftreten von I_a gesetzt werden. Dies misst die *Konfidenz* der Regel:

$$confidence(I_a \rightarrow I_b) = \frac{|X_{Sup}(I_a \cup I_b)|}{|X_{Sup}(I_a)|} \quad (4.45)$$

Ein Konfidenzwert von 1 bedeutet dass I_b immer dann auftritt, wenn I_a auftritt, was eine zugrundeliegende Regel sehr wahrscheinlich macht. Die Umkehrung gilt jedoch nicht, d. h. $confidence(I_a$

$\rightarrow I_b) \neq \text{confidence}(I_b \rightarrow I_a)$. Auch eine *Mindestkonfidenz* kann dem Apriori-Algorithmus als Parameter übergeben werden (Bollinger, 1996).

Algorithmus 10: Apriori-Algorithmus

Quelle: in Anlehnung an (Wu et al., 2008)

```

Input :  $X_{Tr}, Q, \text{minsup}, \text{minconf}$ 
Output : Regelmenge  $R$ 
Function Apriori():
     $I_1 = \{ \{q\} \subset Q : \frac{|X_{\text{sup}}(\{q\})|}{|X_{Tr}|} \geq \text{minsup} \}$ 
     $k \leftarrow 1$ 
     $R \leftarrow \emptyset$ 
    while  $I_k \neq \emptyset$  do
         $I_{k+1} \leftarrow \emptyset$ 
        forall the  $I = \{q_1, \dots, q_{k-1}, q_k\} \in I_k$  do
            forall the  $I' = \{q_1, \dots, q_{k-1}, q'_k\} \in I_k$  do
                 $I_c \leftarrow I \cup I' = \{q_1, \dots, q_{k-1}, q_k, q'_k\}$ 
                if  $\frac{|X_{\text{sup}}(I_c)|}{|X_{Tr}|} \geq \text{minsup}$  then
                     $I_{k+1} \leftarrow I_{k+1} \cup \{I_c\}$ 
             $k \leftarrow k + 1$ 
            forall the  $I \in I_k$  do
                forall the  $I_a, I_b \subset Q : I_a \cap I_b = \emptyset \wedge I_a \cup I_b = I$  do
                    if  $\text{confidence}(I_a \rightarrow I_b) \geq \text{minconf}$  then
                         $R \leftarrow R \cup \{I_a \rightarrow I_b\}$ 
    return  $R$ 

```

Der Apriori-Algorithmus ist ein geschickter Algorithmus zum auffinden *häufiger* Itemsets, d. h. solcher Itemsets, welche den Mindestsupport erreichen, in der Menge aller Itemsets $P(Q)$. Dabei macht sich der Algorithmus eine einfache Beobachtung zunutze: Wenn I ein häufiges Itemset ist, dann ist auch jede Untermenge $I' \subset I$ häufig. Insbesondere gilt: Wenn I häufig ist und $I = I'_a \cup I'_b$, dann sind auch I'_a und I'_b häufig. Wieder gilt die Umkehrung nicht, d. h. aus I_a, I_b sind häufig folgt *nicht*, dass $I_a \cup I_b$ häufig ist. Anders ausgedrückt: Jedes häufige Itemset /mit $|I| > 1$ muss die Vereinigung zweier anderer häufiger Itemsets sein, aber nicht jede Vereinigung zweier Itemsets ist häufig. Sei I_k die Menge der häufigen Itemsets, mit der Eigenschaft $|I| = k \forall I \in I_k$. Der Apriori-Algorithmus (Alg. 10) generiert sukzessive Itemsets der Größe $k + 1$ aus häufigen Itemsets der Größe k indem er Paare von Itemsets aus I_k vereinigt, welche sich in genau einem Element unterscheiden. Diese überprüft er dann bezüglich ihrer Häufigkeit.

Generierung häufiger Itemsets

Tab. 19: Beispieldaten für den Apriori-Algorithmus

Item	Bier	Chips	Nudeln	Saft	Salat
Transaktion	q_1	q_2	q_3	q_4	q_5
x_1	•	•		•	•
x_2		•	•	•	
x_3	•		•		
x_4		•	•	•	
x_5			•	•	
Anzahl	2	3	4	4	1
Support[%]	40	60	80	80	20
I_1	•	•	•	•	

Beispiel

Anhand eines Beispiels soll die Funktionsweise des Apriori- Algorithmus illustriert werden. Der Mindestsupport wird auf 40 % der Transaktionen, die Mindestkonfidenz auf 60 % festgesetzt. Es liegen Beispieldaten über fünf Transaktionen vor (Tab. 19). Jede Transaktion beinhaltet zwei oder mehr Items in Form von gekauften Artikeln. Im ersten Schritt wird zunächst für jedes Item die absolute Häufigkeit bestimmt, um daraus abgeleitet den Support mittels (4.44) zu bestimmen. Das Item „Salat“ erzielt dabei einen Wert unterhalb des Mindestsupports und wird somit nicht in die Menge I_1 aufgenommen und folglich bei der Generierung nicht weiter berücksichtigt. Für alle zwei-Item Kombinationen wird nun erneut der Support bestimmt. Alle zwei-Item Kombinationen mit dem Artikel „Bier“ erreichen den Mindestsupport nicht und werden bei der Generierung der drei-Item Kombinationen außer Acht gelassen.

	$\{q_1, q_2\}$	$\{q_1, q_3\}$	$\{q_1, q_4\}$	$\{q_2, q_3\}$	$\{q_2, q_4\}$	$\{q_3, q_4\}$
x_1	•		•		•	
x_2				•	•	•
x_3		•				
x_4				•	•	•
x_5						•
Anzahl	1	1	1	2	3	3
Support[%]	20	20	20	40	60	60
I_2				•	•	•

Nachdem die Menge I_2 bestimmt wurde, werden daraus Regeln abgeleitet und auf die festgelegte Mindestkonfidenz hin überprüft. Regeln, welche die Mindestkonfidenz erfüllen, werden in die Ergebnismenge aufgenommen.

	$q_2 \rightarrow q_3$	$q_3 \rightarrow q_2$	$q_2 \rightarrow q_4$	$q_4 \rightarrow q_2$	$q_3 \rightarrow q_4$	$q_4 \rightarrow q_3$
Konf[%]	67	50	100	75	75	75

Im nächsten Schritt werden alle möglichen drei-Item Generierungen durchgeführt. Gemäß dem Apriori-Algorithmus müssen dazu diejenigen Itemsets in I_2 berücksichtigt werden, für welche das erste Item gleich und das zweite unterschiedlich ist. Dies ist nur für das Paar $\{q_2, q_3\}$ und $\{q_2, q_4\}$ der Fall. $\{q_3, q_4\}$ muss dagegen nicht weiter berücksichtigt werden, was auch unsinnig wäre, da dadurch keine neue Kombination entstehen würde, sondern nur die bereits vorhandene Kombination $\{q_2, q_3, q_4\}$. Diese „Einsparung“ macht den Apriori-Algorithmus aus und ist bei realistisch großen Datensätzen natürlich größer. Auch das Itemset $\{q_2, q_3, q_4\}$ erfüllt den Mindestsupport.

Aus der Menge I_3 abgeleitet ergeben sich sechs mögliche Regeln, wovon jedoch nur vier auch die Mindestkonfidenz erfüllen.

	$q_2 \rightarrow q_3, q_4$	$q_3, q_4 \rightarrow q_2$	$q_3 \rightarrow q_2, q_4$	$q_2, q_4 \rightarrow q_3$	$q_4 \rightarrow q_2, q_3$	$q_2, q_3 \rightarrow q_4$
K. [%]	67	67	50	67	50	100

Abschließend ergeben sich die in Tab. 20 aufgelisteten Regeln.

Tab. 20: Liste der im Beispiel gefundenen Regeln

Prämisse		Konklusion
Chips	→	Nudeln
Chips	→	Saft
Saft	→	Chips
Nudeln	→	Saft
Saft	→	Nudeln
Chips	→	Nudeln, Saft
Nudeln, Saft	→	Chips
Chips, Saft	→	Nudeln
Chips, Nudeln	→	Saft

4.7 Künstliche Neuronale Netze

Neuronale Netze sind der Untersuchungsgegenstand verschiedener Fachrichtungen aus allen Zweigen der Wissenschaft. KNN sind dabei vereinfachende Modelle biologischer Nervensysteme zum Zwecke der Mustererkennung und des ML. Auch wenn Bezeichnungen und Funktionsweise klar biologisch inspiriert sind, so abstrahieren KNN doch große Teile der biologischen Realität hinfert und sind daher nicht mit einer Simulation realer Neurone zu verwechseln (Lusti, 2002, S. 316). Daher soll die Herleitung von KNN hier auch aus Richtung des ML und nicht der Biologie erfolgen.

Begriffsabgrenzung

Sowohl die logistische Regression (vgl. Tab. 9 und Abschnitt 4.4), als auch die LDA (vgl. Tab. 10 und Abb. 9) klassifizieren Instanzen danach, auf welcher Seite einer linearen Entscheidungsgrenze sie liegen. Algebraisch: Sei in der zuvor verwendeten Notation $X \subseteq \mathbb{R}^{N_{Q_f}}$ und $Y = \{-1, 1\}$. Eine Hyperebene ist die Generalisierung einer Ebene in beliebige Dimensionen. Sie hat stets eine Dimension weniger als der Raum, also in 2D eine Gerade, in 3D eine Ebene etc. Eine Hyperebene h ist gegeben durch alle Punkte $\mathbf{x} \in \mathbb{R}^{N_{Q_f}}$, für die gilt:

Separierende Hyperebenen

$$h : \mathbf{w}_0 + \mathbf{w}\mathbf{x} = 0 \quad (4.46)$$

Dies ist eine einfache Ebenengleichung mit Normalenvektor $\mathbf{w}$. Die Modellklasse H ist hier die Menge aller Hyperebenen gegeben durch den Parameter $\theta = (\mathbf{w}_0, \dots, \mathbf{w}_{N_{Q_f}})$. Trainingsdaten X_{Tr} heißen *linear separierbar* wenn eine Hyperebene h^* existiert, sodass gilt:

$$\forall \mathbf{x}_i \in X_{Tr} : x_{i,q_g} = \text{sgn}(\mathbf{w}_0 + \mathbf{w}\mathbf{x}_{i,Q_f}) \quad (4.47)$$

Wird x_i in die Ebenengleichung eingesetzt, dann gibt das Vorzeichen des Ergebnisses an, auf welcher Seite der Hyperebene x_i liegt. Lineare Separierbarkeit bedeutet, dass alle Instanzen der positiven Klasse auf der einen und alle Instanzen der negativen Klasse auf der anderen Seite der Hyperebene liegen. Eine Hyperebene h^* , welche diese Bedingung erfüllt heißt entsprechend *separierende Hyperebene*. Logistische Regression und LDA garantieren jedoch nicht, eine separierende Hyperebene zu finden, selbst wenn X_{Tr} linear separierbar ist (Hastie et al., 2001, S. 106f.).

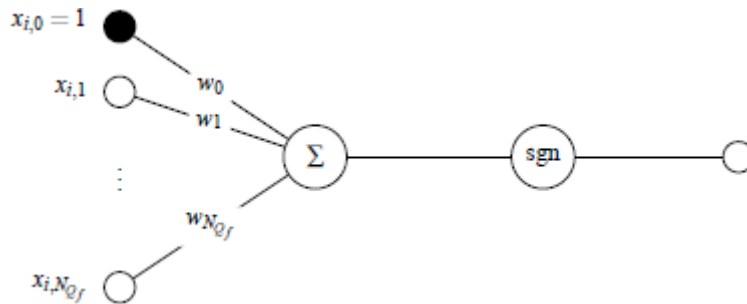


Abb. 19: Das Perceptron als idealisiertes Neuron.
Quelle: entnommen aus (Mitchell, 1997, S. 87).

Perceptron

Ein Perceptron (Abb. 19) ist ein idealisiertes Neuron. Es besteht aus mehreren Eingängen (biolog. Dendriten), der Signalverarbeitung (biolog. Zellkörper) und einem Ausgang (biolog. Axon) (Lusti, 2002, S. 317 u. 320). Der erste Eingang dient dabei nur der Behandlung von w_0 und wird grundsätzlich auf 1 gesetzt. An den anderen Eingängen werden die Werte der einzelnen Attribute (biolog. Erregungspotentiale) angelegt. Das Neuron summiert die Erregungspotentiale $x_{i,j}$ auf, und multipliziert sie dabei mit ihren Gewichten w_j . Die Gewichte können dabei biologisch als erregend bzw. hemmend interpretiert werden. Wenn die gewichtete Summe der Erregungspotentiale 0 übersteigt, „feuert“ das Neuron, d. h. am Ausgang liegt 1 an. Sonst -1 . Das Perceptron ist somit eine Nachbildung der separierenden Hyperebene (Lusti, 2002, Kap. 8.2) (Mitchell, 1997, Kap. 4.4).

Algorithmus 11: Perceptron-Algorithmus

Input : X_{Tr}, Q_f, q_g
Output : $w_0, \mathbf{w}$
Function Perceptron-Learning():
 $w_0, \mathbf{w} \leftarrow \text{random}$
 while $\exists x_i \in X_{Tr} : w_0 + \mathbf{w}x_{i,Q_f} \neq x_{i,q_g}$ **do**
 $\Delta \mathbf{w} \leftarrow \mathbf{0}$
 foreach $x_i \in X_{Tr}$ **do**
 $out \leftarrow w_0 + \mathbf{w}x_{i,Q_f}$
 $\Delta w_0 \leftarrow \Delta w_0 + \eta(out - x_{i,q_g})$
 foreach $q \in Q_f$ **do**
 $\Delta w_q \leftarrow \Delta w_q + \eta(out - x_{i,q_g})x_{i,q}$
 $w_0 \leftarrow w_0 + \Delta w_0$
 foreach $q \in Q_f$ **do**
 $w_q \leftarrow w_q + \Delta w_q$
 return $w_0, \mathbf{w}$

Gesucht ist also ein Algorithmus, der zu gegebenem X_{Tr} eine separierende Hyperebene findet bzw. die Gewichte für das entsprechende Perceptron. Der hier vorgestellte Perceptron-Algorithmus (Alg. 11) tut dies für linear separierbare Daten. Die SVM (Abb. 12) ist eine Generalisierung dieser Idee auf linear nichtseparierbare Daten. Die Idee ist dabei, mit einem zufälligen Gewichtsvektor zu beginnen und die Gewichte schrittweise um einen kleinen Betrag anzupassen, wenn eine Instanz fehlerklassifiziert wird. Die Richtung der Anpassung ergibt sich dabei aus der als korrekt bekannten Klasse der jeweiligen Instanz. Die Größe der Anpassung ergibt sich durch Multiplikation mit einer Lernrate (η). Es handelt sich also um überwachtes Lernen.

Perceptron-Algorithmus

Aus dem Perceptron-Modell folgt: „Ein Neuron ist [in diesem Kontext] eine lernfähige Verarbeitungseinheit.“ (Lusti, 2002, S. 319). Die Mächtigkeit eines einzelnen Neurons ist begrenzt. Ein neuronales Netz verbindet mehrere Neuronen miteinander, indem die Ausgänge von Neuronen mit den Eingängen anderer Neuronen verbunden werden. Aus pragmatischen Gründen würde dazu nicht das Perceptron an sich verwendet, sondern eine differenzierbare Sigmoidfunktion (Mitchell, 1997, S. 96). Dies kann z. B. wieder die logistische Funktion (vgl. (4.24)) oder eine andere Skalierung des Tangens Hyperbolicus sein:

Mehrschichtige Netzwerke

$$f(t) = \frac{1}{1 - e^{-t}} = \frac{1}{2} \left(1 + \tanh \frac{t}{2} \right) \quad (4.48)$$

Ein mehrschichtiges KNN besteht aus einer Eingabeschicht, einer oder mehreren verborgenen Schicht und einer Ausgabeschicht. Jedes Neuron ist genau einer der Schichten zugeordnet. Neuronale Verbindungen sind nur zwischen den Ausgängen der vorherigen Schicht und den Eingängen der nächsten Schicht zulässig, niemals innerhalb einer Schicht oder zurück zu einer vorherigen. Dadurch ist das KNN kreisfrei. Das Signal läuft stets von der Eingabeschicht zur verborgenen Schicht zur Ausgabeschicht (Feedforward-Netzwerk) (Lusti, 2002, S. 325).

Ein mehrschichtiges Feedforward-KNN kann lernen, indem die Eingabe zunächst vorwärts durch das Netzwerk propagiert wird. Dabei wird die Ausgabe für alle Neurone bestimmt. An der Ausgabeschicht kann der Fehler direkt gemessen werden. Dann wird der Fehler rückwärts in das Netzwerk propagiert, sodass auch die Gewichte der vorderen Schichten analog zum Perceptron-Algorithmus angepasst werden können (Mitchell, 1997, S. 98). KNN, die nach diesem Prinzip aufgebaut und trainiert sind, können mit nur einer oder maximal zwei verborgenen Schichten im Prinzip jede Funktion approximieren. Die wesentliche Limitation besteht darin, dass nicht bekannt ist, wie viele Neurone genau nötig sind und die Laufzeit mit steigender Neuronenzahl schnell anwächst (Mitchell, 1997, S. 105). Einer weiteren Interpretation sind die gelernten Gewichte oft nicht zugänglich, sodass KNN in der Praxis häufig als Blackbox eingesetzt werden.

Backpropagation-Algorithmus

4.8 Weitere Klassifizierungsverfahren

Sowohl die logistische Regression als auch das Perceptron mit logistischer Übertragungsfunktion geben im Zweiklassenfall eine geschätzte Wahrscheinlichkeit dafür an, dass eine gegebene Instanz zur positiven Klasse gehört, also

Bayes Klassifikatoren

$$h(x_{i,Q_f}) \propto \mathbb{P}(x_{i,q_g} = 1 | x_{i,Q_f}) \quad (4.49)$$

Allgemein: Sei $Y = \{C_1 \dots C_C\}$ Wäre die wahre a posteriori Wahrscheinlichkeit $\mathbb{P}(x_{i,q_g} = C | x_{i,q_f})$ bekannt, wäre der beste mögliche Klassifikator derjenige der diese maximiert also

$$h(x_{i,q_f}) = \arg \max_{C \in Y} \left\{ \mathbb{P}(x_{i,q_g} = C | x_{i,q_f}) \right\} \quad (4.50)$$

Dies ist der Bayes Klassifikator (benannt nach dem Satz von Bayes). Da die Verteilung aber unbekannt ist, wird sie von verschiedenen Verfahren, wie den zuvor genannten, geschätzt. Jedes Verfahren leitet dabei die Schätzung anders her. Der *naïve* Bayes Klassifikator, nicht zu verwechseln mit dem Bayes Klassifikator, wagt dagegen eine Schätzung indem die Situation haarsträubend naiv vereinfacht wird. Theoretisch entbehren diese Vereinfachungen in den meisten Fällen jeglicher Grundlage, empirisch führt es aber oft trotzdem zu guten Ergebnissen.

Naive Bayes

Gesucht ist $\mathbb{P}(x_{i,q_g} = C | x_{i,q_f})$. Nach dem Satz von Bayes gilt:

$$\mathbb{P}(x_{i,q_g} = C | x_{i,q_f}) = \frac{\mathbb{P}(x_{i,q_f} | x_{i,q_g} = C) \mathbb{P}(x_{i,q_g} = C)}{\mathbb{P}(x_{i,q_f})} \quad (4.51)$$

Da der Nenner gar nicht von C abhängt, genügt es den Zähler zu maximieren. Der Zähler ist zwar auch nicht bekannt, doch in einer ersten naiven Annahme wird geschätzt

$$\mathbb{P}(x_{i,q_g} = C) = \frac{|\{x_i \in X_{Tr} | x_{i,q_g} = C\}|}{|X_{Tr}|} \quad (4.52)$$

Diese Annahme ist meistens unproblematisch. Es fehlt nun noch $\mathbb{P}(x_{i,q_f} | x_{i,q_g} = C)$. Die *wirklich* naive Annahme ist es, die einzelnen Attribute seien nur von der Klasse abhängig und, gegeben eine Klasse, voneinander unabhängig. Dann wäre $\mathbb{P}(x_{i,q_f} | x_{i,q_g} = C)$ das Produkt aus den Wahrscheinlichkeiten der einzelnen Attribute:

$$\mathbb{P}(x_{i,q_f} | x_{i,q_g} = C) = \prod_{q \in Q_f} \mathbb{P}(x_{i,q} = v | x_{i,q_g} = C) \quad (4.53)$$

Doch damit nicht genug. Auch $\mathbb{P}(x_{i,q} = v | x_{i,q_g} = C)$ wird naiv aus den Trainingsdaten geschätzt. Wenn q qualitativ ist, wird der Anteil von Wert $v \in V_q$ an der Klasse verwendet:

$$\mathbb{P}(x_{i,q} = v | x_{i,q_g} = C) = \frac{|\{x_i \in X_{Tr} | x_{i,q} = v \wedge x_{i,q_g} = C\}|}{|\{x_i \in X_{Tr} | x_{i,q_g} = C\}|} \quad (4.54)$$

Wenn q quantitativ ist, wird Normalverteilung *unterstellt*:

$$\mathbb{P}(x_{i,q} = v | x_{i,q_g} = C) = \frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{(v-\mu)^2}{2\sigma^2}} \quad (4.55)$$

Es fehlen noch μ und σ , welche auch geschätzt werden (vgl. (3.26))

$$\mu_{q,C} = \frac{1}{|\{x_i \in X_{Tr} | x_{i,q_g} = C\}|} \sum_{x_i \in X_{Tr} | x_{i,q_g} = C} x_{i,q} \quad (4.56)$$

$$\sigma_{q,C}^2 = \frac{1}{|\{x_i \in X_{Tr} | x_{i,q_g} = C\}|} \sum_{x_i \in X_{Tr} | x_{i,q_g} = C} (x_{i,q} - \mu_{q,C})^2 \quad (4.57)$$

All diese Schätzungen eingesetzt in den Bayes Klassifikator ergeben den *naïven* Bayes Klassifikator (Han & Kamber, 2006, S. 310–313).

Der *k*-Nearest Neighbour Algorithmus ist schnell erklärt, da er als lazy learner (vgl. Tab. 15) kein Modell aufbaut. Im Grunde ist schon der Name Erklärung genug: Weise einer Instanz x_i diejenige Klasse zu, welche unter ihren k nächsten Nachbarn in X_{Tr} in der Mehrheit ist. Die Nachbarschaftsbeziehung wird dabei durch ein Distanzmaß (vgl. Abschnitt 4.5) bestimmt. In der Praxis besteht die eigentliche Schwierigkeit darin, X_{Tr} so abzuspeichern, dass die nächsten Nachbarn einer neuen Instanz effizient gefunden werden können (Runkler, 2010, S. 96).

| *k*-Nearest Neighbour |

5 Phase III: Nachbereitende Schritte

Rolle im KDD-Prozess | Nachdem die Daten in der ersten Phase (vgl. Kapitel 3) ausgewählt und vorbereitet wurden und in der zweiten Phase (vgl. Kapitel 4) ein DM-Algorithmus ausgewählt und angewendet wurde, erfolgt in der dritten Phase die Nachbereitung. Diese Phase umfasst die letzten Schritt zur Generierung von Wissen aus Daten im Sinne von KDD (vgl. Tab. 2). Hierzu müssen die Ergebnisse evaluiert und interpretiert werden, sodass sie in die Lösung des zugrundeliegenden Entscheidungsproblems einfließen können.

5.1 Evaluation des Modells und der Ergebnisse

Kriterien zur Beurteilung | Es ist wichtig, zunächst das entwickelte Modell zu evaluieren, bevor dieses dann auf neue Daten bzw. das relevante Entscheidungsproblem angewandt wird. Nachdem aufgrund der Trainingsdaten die Entscheidung für ein bestimmtes Modell $h \in H$ gefallen ist, erfolgt die Evaluation des Modells, indem es auf die zuvor reservierten Testdaten angewendet wird (vgl. S. 44). Zur Evaluation der mit den Testdaten erzielten Ergebnisse und somit gleichzeitig des entwickelten Modells, gibt es keine für alle DM-Methoden allgemeingültigen Kennzahlen und Methoden. Daher werden im Folgenden zunächst Kriterien zur Bewertung von DM-Ergebnissen beispielhaft vorgestellt, bevor im Anschluss daran eine Auswahl verschiedener Kennzahlen beschrieben wird. Diese Kennzahlen lassen sich jedoch nicht unabhängig von der Interpretation der Daten bewerten.

Validität | Ein wesentlicher Aspekt ist die Validität der Ergebnisse. Die Validität eines Modells betrifft die Frage, wie gut das Modell, bzw. die Annahmen, die dem Modell zugrunde liegen, die Realität wiedergeben (Fonseca & Fieller, 2006). DM bzw. ML unterscheiden sich hierbei von der klassischen Statistik und herkömmlichen Optimierungsverfahren darin, dass nicht die Anpassung des Modells an vorhandene Daten das oberste Ziel ist, sondern die Generalisierung auf bislang unbekannte Instanzen (Domingos, 2012). Es muss also sorgfältig zwischen der Abbildungsgüte bezüglich X und der Abbildungsgüte bezüglich U unterschieden werden (vgl. Abb. 6 und S. 43). Die Evaluation des Modells mit unabhängigen Testdaten kann hier ein Weg sein, ein Stück unabhängige Realität zu simulieren. Nicht nur Methoden des überwachten Lernens neigen dazu, dass die gewonnenen Ergebnisse auf die verwendeten Trainingsdaten „zugeschnitten“ sind (Overfitting, vgl. S. 45). Es kann z. B. auch sein, dass ein mit Rule Induction ermittelter Cross-Selling-Effekt lediglich auf die verwendeten Trainingsdaten zutrifft, jedoch darüber hinaus nicht erkennbar ist. Abhilfe kann hier einerseits die sorgfältige Auswahl und Vorbereitung der Trainingsdaten schaffen (vgl. S. 15) und andererseits eine Plausibilitätsprüfung mit Testdaten. Nur wenn die Validität des Modells gewährleistet werden kann, können daraus allgemeine Handlungsempfehlungen und Veränderungen abgeleitet werden. Andernfalls würden die Empfehlungen nur in einzelnen Fällen, z. B. für eine bestimmte Kundengruppe, gültig sein.

Neuigkeitsgehalt und Verständlichkeit | Ein weiteres Kriterium ist gemäß der KDD-Definition (vgl. Abschnitt 2.1) der Neuigkeitsgehalt der gewonnenen Ergebnisse. Die verschiedenen DM-Methoden generieren unterschiedlich umfangreiche Ergebnisse. Insbesondere bei explorativen Methoden wie der Assoziationsanalyse werden sehr viele Regeln gefunden, die jedoch in der ganzen Fülle zum einen nicht mehr überschaubar sind und zum anderen teilweise keinen Neuigkeitsgehalt

haben. So würde die Regel „wenn Butter, dann keine Margarine“ aufgrund der Tatsache, dass es sich bei diesen Produkten um Substitute handelt, keinen wirklichen Neuigkeitsgehalt aufweisen. Dieser Fall tritt besonders häufig auf, wenn es sich um zyklische Auswertungen von Daten handelt. Beispielsweise könnten Unternehmen quartalsweise eine Assoziationsanalyse durchführen um saisonabhängige Assoziationsregeln zu entdecken. Viele der durch diese Analyse gewonnenen Ergebnisse sind jedoch saisonunabhängig und weisen folglich keinen Neuigkeitsgehalt auf, sondern wurden bereits in vorherigen Analysen entdeckt. Das Kriterium der Kompaktheit überschneidet sich teilweise mit dem Neuigkeitsgehalt. Informationen sollen dem Entscheider möglichst kompakt zur Verfügung gestellt werden. Insbesondere Entscheidungsträger im oberen Managementbereich haben häufig nicht die Zeit, sich zunächst in umfangreiche Ergebnisse einzulesen, um danach Schlüsse und letztlich wertvolle Informationen ziehen zu können. Vielmehr müssen die wirklich wichtigen Ergebnisse in kompakter Weise dargestellt werden können. Dies wiederum tangiert die Verständlichkeit der Informationen. Informationen müssen dem Entscheider in verständlicher Weise vorgelegt werden. Dabei müssen natürlich die Kenntnisse und das Wissen des Entscheiders berücksichtigt werden⁷ (Hippner & Wilde, 2001).

Neben den Kriterien der Validität sowie dem Neuigkeitsgehalt und der Verständlichkeit ist die Nützlichkeit ein weiteres wichtiges Kriterium. Die gewonnenen Informationen sind genau dann nützlich, wenn aus den Informationen Handlungen abgeleitet werden können. Wenn beispielsweise eine Assoziationsanalyse ergeben hat, dass an einem Freitag häufig Chips und Bier zusammen verkauft werden, so können daraus zunächst keine Handlungsempfehlungen abgeleitet werden. Vielmehr müssen darauf basierend weitere Analysen erfolgen, um abschließend z. B. die Produkte im Laden möglichst nah beieinander zu positionieren oder aber möglichst weit voneinander entfernt zu positionieren, um so den Kunden zunächst an anderen Produkten vorbeizuleiten und so einen möglichen Umsatzanstieg zu erzeugen. Offensichtlich ist dies in hohem Maße Sache der Interpretation.

Nützlichkeit

Fragestellungen des überwachten Lernens haben den Vorteil, dass direkt gemessen werden kann, wie gut das Modell die Daten abbildet. Im Training erfolgt diese Messung mit Trainingsdaten, z. B. im Rahmen einer Kreuzvalidierung. Durch die spätere erneute Messung mit *unabhängigen* Testdaten, kann eingeschätzt werden, wie gut das Modell auf unbekannte Daten generalisiert. Sei wieder $Y = \{-1; +1\}$. Je nach Kombination von tatsächlichem x_{i,q_g} und vorhergesagtem $h(x_{i,q_f})$ ergeben sich vier Fälle, die in einer sogenannten *Konfusionsmatrix* dargestellt werden (Tab. 21).

Beurteilung der Klassifikationsgüte

Tab. 21: Konfusionsmatrix

	$x_{i,q_g} = -1$	$x_{i,q_g} = +1$
$h(x_{i,q_f}) = -1$	richtig negativ (<i>rn</i>)	falsch negativ (<i>fn</i>)
$h(x_{i,q_f}) = +1$	falsch positiv (<i>fp</i>)	richtig positiv (<i>rp</i>)

Zwei der Fälle bezeichnen eine Übereinstimmung zwischen Vorhersage und tatsächlichem Wert und daher eine korrekte Klassifikation. Die anderen beiden Fälle bezeichnen fehlende Übereinstimmung und daher falsche Klassifikation. Die Differenzierung ist dennoch sinnvoll, da für viele

⁷ vgl. auch Einheit 1.

Anwendungen eine Klasse mehr von Interesse ist als die andere. In der medizinischen Diagnostik etwa ist besonders von Interesse, ob der Patient die Krankheit auch wirklich hat. Daher wird das Vorliegen der Krankheit als der „positive“ Fall definiert. Gerade bei seltenen Krankheiten wird mehr Wert darauf gelegt, die Krankheit auch wirklich zuverlässig zu erkennen, also die falsch negativen Fälle zu minimieren, als darauf die Anzahl der Fehlalarme, also der falsch positiven Fälle zu minimieren. Dies führt zuweilen zu unintuitiven Ergebnissen.

Kennzahlen von Klassifikatoren

men:

Durch Eintragen der jeweiligen Anzahl von Testinstanzen in die Konfusionsmatrix lässt sich eine Reihe von Kennzahlen für Klassifikationsmodelle bestimmen:

$$\frac{rp + rn}{|X_{Te}|} \quad (\text{Korrektklassifikationsrate}) \quad (5.1)$$

$$\frac{fp + fn}{|X_{Te}|} \quad (\text{Falschklassifikationsrate}) \quad (5.2)$$

$$\frac{rp}{rp + fn} \quad (\text{Recall}) \quad (5.3)$$

$$\frac{fp}{fp + rn} \quad (\text{Falschpositivrate}) \quad (5.4)$$

$$\frac{rp}{rp + fp} \quad (\text{Precision}) \quad (5.5)$$

Wenn beide Klassen gleich wichtig sind, ist die Korrektklassifikationsrate in der Regel die maßgebliche Kennzahl. Wenn jedoch die positive Klasse von größerer Bedeutung ist, ist oft die Richtigpositivrate, auch Recall genannt, entscheidend. Leider hat der Klassifikator, der alle Instanzen der positiven Klasse zuordnet offensichtlich ein Recall von 1, obwohl es sich um einen extrem schlechten Klassifikator handelt. Deshalb ist es zum Vergleich verschiedener Klassifikationsmodelle oder verschiedener Modellparameter sinnvoll, jeweils Recall und Falschpositivrate oder Recall und Precision gemeinsam in einem gemeinsamen Diagramm abzutragen um so einen objektiveren Vergleich zu ermöglichen (Runkler, 2010, S. 86–89).

5.2 Interpretation und Verwendung der Ergebnisse

Einordnung in den Kontext

Das Ziel des KDD-Prozesses ist nicht weniger als die Generierung von *Wissen*. Wie in Einheit 1 dieses Lehrbriefs umfassend dargelegt, ist dies nur unter Berücksichtigung eines gegebenen Kontexts möglich, der für die BI mindestens aus dem Unternehmen, dessen Umfeld, sowie den Entscheidern und Applikationen im Unternehmen besteht. Aussagen zur Interpretation und Verwendung der DM-Ergebnisse zu treffen, die immer und für alle Kontexte gelten ist unmöglich. Vielmehr wird in diesem Lehrbrief die Auffassung vertreten, dass der Kontext stets in allen Phasen des KDD-Prozesses mitgedacht werden muss. Deshalb wurde auch schon zu Beginn, in Abschnitt 2.3 dargestellt, wie sich KDD in den Unternehmenskontext und insbesondere in den Managementkreislauf einordnen lässt.

Ableitung von Handlungsempfehlungen

Nützlich, so wurde im vorherigen Abschnitt festgehalten, ist ein DM-Ergebnis dann, wenn sich daraus Handlungsempfehlungen herleiten lassen. Wenn das zugrundeliegende Entscheidungsproblem (vgl. Abschnitt 3.1) während des gesamten KDD-Prozesses im Blick gehalten wird, dann wird das Ergebnis fast zwangsläufig zum besseren Verständnis des Entscheidungsproblems führen. Gegebenenfalls muss der Entscheider

den Bezug zwischen Ergebnis und den vorliegenden Handlungsalternativen noch unter Berücksichtigung seines Vorwissens herstellen, doch ein solcher Bezug wird immer vorhanden sein, wenn kein gravierender Fehler im KDD-Prozess gemacht wurde. Die Ergebnisverwendung besteht dann in der Umsetzung der jeweiligen Handlungsalternative oder deren Übernahme in die Prozesse des Unternehmens.

Die Interpretation eines Ergebnisses wird häufig mit seiner Aufbereitung für einen Entscheider oder eine Entscheidungssituation gleichgesetzt. Da aber der

Aufbereitung der Ergebnisse

Kontext und insbesondere Vorwissen und Informationsbedarfe von Entscheidern variieren, ist es nicht ausreichend sich auf eine Standardaufbereitung zu verlassen. Eine fundierte Kenntnis dessen, was die einzelnen DM-Algorithmen eigentlich tun (Abschnitte 4.3–4.8) liefert die Rohinformation, die in eine individuelle Aufbereitung fließen kann. Viele Verfahren bringen dabei eine übliche Visualisierung mit, auf die zurückgegriffen werden kann. Auch muss oft zwischen Güte und Verständlichkeit von Modellen abgewogen werden.

Zur Interpretation eines DM-Modells zählt nicht nur das Verständnis der generierten Hypothese, sondern auch das Wissen darüber, welche Hypothesen das

Grenzen des DM

betroffene Verfahren überhaupt bilden kann und welche nicht (Tab. 11–Tab. 15). Darüber hinaus haben DM und ML theoretische prinzipielle Grenzen, die kein Verfahren durchbrechen kann (vgl. Abschnitt 4.2). Diese Grenzen zu kennen und zu berücksichtigen macht einen wesentlichen Teil der korrekten Interpretation des Ergebnisses aus. Außerdem gelten die theoretischen Zusagen der einzelnen Algorithmen nur unter klar umrissenen Bedingungen, z. B. bezüglich der Skalierung und Verteilung der Daten (vgl. Abschnitt 3.4).

Jedes DM-Ergebnis ist zunächst einmal eine Aussage über die Daten. Nur durch Kenntnis des Zusammenhangs zwischen Daten, Modell und Realität (vgl. Ab-

Bedeutung des Modells

schnitt 3.2–3.3) kann eine Aussage über die Realität getroffen werden und echtes Wissen entstehen. Damit schließt sich der Kreis zum Beginn des KDD-Prozesses. Vordergründig erfüllt ein DM-Modell die ihm zugedachte Aufgabe (vgl. Abschnitt 4.1) und der Entscheider hat zu beurteilen wie gut sie erfüllt wird. Die größte interpretatorische Herausforderung ist es jedoch, zu verstehen, was es *bedeutet*, dass ein bestimmtes Modell zur Erklärung der Daten aussieht wie es aussieht. Während der Statistiker erst eine Theorie bildet und davon ausgehend ein Modell erzeugt, ist der Datenanalyst hier gefragt im Nachhinein eine Erklärungshypothese zu liefern. Diese Aufgabe kann ihm kein Algorithmus abnehmen.

6 Zusammenfassung

Die vorliegende Einheit 2 hat gezeigt, dass Unternehmen nur dann einen Wettbewerbsvorteil generieren können, wenn sie aus ihren großen Datenmengen, die in den verschiedenen Applikationen zur Verfügung stehen, strukturiert Informationen und schließlich auch Wissen generieren. Um dies zu erreichen, ist es für Unternehmen sinnvoll, anhand des KDD-Prozesses vorzugehen. Dieser Prozess legt dabei ein ganzheitliches Begriffsverständnis von BI zu Grunde, da er nicht nur die technische Sicht berücksichtigt, sondern auch problemorientierte Komponenten. In der ersten Phase werden die vorbereitenden Schritte zur eigentlichen Analyse der Daten durchgeführt. Nur wenn dem Entscheider klar ist, welche Zielsetzung eigentlich mit der Analyse verfolgt wird, kann dies zum Erfolg führen. Ebenso müssen die Analysedaten selektiert und aufbereitet werden, damit die eigentliche Analyse in der zweiten Phase mit einem überschaubaren finanziellen und zeitlichen Aufwand einhergehen kann. Das eigentliche DM in der zweiten Phase erfordert ebenfalls IKT-Unterstützung durch die Anwendung eines DM-Werkzeugs, welche bekannte DM-Algorithmen implementiert. Die Auswahl eines geeigneten Algorithmus sollte auf der aus der ersten Phase abgeleiteten Grundaufgabe basieren, um ein gutes Ergebnis sicherzustellen. Schließlich werden in der dritten und letzten Phase nachbereitende Schritte durchgeführt. So sollte das Modell sowie die Ergebnisse kritisch betrachtet und evaluiert werden. Hierzu gehört beispielsweise der Rückbezug zur Zielsetzung. Der Entscheider muss sich die Frage stellen, ob die Zielsetzung mit den gewonnenen Ergebnissen tatsächlich erreicht wurde oder ob sich die Ergebnisse z. B. neue Fragestellungen ergeben haben. Wenn die gewonnenen Ergebnisse zu neuem Wissen geführt haben, sollten die neuen Erkenntnisse nach Möglichkeit in die bestehenden Prozesse integriert werden, um so eine langfristige Verbesserung der Geschäftsprozesse und somit der Wettbewerbsfähigkeit eines Unternehmens zu erzielen.

Die vorliegende Einheit verdeutlicht aber auch die Bedeutung der Qualität der Daten sowie deren Bereitstellung. Um eine möglichst effektive Datenhaltung in einem Unternehmen sicherzustellen, setzen viele Unternehmen ein DWH ein. Sowohl das Datenqualitätsmanagement, als auch die Besonderheiten, Vor- und Nachteile von DWHs sind Inhalt der Einheit 3 dieses Lehrbriefs.

7 Übungsaufgaben

1. Grenzen Sie die Begriffe KDD und DM voneinander ab!
2. Erläutern Sie die folgenden Eigenschaften des KDD-Prozesses: nicht-trivial, valide, potentiell nützlich und leicht verständlich!
3. Erläutern Sie anhand des Beispiels der Entscheidung über die von einem Unternehmen angebotene Produktpalette, wie sich der KDD-Prozess in den Managementkreislauf einbettet!
4. Erläutern Sie das Prinzip des „Garbage in – Garbage out“!
5. Bei einer Überweisung gibt ein Kunde noch Kontonummer und Bankleitzahl statt der IBAN an. Um welche Fehlerart handelt es sich? Welches Vorgehen würden Sie der Bank bei diesem Fehler raten? Erläutern Sie in diesem Zusammenhang die verschiedenen Umgangsmöglichkeiten mit Fehlern!
6. Führen Sie auf Basis des folgenden Konzepts eine Transformation des Attributs Einkommen durch! Die Eintragungen sollen auf das Intervall $[0,1]$ normalisiert werden.

Nr.	Geschlecht	Alter	Einkommen
1	weiblich	32	62.000
2	weiblich	37	31.500
3	männlich	23	45.000
4	weiblich	45	42.000
5	männlich	58	37.000
6	männlich	25	75.000
7	männlich	61	22.000
8	weiblich	41	33.500
9	weiblich	49	49.000
10	männlich	34	57.000

7. Grenzen Sie die beiden Clusteringverfahren agglomerativ und divisiv voneinander ab!
8. Erläutern Sie die Grundidee der Clusteranalyse in einfachen Worten!
9. Erläutern Sie die Begriffe „falsch-positiv“ und „falsch-negativ“ am Beispiel der Klassifizierung von Bankkunden, bei der die Kunden in die beiden Klassen „kreditwürdig“ und „nicht kreditwürdig“ eingeteilt werden sollen!

10. Folgende Ausgangsdistanzmatrix ist gegeben. Geben Sie jeweils die Distanzmatrix nach den ersten drei Iterationen des complete-linkage Clustering an!

	A	B	C	D	E
A					
B	10				
C	5	7			
D	8	9	5		
E	5	8	4	7	

Literaturverzeichnis

- Alkharusi, H. (2012). Categorical variables in regression analysis: A comparison of dummy and effect coding. *International Journal of Education*, 4(2).
- Apel, D., Brehme, W., Eberlein, R. & Merighi, C. (2010). *Datenqualität erfolgreich steuern* (2., vollständig überarbeitete Aufl.). München, Wien: Hanser.
- Backhaus, K., Erichson, B., Plinke, W. & Weiber, R. (2006). *Multivariate Analysemethoden* (11. überarbeitete Aufl.). Berlin, Heidelberg: Springer.
- Bamberg, G., Coenenberg, A. G. & Krapp, M. (2008). *Betriebswirtschaftliche Entscheidungslehre* (14., überarbeitete Aufl.). München: Franz Vahlen.
- Bauer, A. & Günzel, H. (2001). *Data Warehouse Systeme – Architektur, Entwicklung, Anwendung*. Heidelberg: dpunkt.verlag.
- Bishop, C. M. (2006). *Pattern recognition and machine learning*. New York: Springer US.
- Bollinger, T. (1996). Assoziationsregeln – Analyse eines Data Mining Verfahrens. *Informatik-Spektrum*, 19(5), 257–261.
- Bousquet, O., Boucheron, S. & Lugosi, G. (2004). Introduction to statistical learning theory. In O. Bousquet, U. von Luxburg & G. Rätsch (Hrsg.), *Advanced lectures on machine learning* (S. 169–207). Springer.
- Brachman, R. J. & Anand, T. (1996). The process of knowledge discovery in databases. In U. M. Fayyad, G. Piatetsky-Shapiro, P. Smyth & R. Uthurusamy (Hrsg.), *Advances in knowledge discovery and data mining* (S. 37–57). Menlo Park, CA, USA: American Association for Artificial Intelligence.
- Bramer, M. (2007). *Principles of data mining*. London: Springer.
- Breiman, L., Friedman, J. H., Olshen, R. A. & Stone, C. Chattamvelli, R. (2009). *Data mining methods*. Oxford, UK: Alpha Science International Ltd.
- Cios, K. J., Pedrycz, W. & Swiniarski, R. W. (1998). *Data mining methods for knowledge discovery*. Boston, Dordrecht, London: Kluwer.
- Crone, S. F., Lessmann, S. & Stahlbock, R. (2006). The impact of preprocessing on data mining: An evaluation of classifier sensitivity in direct marketing. *European Journal of Operational Research* (173), 781–800.
- Deming, W. E. (1982). *Out of the crisis*. Cambridge, MA: MIT Press.
- Domingos, P. (1999). The role of occam's razor in knowledge discovery. *Data Mining and Knowledge Discovery*, 3(4), 409–425.
- Domingos, P. (2012). A few useful things to know about machine learning. *Commun. ACM*, 55(10), 78–87.

- Düsing, R. (2006). Knowledge Discovery in Databases – Begriff, Forschungsgebiet, Prozess und System. In P. Chamoni & P. Gluchowski (Hrsg.), *Analytische Informationssysteme* (S. 241–262). Berlin, Heidelberg: Springer.
- Eckstein, P. P. (2014). Regressionsanalyse. In *Statistik für Wirtschaftswissenschaftler* (S. 315–346). Wiesbaden: Springer.
- Eisenführ, F. & Weber, M. (2003). *Rationales Entscheiden* (4., neu bearbeitete Aufl.). Berlin, Heidelberg: Springer.
- Elmasri, R. & Navathe, S. B. (2002). *Grundlagen von Datenbanksystemen* (3., überarbeitete Aufl.). München: Pearson Studium.
- Ester, M. & Sander, J. (2000). *Knowledge discovery in databases: Techniken und Anwendungen*. Berlin, Heidelberg: Springer.
- Fayyad, U. M., Piatetsky-Shapiro, G. & Smyth, P. (1996). From data mining to knowledge discovery: an overview. In U. M. Fayyad, G. Piatetsky-Shapiro, P. Smyth & R. Uthurusamy (Hrsg.), *Advances in knowledge discovery and data mining* (S. 1–34). Menlo Park, CA, USA: American Association for Artificial Intelligence.
- Felici, G. & Weitschek, E. (2012, 9–11.). Mining logic models in the presence of noisy data. In *International symposium on artificial intelligence and mathematics (ISAIM 2012)*. Fort Lauderdale, Florida.
- Fonseca, V. & Fieller, N. (2006). Distortion in statistical inference: the distinction between data contamination and model deviation. *Metrika*, 63, 169–190.
- Frawley, W. J., Piatetsky-Shapiro, G. & Matheus, C. J. (1992). Knowledge discovery in databases: an overview. *AI Magazine*, 13(3), 57–70.
- Freitas, A. A. (2002). *Data mining and knowledge discovery with evolutionary algorithms*. Berlin, u. a.: Springer.
- Giuffrida, G., Chu, W. W. & Hanssens, D. M. (2000). Mining classification rules from datasets with large number of many-valued attributes. In C. Zaniolo, P. C. Lockemann, M. H. Scholl & T. Grust (Hrsg.), *Advances in database technology - EDBT 2000* (S. 335–349). Berlin, Heidelberg: Springer.
- Grabmeier, J. (2001). Segmentierende und clusterbildene Methoden. In H. Hippner, U. Küsters, M. Meyer & K. Wilde (Hrsg.), *Handbuch Data Mining im Marketing – Knowledge discovery in marketing databases* (S. 299–362). Wiesbaden: Vieweg Gabler.
- Grimmer, U. & Mucha, H.-J. (1998). Datensegmentierung mittels Clusteranalyse. In G. Nakhaeizadeh (Hrsg.), *Data Mining* (S. 109–141). Heidelberg: Physica.
- Guckelsberger, U. & Unger, F. (1999). *Statistik in der Betriebswirtschaftslehre*. Wiesbaden: Gabler.
- Hahn, D. (1996). *PuK – Controllingkonzepte* (5., überarbeitete und erweiterte Aufl.). Wiesbaden: Gabler.
- Han, J. & Kamber, M. (2006). *Data mining: Concepts and techniques* (2. Aufl.). San Francisco: Morgan Kaufmann.

- Hastie, T., Tibshirani, R. & Friedman, J. H. (2001). *The elements of statistical learning: data mining, inference, and prediction*. New York, u.a.: Springer.
- Heinen, E. (1992). *Einführung in die Betriebswirtschaftslehre* (9., verbesserte Aufl.). Wiesbaden: Gabler. (Originalausgabe 1985)
- Hilbert, M. & López, P. (2011). The world's technological capacity to store, communicate, and compute information. *Science*, 332 (6025), 60–65.
- Hippner, H. & Wilde, K. D. (2001). Der Prozess des Data Mining im Marketing. In H. Hippner, U. Küsters, M. Meyer & K. Wilde (Hrsg.), *Handbuch Data Mining im Marketing – Knowledge discovery in marketing databases* (S. 22–91). Wiesbaden: Vieweg Gabler.
- Hosking, J. R. M., Pednault, E. P. D. & Sudan, M. (1997). A statistical perspective on data mining. *Future Gener. Comput. Syst.*, 13 (2–3), 117–134.
- Jammerneegg, W., Luptáčík, M., Nakhaeizadeh, G. & Schnabl, A. (1998). Ist ein fairer Vergleich von Data Mining Algorithmen möglich? In G. Nakhaeizadeh (Hrsg.), *Data Mining* (S. 225–247). Heidelberg: Physica.
- Kamakura, W. A., Wedel, M., Rosa, F. de & Mazzon, J. A. (2003). Cross-selling through database marketing: a mixed data factor analyzer for data augmentation and prediction. *International Journal of Research in Marketing*, 20 (1), 45–65.
- Klein, D., Tran-Gia, P. & Hartmann, M. (2013). Big data. *Informatik-Spektrum*, 36 (3), 319–323.
- Kohavi, R. & John, G. H. (1997). Wrappers for feature subset selection. *Artif. Intell.*, 97 (1-2), 273–324.
- Krahl, D., Windheuser, U. & Zick, F.-K. (1998). *Data Mining: Einsatz in der Praxis*. Bonn: Addison-Wesley.
- Larose, D. T. (2005). *Discovering knowledge in data: An introduction to data mining*. Hoboken, NJ: Wiley & Sons.
- Laux, H., Gillenkirch, R. M. & Schenk-Mathes, H. Y. (2012). *Entscheidungstheorie*. Berlin, Heidelberg: Springer.
- Leopold, E. & Kindermann, J. (2006). Content classification of multimedia documents using partitions of low-level features. *JVRB - Journal of Virtual Reality and Broadcasting*, 3 (6).
- Linoff, G. S. & Berry, M. J. A. (2011). *Data mining techniques - for marketing, sales, and customer relationship management* (3. Aufl.). Indianapolis: Wiley.
- Lusti, M. (2002). *Data Warehousing und Data mining – Eine Einführung in entscheidungsunterstützende Systeme* (2. Aufl.). Berlin, Heidelberg: Springer.
- MacKay, D. J. C. (2003). *Information theory, inference, and learning algorithms*. Cambridge: Cambridge University Press.
- Magdalena, L. (2010). What is soft computing? Revisiting possible answers. *International Journal of Computational Intelligence Systems*, 3 (2), 148–159.

- McCarty, J. A. & Hastak, M. (2007). Segmentation approaches in data-mining: A comparison of RFM, CHAID, and logistic regression. *Journal of Business Research*, 60 (6), 656–662.
- Mertens, P. & Wieczorrek, H. W. (2000). *Data X Strategien – Data Warehouse, Data Mining und operationale Systeme für die Praxis*. Berlin, Heidelberg: Springer.
- Mitchell, T. M. (1997). *Machine learning* (1. Aufl.). New York, NY: McGraw-Hill.
- Müller, H. & Freytag, J.-C. (2003). *Problems, methods, and challenges in comprehensive data cleansing* (Arbeitsbericht Nr. HUB-IB-164). Berlin: Humboldt Universität.
- Nonaka, I. & Takeuchi, H. (1995). *The knowledge-creating company: How Japanese companies create the dynamics of innovation*. New York: Oxford University Press.
- Petersohn, H. (2005). *Data Mining – Verfahren, Prozesse, Anwendungsarchitektur*. München: Oldenbourg.
- Piatetsky-Shapiro, G. (1991). Knowledge discovery in real databases: A report on the IJCAI-89 workshop. *AI Magazine*, 11 (5), 68–70.
- Pietsch, T. & Memmler, T. (2003). *Balanced Scorecard erstellen: Kennzahlenermittlung mit Data Mining*. Berlin: Erich Schmidt Verlag.
- Pyle, D. (1999). *Data preparation for data mining*. San Francisco: Morgan Kaufmann.
- Quinlan, J. R. (1986). Induction of decision trees. *Machine Learning*, 1 (1), 81–106.
- Quinlan, J. R. (1993). *C4.5: Programs for machine learning*. San Francisco, CA: Morgan Kaufmann.
- Reichmann, T. (2011). *Controlling mit Kennzahlen: Die systemgestützte Controlling-Konzeption mit Analyse- und Reportinginstrumenten* (8., überarbeitete und erweiterte Aufl.). München: Franz Vahlen.
- Rokach, L. & Maimon, O. (2010). Decision trees. In O. Maimon & L. Rokach (Hrsg.), *Data mining and knowledge discovery handbook* (S. 165–192). New York: Springer US.
- Runkler, T. A. (2010). *Data mining*. Wiesbaden: Vieweg+Teubner.
- Shmueli, G., Patel, N. R. & Bruce, P. C. (2010). *Data mining for business intelligence: Concepts, techniques, and applications in Microsoft Office Excel[®] with XLMiner[®]* (2. Aufl.). Hoboken, NJ: Wiley.
- Suarez-Alvarez, M. M., Pham, D.-T., Prostov, M. Y. & Prostov, Y. I. (2012). Statistical approach to normalization of feature vectors and clustering of mixed datasets. *Proceedings of the Royal Society A: Mathematical, Physical and Engineering Science*.
- Sutton, C. D. (2005). Classification and regression trees, bagging, and boosting. In C. R. Rao, E. J. Wegman & J. L. Solka (Hrsg.), *Data mining and data visualization* (Bd. 24, S. 303–329). Elsevier.
- Tukey, J.W. (1977). *Exploratory data analysis*. Reading, MA: Addison-Wesley.
- Ville, B. de. (2006). *Decision trees for business intelligence and data mining: Using SAS[®] Enterprise Miner[™]*. Cary, NC: SAS Institute.

- Wiedmann, K.-P., Buckler, F. & Buxel, H. (2001). Data Mining – ein einführender Einblick. In K.-P. Wiedmann & F. Buckler (Hrsg.), *Neuronale Netze im Marketing-Management* (S. 15–33). Wiesbaden: Gabler.
- Wilde, K. D. (2001). Data Warehouse, OLAP und Data Mining im Marketing — Moderne Informationstechnologien im Zusammenspiel. In H. Hippner, U. Küsters, M. Meyer & K. Wilde (Hrsg.), *Handbuch Data Mining im Marketing – Knowledge discovery in marketing databases* (S. 1–19). Wiesbaden: Vieweg Gabler.
- Witten, I. H., Frank, E. & Hall, M. A. (2011). *Data mining: Practical machine learning tools and techniques* (3. Aufl.). San Francisco: Morgan Kaufmann.
- Wolpert, D. H. (1996). The lack of a priori distinctions between learning algorithms. *Neural Comput.*, 8(7), 1341–1390.
- Wu, X., Kumar, V., Ross Quinlan, J., Ghosh, J., Yang, Q., Motoda, H. et al. (2008). Top 10 algorithms in data mining. *Knowledge and Information Systems*, 14(1), 1–37.
- Yang, Y., Webb, G. I. & Wu, X. (2010). Discretization methods. In O. Maimon & L. Rokach (Hrsg.), *Data mining and knowledge discovery handbook* (S. 101–116). New York: Springer US.
- Zwirner, M. (2008). Datenbereinigung zielgerichtet eingesetzt zur permanenten Datenqualitätssteigerung. In K. Hildebrand, M. Gebauer, H. Hinrichs & M. Mielke (Hrsg.), *Daten- und Informationsqualität* (S. 102–122). Wiesbaden: Vieweg+Teubner.

Lösungen zu den Übungsaufgaben

1 . Grenzen Sie die Begriffe KDD und DM voneinander ab!

KDD umfasst den gesamten Prozess von der Definition der Problemstellung bis hin zur Generierung von Wissen. DM hingegen stellt lediglich einen Schritt im Rahmen des KDD-Prozesses dar. Erst nachdem die Daten vorausgewählt wurden und eine Aufbereitung sowie Transformation stattgefunden hat, kann DM angewendet werden. DM beinhaltet dann die eigentliche Suche nach interessanten, nicht-trivialen Mustern in den zuvor selektierten und transformierten Daten. Ohne die vorherigen Schritte wäre eine Anwendung von DM jedoch nur schwer möglich bzw. die gewonnenen Ergebnisse wären wenig aussagekräftig. Genauso wichtig wie die vorgelagerten Schritte sind die nachgelagerten Schritte im Rahmen des KDD-Prozesses. So müssen die durch die Anwendung von DM gewonnen Ergebnisse interpretiert und danach in die Prozesse des Unternehmens integriert werden. Nur wenn der gesamte KDD-Prozess durchlaufen wird, kann das Unternehmen auch durch die Anwendung von DM profitieren.

DM umfasst lediglich Methoden zur Analyse der Daten. Um jedoch gute Ergebnisse mit den DM-Methoden erzielen zu können, bedarf es vor- sowie nachgelagerter Schritte, die mit dem KDD-Prozess abgedeckt werden können. Nur wenn die Problemstellung erkannt wurde, können die Daten zielgerichtet selektiert und aufbereitet werden. Ohne diese Schritte wäre eine Analyse der Daten nicht zielführend und würde einem Unternehmen höchstens durch Zufall Nutzen bringen. Ebenso würde lediglich die Analyse der Daten ohne die entsprechend nachgelagerten Schritte keine Veränderungen hervorrufen. Denn nur, wenn die Ergebnisse auch in die Entscheidungen im Unternehmen einfließen, können langfristige Verbesserungen erzielt werden. Somit dienen alle Phasen des KDD-Prozesses der Entscheidungsunterstützung.

2 . Erläutern Sie die folgenden Eigenschaften des KDD-Prozesses: nichttrivial, valide, potentiell nützlich und leicht verständlich!

Nicht-trivial: Zusammenhänge sollten nicht einfach durch die Aggregation der einzelnen Daten ermittelt werden können, sondern es sollte ein mehr oder weniger komplexes Modell entwickelt werden, das tiefer gehende Zusammenhänge abbilden kann. Valide, potentiell nützlich, leicht verständlich: Diese Eigenschaften beziehen sich auf die Güte der gefundenen Zusammenhänge. Valide bedeutet, dass das Modell die Realität möglichst gut abbildet. Dabei sollten die Ergebnisse für den Anwender leicht verständlich und interpretierbar sein. Dies bedeutet, dass auch das Vorwissen des Entscheiders bei der Aufbereitung der Ergebnisse berücksichtigt werden muss. Potenziell nützlich bedeutet, dass die gewonnenen Ergebnisse sich in Handlungsempfehlungen umsetzen lassen, beispielsweise in eine Anpassung der Geschäftsprozesse welche dem Unternehmen Vorteile bringt.

3 . Erläutern Sie anhand des Beispiels der Entscheidung über die von einem Unternehmen angebotene Produktpalette, wie sich der KDD-Prozess in den Managementkreislauf einbettet!

Der Managementkreislauf besteht aus den Phasen Entscheidungsproblem, Entscheidungsvorbereitung, Entscheidung und Wirkungsanalyse. Der KDD-Prozess lässt sich vor allem der Entscheidungsvorbereitung und der Wirkungsanalyse zuordnen. Das Entscheidungsproblem könnte beispielsweise sein, ob die Produktpalette eines Unternehmens erweitert werden soll. Bevor die eigentliche Aktion, nämlich die angestrebte Erweiterung durchgeführt werden kann, sollten mögliche Alternativen betrachtet werden, beispielsweise, ob die Produktpalette flächendeckend erweitert wird oder ob die Produktpalette nur für einen bestimmten Zeitraum erweitert wird. In der Entscheidungsvorbereitung wird versucht zu prognostizieren, welche Auswirkungen die zuvor festgelegten Planungen und möglichen Aktionen auf das Unternehmen haben können. Bei der Wirkungsanalyse wird dann untersucht, welches die real eingetretene Wirkung war. Hierzu wird der KDD-Prozess durchlaufen, wobei die erste Phase, die Definition der Problemstellung bereits abgedeckt ist. Wichtig bei der Definition der Problemstellung ist aber, dass die Problemstellung klar und so definiert ist, dass diese tatsächlich im Rahmen der Anwendung des KDD-Prozesses gelöst werden kann. Bei der Entscheidung über die Veränderung der Produktpalette greifen die DM-Werkzeuge auf Daten, wie z. B. historische Verkaufsdaten oder auch allgemeine Produktdaten und Preise zu, die im DWH bereitgestellt werden. Die letzte Phase des KDD-Prozesses, die Anwendung überschneidet sich teilweise wiederum mit dem Managementkreislauf. Wenn der KDD-Prozess und insbesondere die Anwendung von DM beispielsweise aufzeigen, dass die Produktpalette nur für bestimmte Bundesländer erweitert werden sollte, weil sich das Kaufverhalten der Kunden dort stark von dem der restlichen Bundesländer unterscheidet, dann muss das Management diese Veränderung wiederum planen. Somit lässt sich auch die Schleife im Managementkreislauf erklären. Abschließend kann die Wirkungsanalyse zeigen, inwiefern sich die Veränderung der Produktpalette z. B. auf das Kaufverhalten der Kunden auswirkt oder ob es z. B. interessante Assoziationsregeln gibt.

4 . Erläutern Sie das Prinzip des „Garbage in – Garbage out“!

Ein Output kann nur so gut sein, wie der Input ist. Wenn also für die Anwendung von DM Daten mit einer eher schlechten Datenqualität oder irrelevante Daten verwendet werden, dann können die gewonnenen Ergebnisse nur so gut sein, wie es die Eingangsdaten es zulassen. Wenn also „Müll“ in das DM hineingegeben wird, kann auch nur „Müll“ ausgegeben werden. Dies verdeutlicht insbesondere nochmals die hohe Bedeutung der Datenqualität für die Anwendung von DM.

5 . Bei einer Überweisung gibt ein Kunde noch Kontonummer und Bankleitzahl statt der IBAN an. Um welche Fehlerart handelt es sich? Welches Vorgehen würden Sie der Bank bei diesem Fehler raten? Erläutern Sie in diesem Zusammenhang die verschiedenen Umgangsmöglichkeiten mit Fehlern!

Bei dem beschriebenen Fehler handelt es sich um im Grunde korrekte Werte, die aber nicht dem erwarteten Format entsprechen. Somit liegt ein syntaktischer Fehler vor. Die Bank hat drei Möglichkeiten, mit diesem Fehler umzugehen. Beim Laissez Faire Prinzip werden keine Maßnahmen zur Behebung des Fehlers vorgenommen. Beim reaktiven Vorgehen wird der Fehler sofort einmalig

behalten. Es wird davon ausgegangen, dass sich der Fehler nicht wiederholt. Beim proaktiven Vorgehen wird der Fehler bereinigt und Maßnahmen zur Vermeidung der zukünftigen Wiederholung des Fehlers durchgeführt.

Bei dem erläuterten Beispiel sollte die Bank das Laissez Faire Prinzip vermeiden. Nichts zu unternehmen führt dazu, dass die Überweisung nicht durchgeführt werden kann. Es ist davon auszugehen, dass dieser Fehler während der Zeit der Umstellung (zum Zeitpunkt der Aufgabenstellung gerade aktuell), häufiger auftritt.

Ein reaktives und kundenfreundliches Verhalten wäre es, die richtige IBAN manuell aus Kontonummer und BLZ zu konstruieren. Da jedoch davon auszugehen ist, dass dieser Fehler häufiger auftritt, würde das reaktive Vorgehen dauerhaft hohe Kosten verursachen. Folglich ist Bank zum proaktiven Vorgehen zu raten. Demnach sollte die Bank den Fehler in ihren Prozess aufnehmen, z. B. dadurch, dass bei Auftritt des Fehlers der Kunde eine Benachrichtigung erhält, dass Überweisungen im alten Format künftig nicht mehr durchgeführt werden.

6. Führen Sie auf Basis des folgenden Konzepts eine Transformation des Attributs Einkommen durch! Die Eintragungen sollen auf das Intervall $[0,1]$ normalisiert werden.

Nr.	Geschlecht	Alter	Einkommen
1	weiblich	32	62.000
2	weiblich	37	31.500
3	männlich	23	45.000
4	weiblich	45	42.000
5	männlich	58	37.000
6	männlich	25	75.000
7	männlich	61	22.000
8	weiblich	41	33.500
9	weiblich	49	49.000
10	männlich	34	57.000

Der maximale Wert der Eintragungen beträgt 75.000, der minimale Wert liegt bei 22.000. Diese beiden Werte entsprechen v_{max} und v_{min} . Mit Hilfe der Formel $v' = \frac{v - v_{min}}{v_{max} - v_{min}}$ können nun die normalisierten Werte berechnet werden. Somit ergibt sich folgende Tabelle:

Nr.	Geschlecht	Alter	Einkommen
1	weiblich	32	0,75
2	weiblich	37	0,18
3	männlich	23	0,43
4	weiblich	45	0,38
5	männlich	58	0,28
6	männlich	25	1
7	männlich	61	0
8	weiblich	41	0,22
9	weiblich	49	0,51
10	männlich	34	0,66

7. Grenzen Sie die beiden Clusteringverfahren agglomerativ und divisiv voneinander ab!

Die beiden Clustering-Verfahren agglomerativ und divisiv sind beides hierarchische Verfahren. Die Besonderheit an diesen Verfahren ist, dass in jedem Schritt zwei Cluster entweder zusammengefügt oder getrennt werden. Bei den divisiven Verfahren besteht zu Beginn genau ein einziges Cluster, welches alle Instanzen beinhaltet. Bei dem agglomerativen Verfahren entspricht zu Beginn die Anzahl der Cluster der Anzahl der Instanzen. Dabei werden schrittweise mehrere Datensätze zu einem Cluster zusammengefasst. Bei dem divisiven Verfahren wird das anfängliche Cluster schrittweise in mehrere Cluster zerteilt. Das divisive Verfahren ist beendet, wenn die Anzahl der Cluster der Anzahl der Instanzen entspricht. Das agglomerative Verfahren endet mit nur einem Cluster, in dem sich alle Instanzen befinden. Somit agieren die beiden Verfahren agglomerativ und divisiv, genau gegenläufig.

8 . Erläutern Sie die Grundidee der Clusteranalyse in einfachen Worten!

Die Idee der Clusteranalyse ist es, Instanzen so in Cluster einzuordnen, dass die Cluster in sich möglichst homogen, untereinander aber möglichst heterogen sind. Dies kann gemessen werden, indem Instanzen paarweise miteinander verglichen werden und die Ähnlichkeit zwischen diesen Instanzen bestimmt wird. Die Ähnlichkeit lässt sich mit Hilfe verschiedener Distanzfunktionen wie der Euklidischen Distanz und der Manhattan-Distanz bestimmen.

9 . Erläutern Sie die Begriffe „falsch-positiv“ und „falsch-negativ“ am Beispiel der Klassifizierung von Bankkunden, bei der die Kunden in die beiden Klassen „kreditwürdig“ und „nicht kreditwürdig“ eingeteilt werden sollen!

Bei der Klassifizierung von Bankkunden in die beiden Klassen „kreditwürdig“ und „nicht-kreditwürdig“ lässt sich argumentieren, dass die nicht-Kreditwürdigkeit das entscheidende Merkmal ist. Falsch-negativ würde hier bedeuten, dass nicht-kreditwürdige Kunden fälschlicherweise in die Klasse der kreditwürdigen Kunden eingeordnet werden. Dies kann für die Bank zu hohen Kosten führen, weil die Wahrscheinlichkeit, dass der Kunde den Kredit nicht zurückzahlen kann, hoch ist. Dagegen bedeutet falsch-positiv, dass ein kreditwürdiger Kunde fälschlicherweise als nicht-kreditwürdig eingestuft wird. Dies wäre für die Bank zwar unglücklich, jedoch sind die entstandenen Opportunitätskosten vermutlich vergleichsweise gering. Somit kann ein falsch-negativer Fehler am genannten Beispiel weitaus höhere Kosten verursachen als ein falsch-positiver Fehler.

10. Folgende Ausgangsdistanzmatrix ist gegeben. Geben Sie jeweils die Distanzmatrix nach den ersten drei Iterationen des complete-linkage Clustering an!

	A	B	C	D	E
A					
B	10				
C	5	7			
D	8	9	5		
E	5	8	4	7	

	A	B	C,E	D
A				
B	10			
C,E	5	8		
D	8	9	7	

	A,C,E	B	D
A,C,E			
B	10		
D	8	9	

	A,C,D,E	B
A,C,D,E		
B	10	

Diese Seite bleibt aus technischen Gründen frei!

S002664968
(04/26)

32711-02-S#1



Alle Rechte vorbehalten
© 2026 FernUniversität in Hagen
Fakultät für Wirtschaftswissenschaft